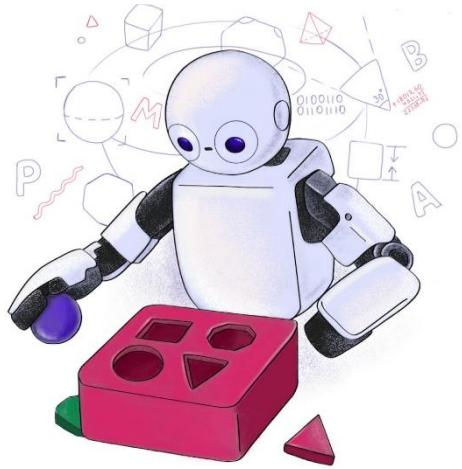
TP557 - Tópicos avançados em IoT e Machine Learning:

Datasets



Inatel

Felipe Augusto Pereira de Figueiredo
felipe.figueiredo@inatel.br

O que vamos ver?

- Anteriormente, aprendemos como criar ***classificadores***, em particular para classificação de imagens, ***utilizando redes neurais densas***.
- Após treinar o modelo, medimos sua ***acurácia*** e, após alguns testes básicos, verificamos que ***o modelo treinado reconhece as imagens muito bem***.
- Porém, essa ***análise simplista pode nos levar a uma falsa sensação de segurança***.
- Assim, neste tópico vamos ***explorar alguns problemas*** em torno desta análise superficial e ***aprender algumas abordagens*** que podemos utilizar para evitar erros ao treinarmos uma rede neural de forma ingênua.

Reconhecendo calçados













- Imaginem uma situação onde queremos treinar uma ***rede neural*** para ***reconhecer diferentes tipos de calçados***.
- É uma tarefa similar a ***ensinar alguém que nunca viu um calçado antes sobre o que eles realmente são*** para que no futuro quando essa pessoa ver um objeto destes, ela possa decidir se ele é um calçado ou não.

Passos a serem seguidos

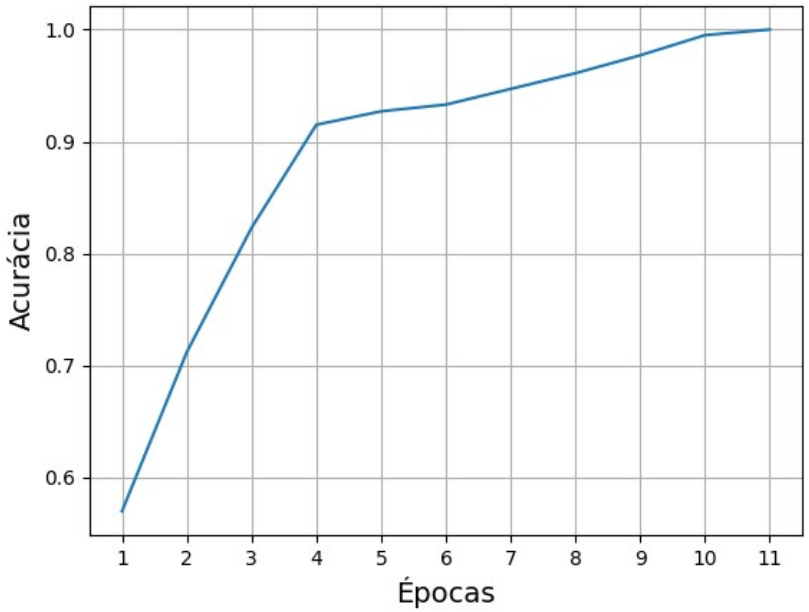


Quanto mais exemplos de calçados em nossa base de dados, melhor!

- Sabemos que ***há uma enorme variedade de calçados e não há uma regra rígida*** sobre o que faz de um calçado um calçado.
- Normalmente, seguindo o *workflow* do ML, nós:
  - coletaríamos o maior número possível de imagens de calçados,
  - encontraríamos a arquitetura mais apropriada de rede neural,
  - treinaríamos o modelo usando esse conjunto,
  - e usaríamos o modelo treinado (i.e., inferências).

Resultados do treinamento

Acurácia de treinamento: **0.570**
Acurácia de treinamento: **0.712**
Acurácia de treinamento: **0.823**
Acurácia de treinamento: **0.915**
Acurácia de treinamento: **0.927**
Acurácia de treinamento: **0.933**
Acurácia de treinamento: **0.947**
Acurácia de treinamento: **0.961**
Acurácia de treinamento: **0.977**
Acurácia de treinamento: **0.995**
Acurácia de treinamento: **1.000**



- Durante o treinamento, poderíamos observar resultados como os mostrados ao lado.
- O modelo atinge uma ***acurácia de 100% em apenas 11 épocas!***
- Isso pode significar que criamos um modelo incrível que pode reconhecer qualquer calçado.
- Então vamos usá-lo para realizar ***inferências com imagens inéditas de calçados!***

Hora de usar o modelo treinado



- Mas então mostramos um sapato como este ao lado e ***ele falha em reconhecê-lo como um calçado***.
- Pensamos que o modelo era 100% preciso em reconhecer calçados...

Hora de usar o modelo treinado



- Na realidade, o que temos é ***100% de acurácia no reconhecimento dos tipos de calçados nos quais treinamos a rede.***
- Essa acurácia de 100% nos levou a uma ***falsa sensação*** de segurança de que o ***modelo funcionaria muito bem com qualquer outra imagem***.

Acabamos de verificar que nosso modelo, inicialmente, perfeito não é tão perfeito assim...

O que podemos fazer?

Sobreajuste





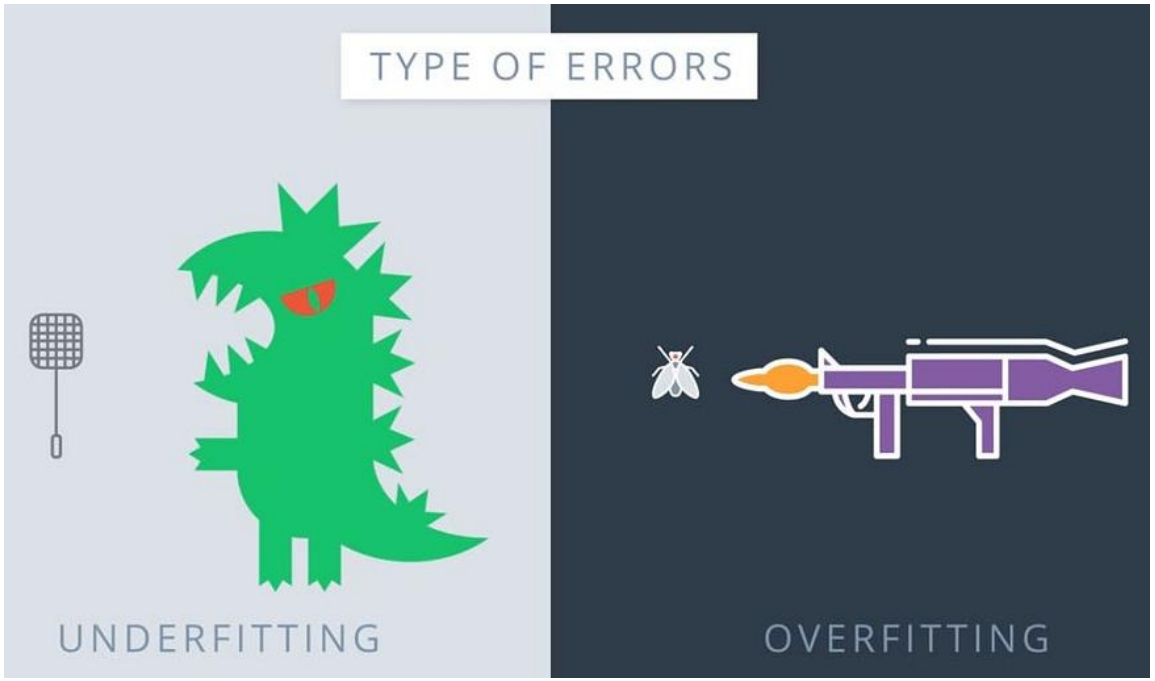
- O modelo falhou em atingir o ***objetivo final***, que era ***generalizar***.
- Muito provavelmente ele ficou demasiadamente bom para reconhecer calçados apenas no conjunto em que foi treinado.
  - Problema conhecido somo ***sobreajuste***.
- O ***modelo é provavelmente tão complexo que "memorizou" o conjunto de treinamento.***

Subajuste



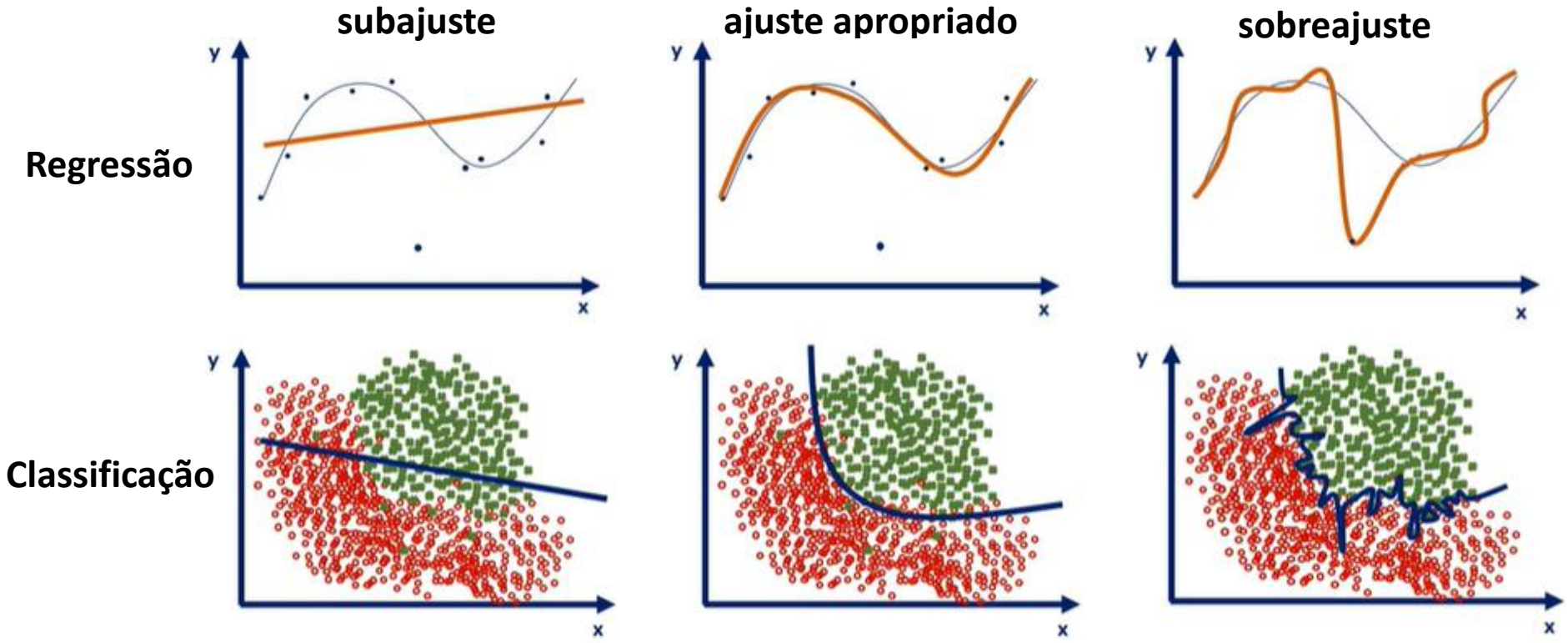
- Um outro problema que podemos encontrar, mas ***em uma escala muito menor***, é o extremo oposto do sobreajuste, chamado de ***subajuste*** (*underfitting*).
- Nesse caso, o ***modelo é muito simples para aprender*** a relação subjacente nos dados de treinamento, ou seja, ***o padrão geral por trás dos dados de treinamento***.

Como detectar tais problemas?



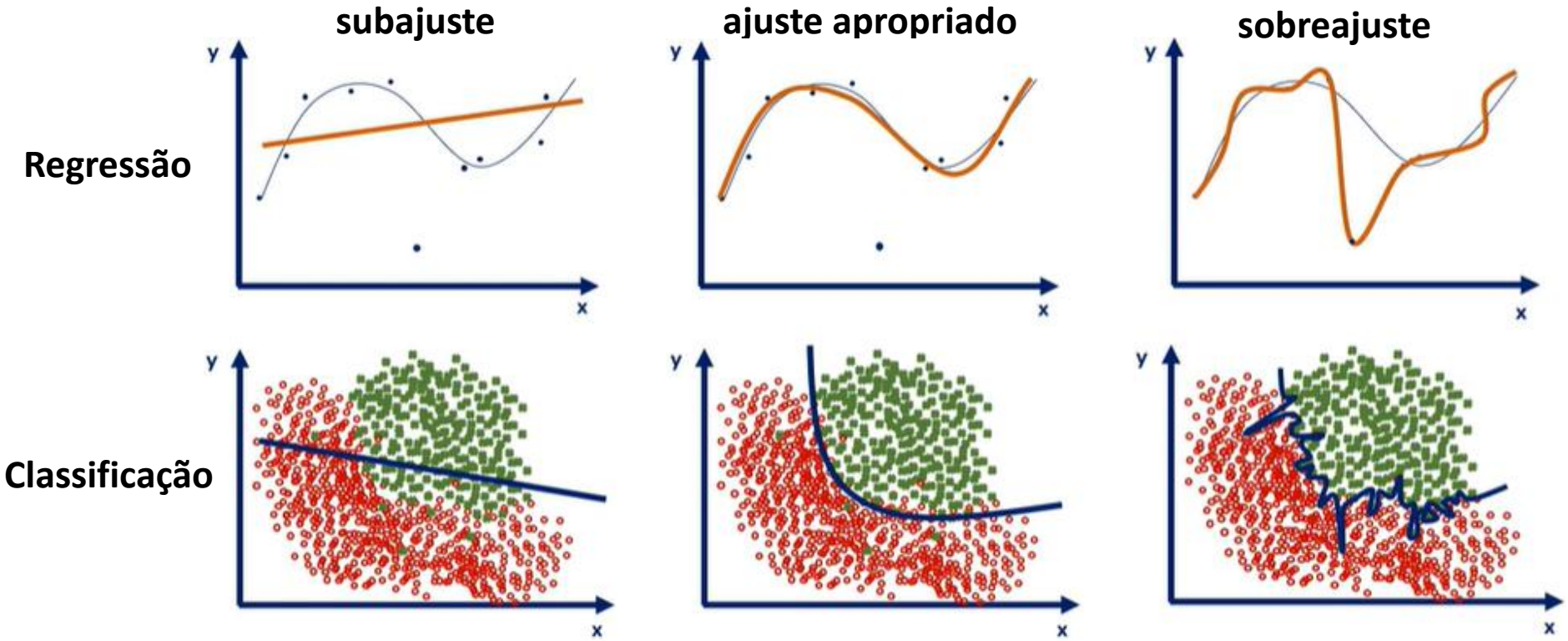
- Precisamos de ***uma forma para analisar e evitar que o modelo se subajuste ou sobreajuste*** aos dados do conjunto de treinamento.
- Para isso, ***dividimos o conjunto total de exemplos em subconjuntos***.

Subajuste e sobreajuste



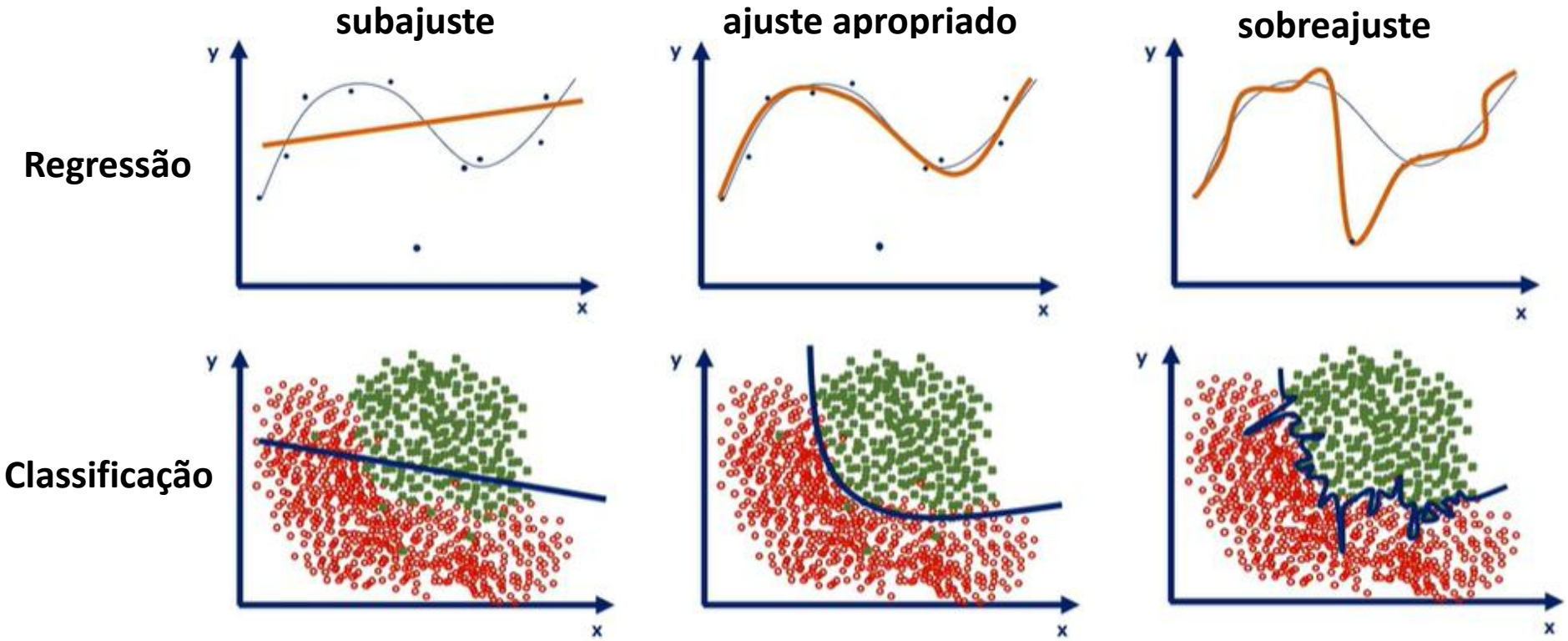
- Antes de falarmos sobre a divisão do conjunto total de dados, vamos falar um pouco mais sobre esses dois problemas que modelos de ML podem apresentar, o ***subajuste*** e o ***sobreajuste***.

Subajuste e sobreajuste



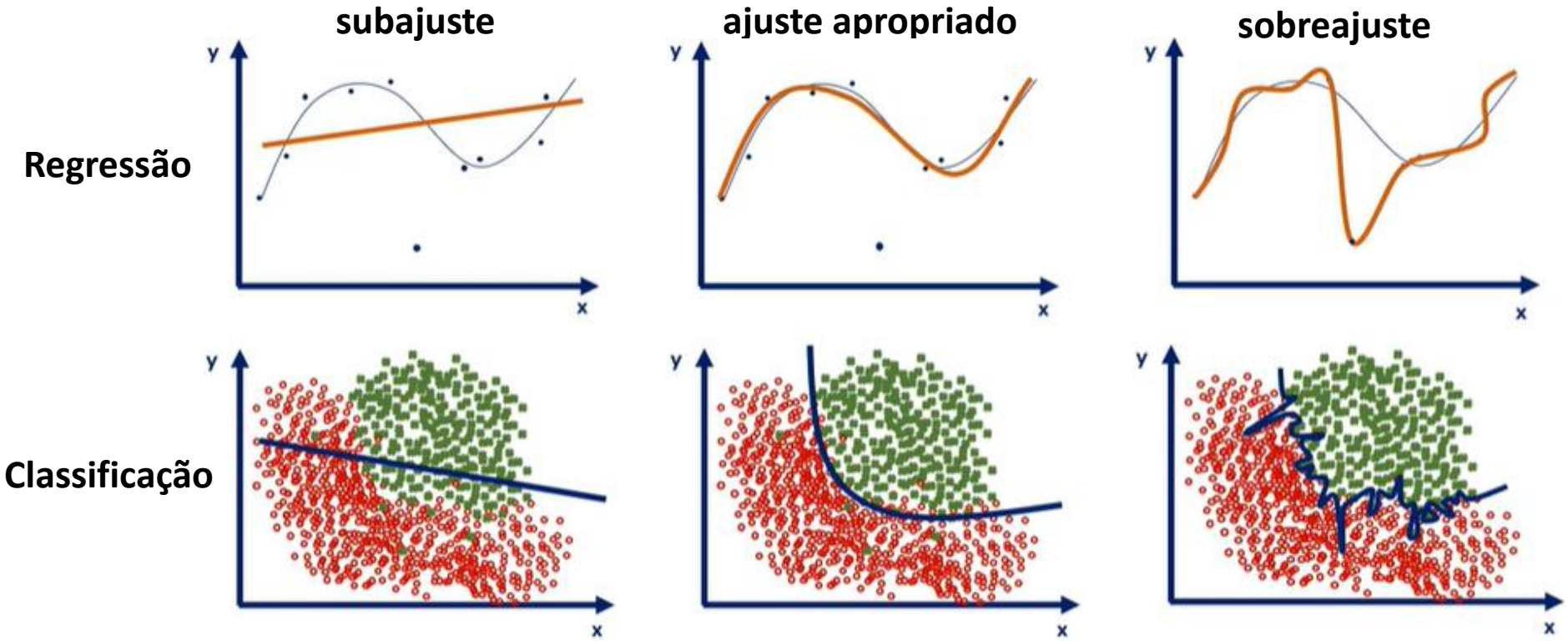
- O ***subajuste*** ocorre quando um modelo ***não é capaz de capturar*** adequadamente as ***relações e padrões presentes nos dados de treinamento***.
  - Em outras palavras, o ***modelo é muito simples*** para representar a complexidade dos dados.
- O modelo apresenta um ***erro muito alto no conjunto de treinamento***.

Subajuste e sobreajuste



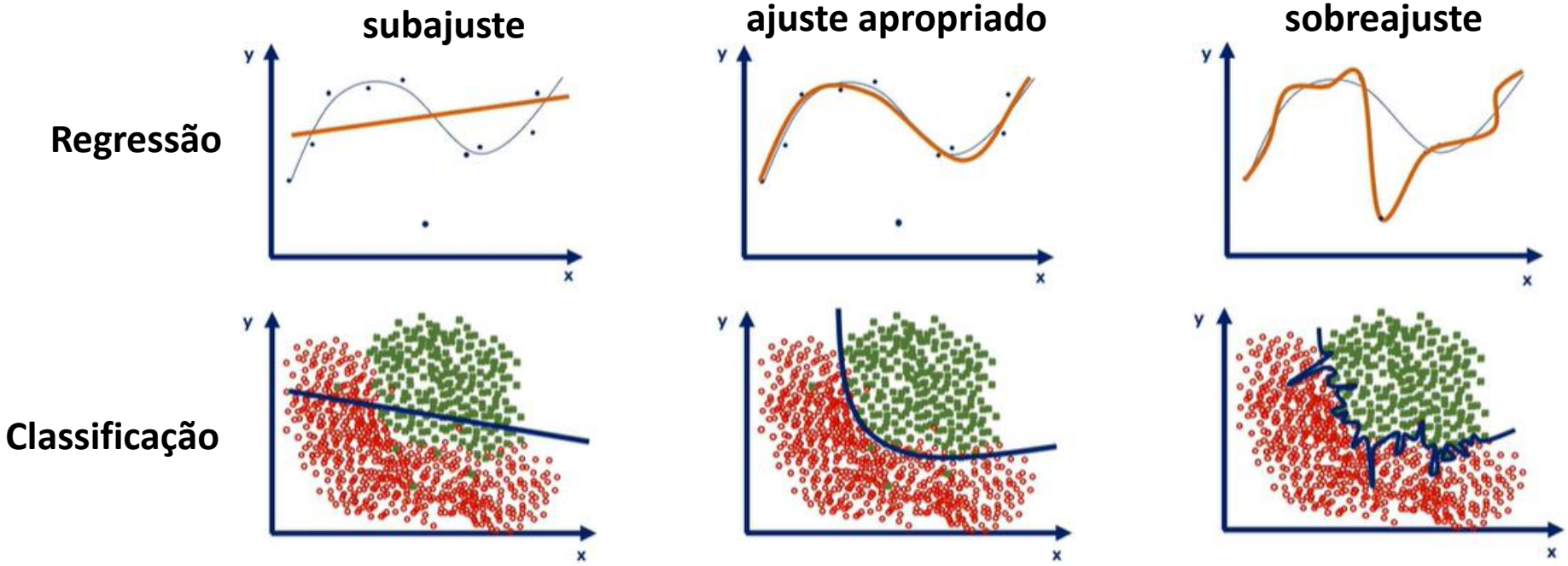
- Algumas causas do ***subajuste*** são: ***modelo muito simples*** (i.e., sem complexidade), ***poucas épocas de treinamento*** e ***falta de dados*** (modelo falha em aprender as características relevantes).

Subajuste e sobreajuste



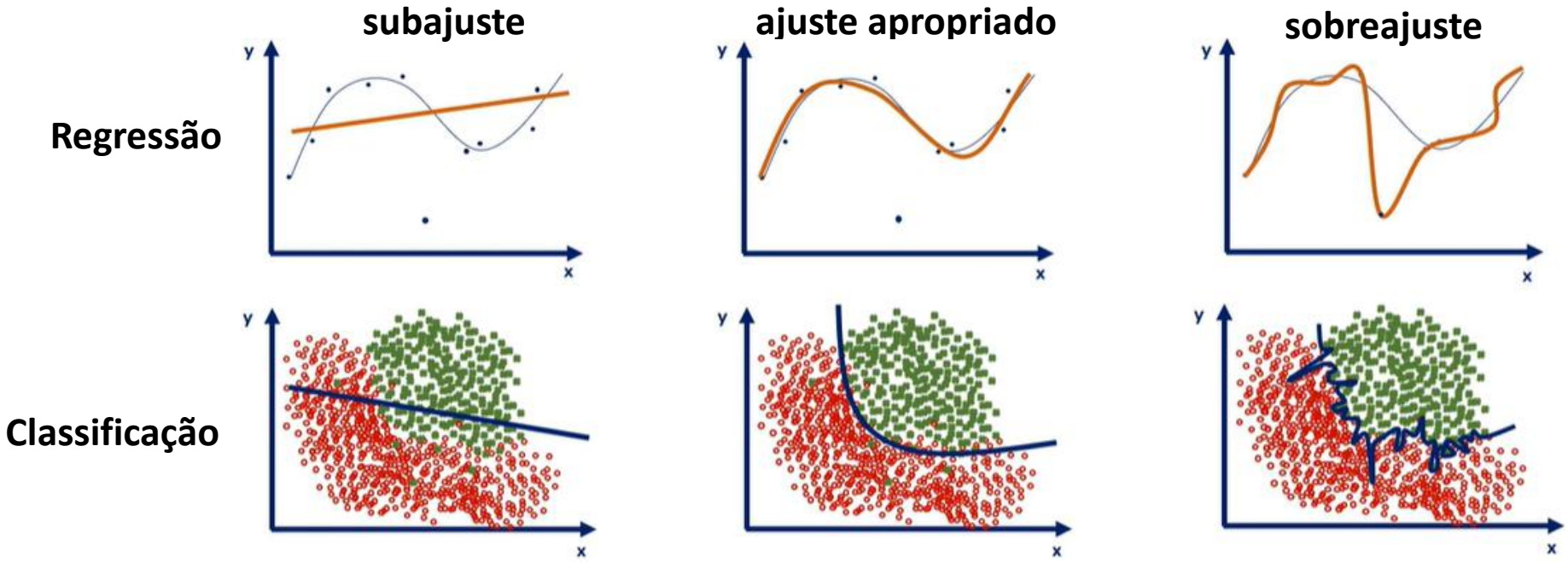
- Para mitigar o problema, podemos ***aumentar a complexidade*** do modelo (e.g., aumentar camadas e neurônios), ***ajustar os hiperparâmetros*** (e.g., passo de aprendizagem), ***treinar por mais épocas*** e ***aumentar o conjunto de treinamento***, se possível.

Subajuste e sobreajuste



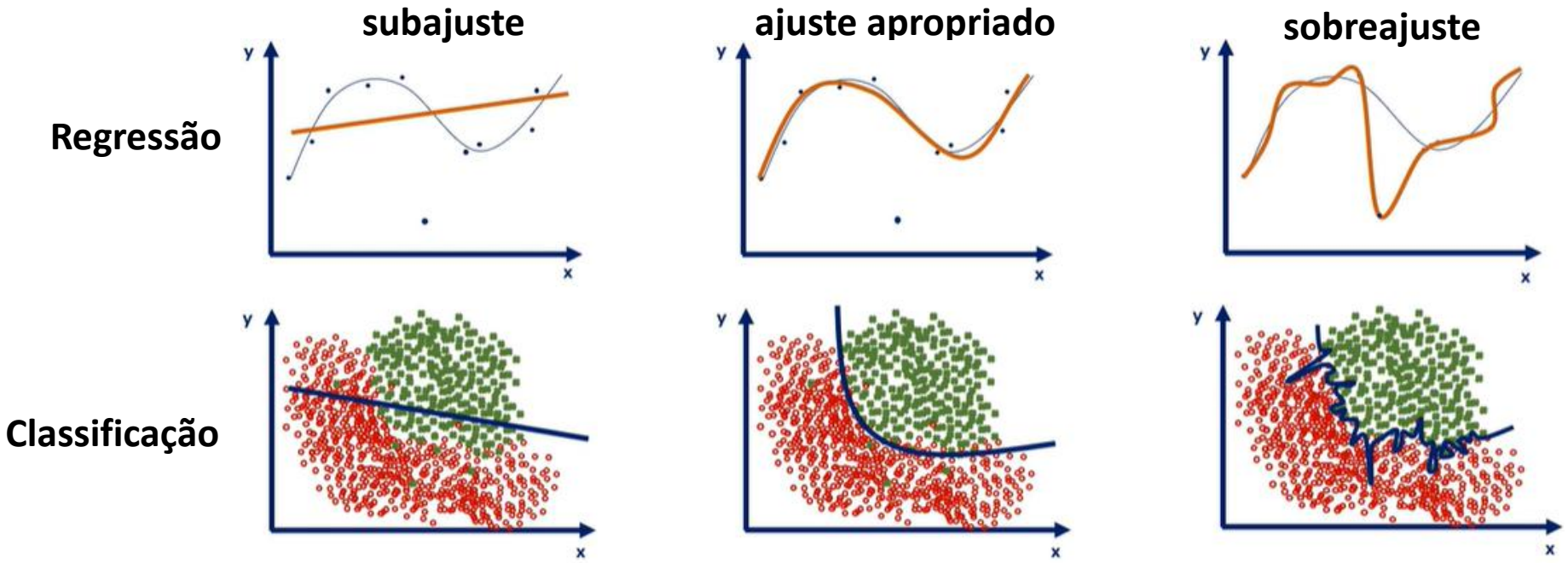
- O ***sobreajuste*** ocorre quando o modelo se ***ajusta excessivamente aos dados de treinamento*** e acaba ***perdendo a capacidade de generalizar*** para dados que não foram vistos durante o treinamento.
  - Em outras palavras, o modelo ***"memoriza" os dados de treinamento*** em vez de aprender padrões gerais.
- O modelo apresenta ***erro muito baixo no conjunto de treinamento***, próximo de zero.

Subajuste e sobreajuste



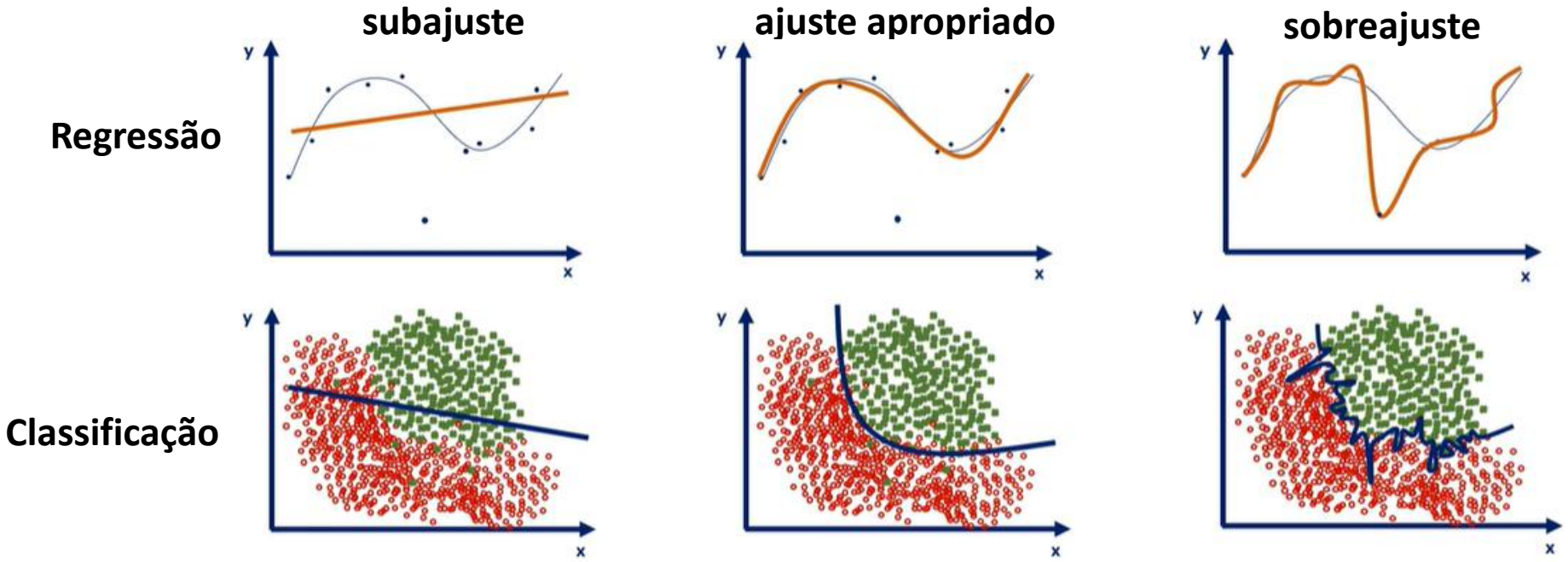
- Algumas causas do ***sobreajuste*** são: ***modelo muito complexo*** em relação à quantidade de dados, ***treinamento excessivo*** (leva à memorização dos dados de treinamento), ***falta de dados*** (modelo tem poucos exemplos para aprender os padrões gerais).

Subajuste e sobreajuste



- Para mitigar o problema, podemos ***aumentar os dados de treinamento***, se possível, ***reduzir a complexidade do modelo*** (e.g., removendo camadas ou neurônios), ***aplicar técnicas de regularização*** (e.g., *dropout*, penalizações L1 ou L2, ***early-stopping***, o qual encerra o treinamento quando o modelo começa sobreajustar).

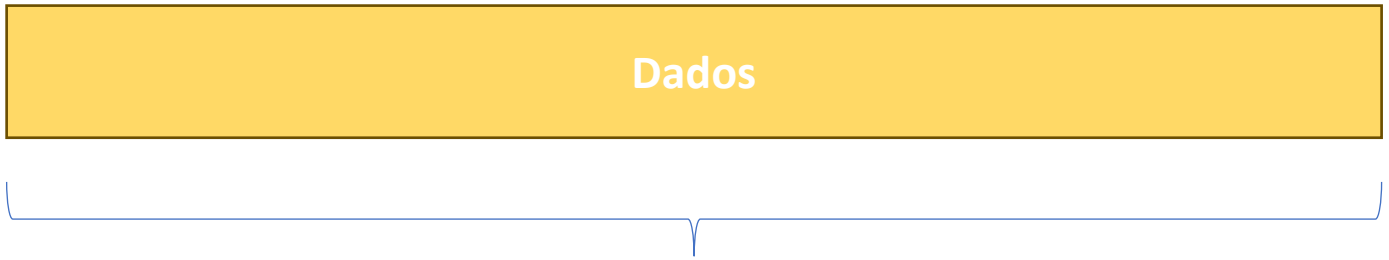
Subajuste e sobreajuste



- Encontrar o ***equilíbrio certo entre a complexidade*** (ou flexibilidade) do modelo e sua ***capacidade de generalização*** é essencial para obter um modelo com bom desempenho.
- Ou seja, devemos encontrar um ***equilíbrio entre*** um ***modelo muito simples*** (subajuste) e um ***modelo muito complexo*** (sobreajuste).

Como encontramos esse equilíbrio?

Dividir para conquistar!



Conjunto de dados com todos os exemplos (e.g., imagens) que foram coletados.

- Nossa ideia inicial foi ***treinar o modelo com todos os exemplos***, pois ***quanto mais dados*** tivermos, ***mais ele irá aprender***.
- Porém, quando o modelo é treinado com todos os exemplos que possuímos, nós ***não conseguimos mensurar o quão bem ele se sai com dados inéditos***.
  - Generalizar vs. Sobreajustar.

Dividir para conquistar!



- E se dividirmos o conjunto total de exemplos em conjuntos menores?
  - ***Conjunto de treinamento***
  - ***Conjunto de validação***
  - ***Conjunto de teste***

Conjunto de treinamento



- ***Conjunto de treinamento***: usado para atualizar os parâmetros (i.e., pesos) do modelo.
- É o maior dos três subconjuntos.
- ***Tamanho do subconjunto***: geralmente de 70% a 80% do total.

Conjunto de validação



- ***Conjunto de validação***: usado para avaliar o ***desempenho do modelo em dados inéditos e ajustar hiperparâmetros****.
- ***Tamanho do subconjunto***: geralmente de 10% a 15% do total.
- Avaliar o desempenho do modelo nesse conjunto é importante para evitar o subajuste e o sobreajuste.

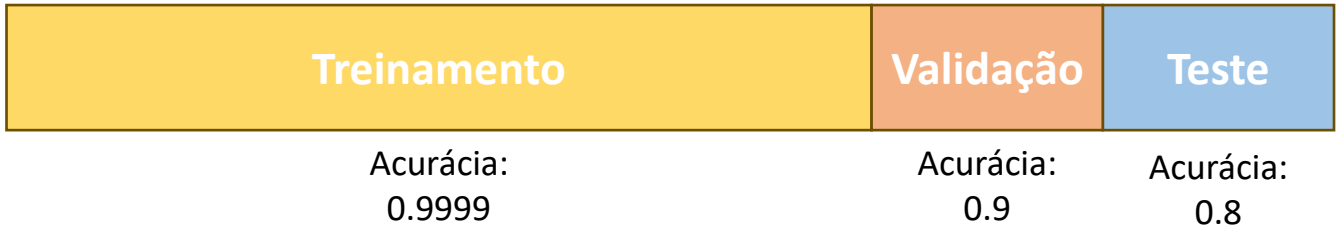
***Hiperparâmetros**: parâmetros que não são aprendidos durante o treinamento do modelo, mas que influenciam seu aprendizado

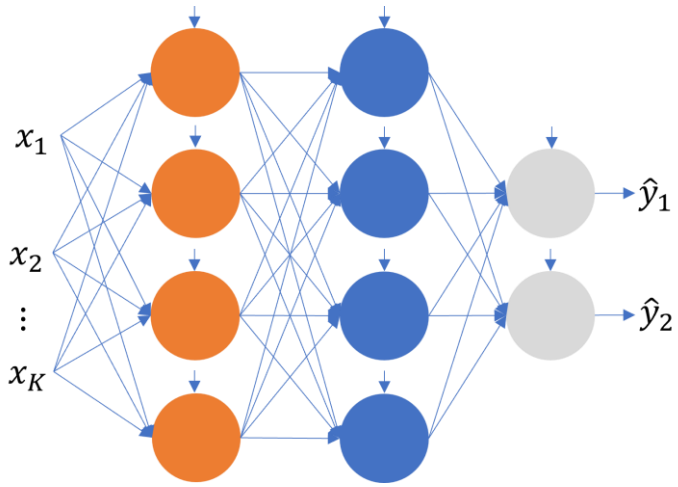
Conjunto de teste



- ***Conjunto de teste*:** conjunto ***mantido*** completamente ***separado durante todo o processo de desenvolvimento*** do modelo.
- É ***usado apenas no final*** para avaliar o desempenho do modelo em dados inéditos.
- ***Tamanho do subconjunto***: geralmente de 10% a 15% do total.

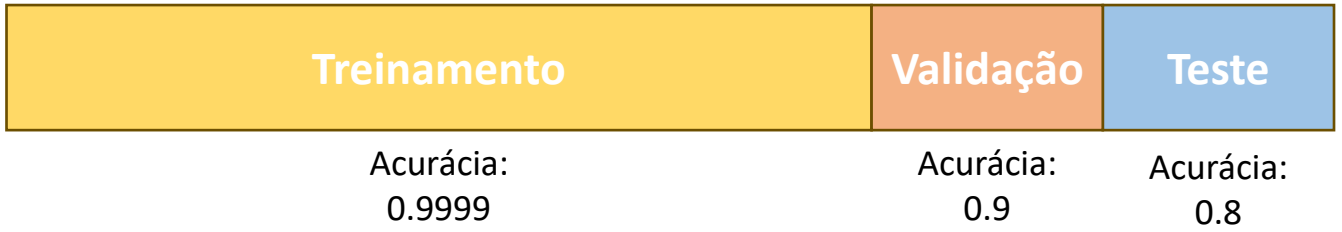
Aplicando a metodologia

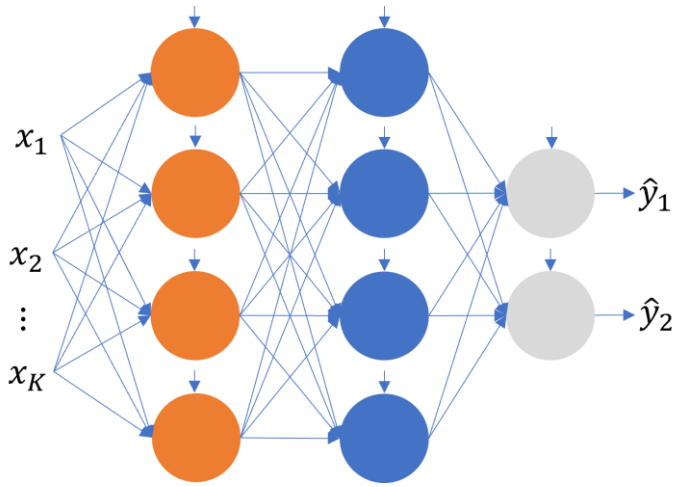




- Seguindo essa metodologia, poderíamos, por exemplo, escolher uma arquitetura de rede neural e treiná-la para resolver um problema de classificação.
- Nos dados de ***treinamento, a acurácia é de 99.99%***.
- Mas ela é pior nos outros dois conjuntos.
- Ela é de ***90% e 80% nos conjunto de validação e teste***, respectivamente.
- O que isso pode indicar?

Aplicando a metodologia

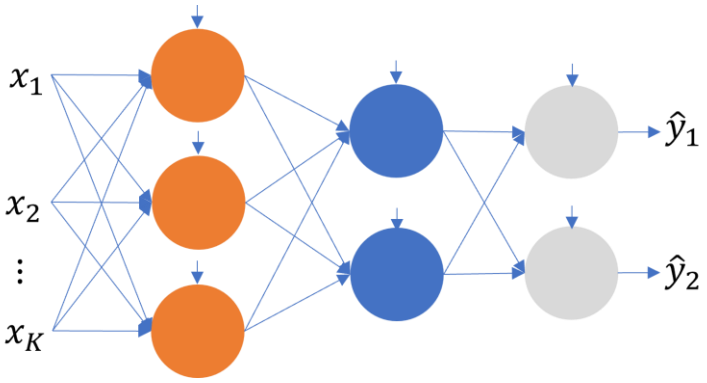




- Podemos estar diante de uma situação igual a da detecção de calçados.
- Projetamos uma rede neural que é ***ótima nos dados de treinamento, mas não tão boa nos outros dados***.
  - Uma indicação de sobreajuste.
- Os 99.99% nos fazem pensar que temos uma rede muito melhor do que realmente temos.
- E se reprojetarmos a rede, reduzindo sua complexidade, e tentamos novamente?

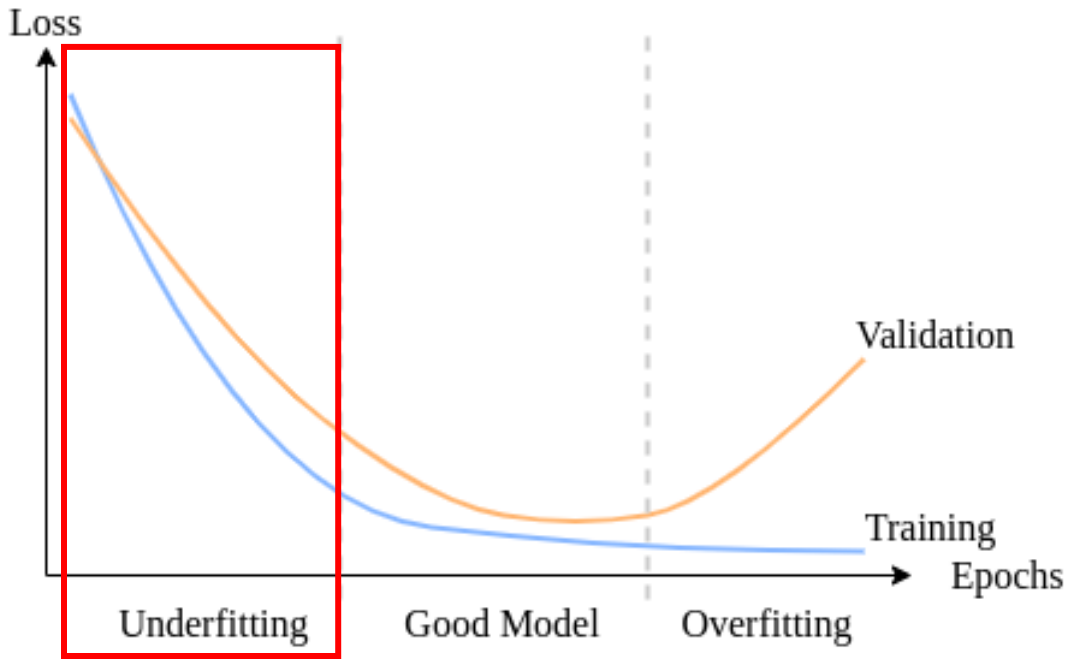
Aplicando a metodologia





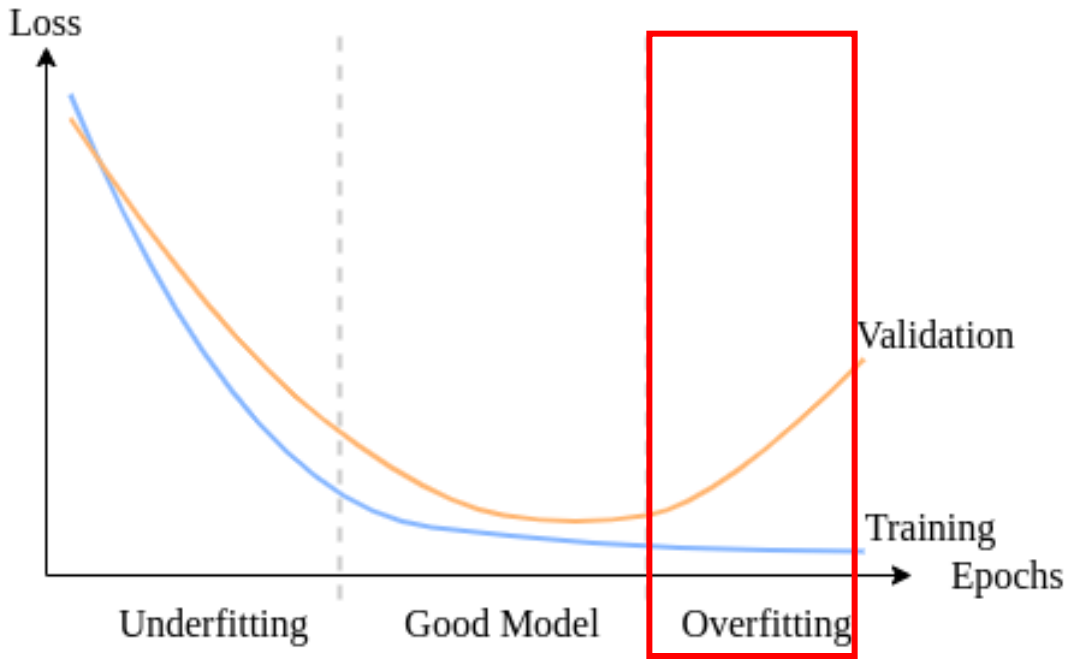
- Com uma rede menor, a acurácia no conjunto de treinamento pode até diminuir, mas o ***mais importante é manter a acurácia da rede nos conjuntos de validação e teste o mais próximos do treinamento***.
- Essa proximidade dos valores nos dará uma ***forte indicação da verdadeira acurácia da rede***.

Analisando os erros ao longo do treinamento



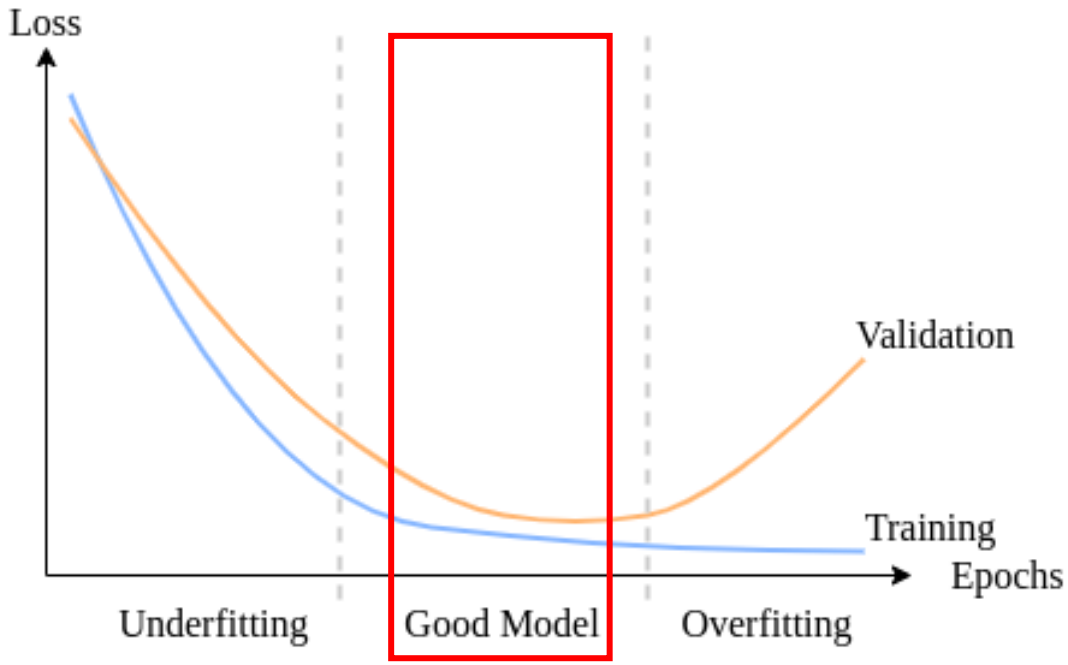
- Podemos extrair informações muito importantes a partir dos ***resultados de erro ao longo das épocas de treinamento*** de um modelo.
- Por exemplo, se ***ambos os erros são altos***, temos a ocorrência de ***subajuste.***
- O ***modelo não tem complexidade*** o suficiente para encontrar um padrão geral e/***ou ainda não treinou o suficiente*** (i.e., épocas iniciais).
- Aumentar o passo de aprendizagem pode ajudar a mitigar o problema.

Analisando os erros ao longo do treinamento



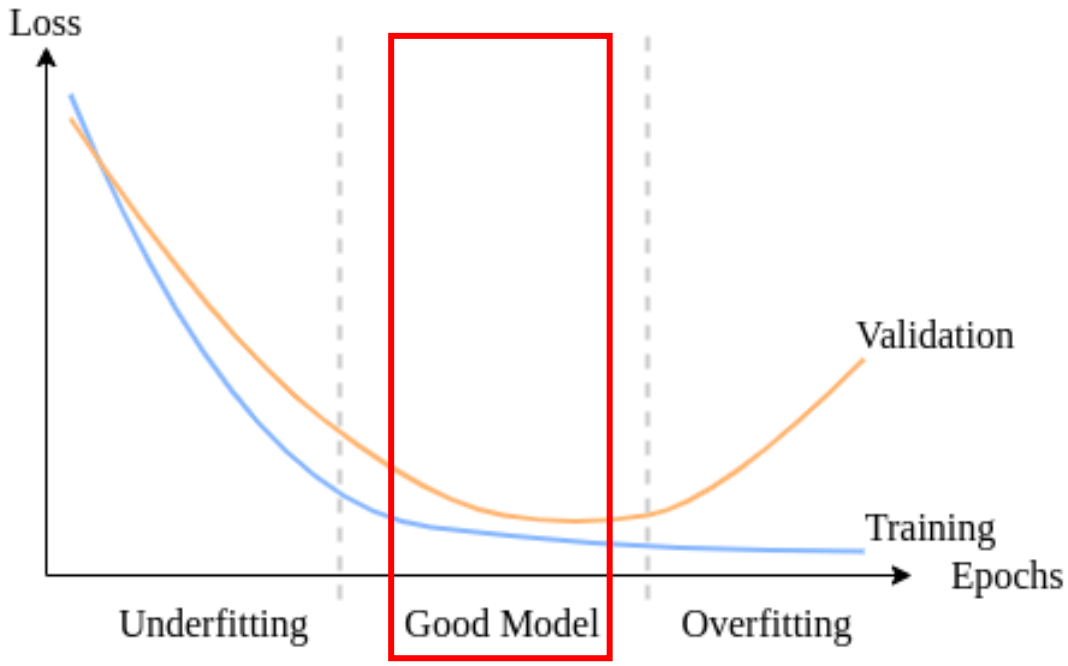
- Por outro lado, se o ***erro de treinamento for pequeno e o erro de validação for alto***, estamos diante de um caso de ***sobreajuste***.
- O modelo ***tem complexidade maior do que a necessária*** e/ou ***treinou por um número muito grande de épocas***.
  - Quando um modelo vê o mesmo conjunto muitas vezes, a tendência é que ele memorize os dados, da mesma forma que ocorre conosco.

Analisando os erros ao longo do treinamento



- Se ***ambos os erros são pequenos e próximos***, temos um modelo que ***generaliza bem***.
- O modelo atinge um ***balanço entre complexidade e capacidade de generalização***.

Analisando os erros ao longo do treinamento

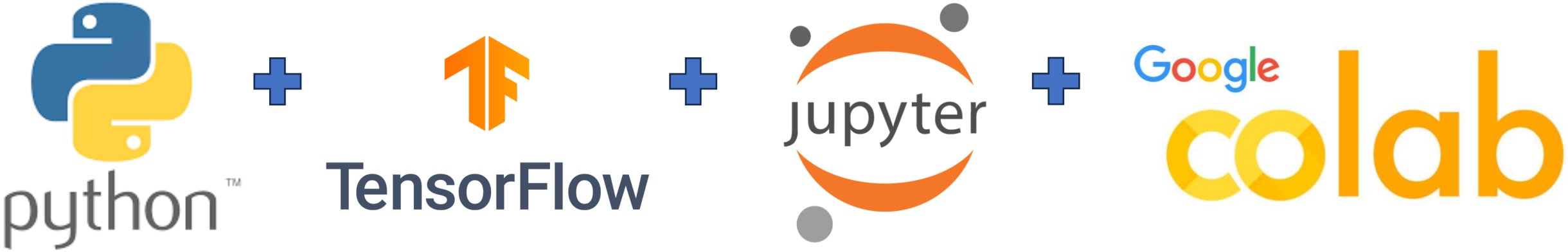


- O modelo tem a ***complexidade ideal*** para ***capturar o padrão geral por trás dos dados*** e com isso generalizar bem.
- Uma forma de obter o ***melhor modelo*** seria ***armazenar os pesos que resultarem no menor erro de validação*** e ***encerrar o treinamento*** assim que o ***erro de validação começar a aumentar*** consistentemente.
- Essa estratégia é chamada de ***early-stopping***.

À luz dessas informações, vamos revisitar nosso exemplo dos dígitos escritos à mão.

Exemplo

- Exemplo: Detecção de dígitos escritos à mão com dados de validação e teste

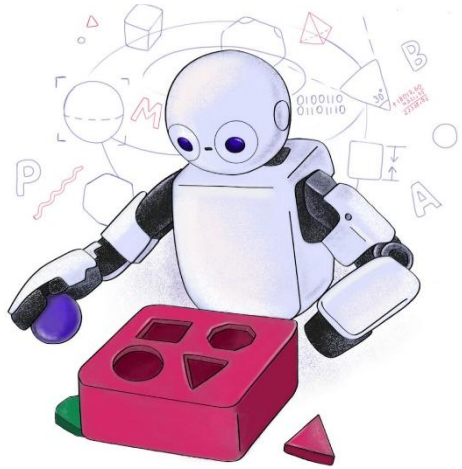


Atividades

- Quiz: “***TP557 - Datasets***”.
- Exercício: Analisando os resultados do treinamento de um modelo de ML.

TP557 - Tópicos avançados em IoT e Machine Learning:

Métricas para análise de classificadores



Inatel

Felipe Augusto Pereira de Figueiredo
felipe.figueiredo@inatel.br

O que vamos ver?

- Anteriormente, discutimos ***problemas*** que podemos nos deparar quando trabalhando com modelos de ML: ***subajuste*** e ***sobreajuste***.
- Vimos como a ***divisão do conjunto total de amostras*** em conjuntos de treinamento, validação e teste pode nos ***ajudar a analisar o desempenho de um modelo*** de ML e entender se o modelo está ***subajustando***, ***sobreajustando*** ou ***generalizando***.
- Para fazer a análise do desempenho de um classificador, usamos apenas a acurácia.
- Porém, existem outras métricas que podem ser usadas para se medir a qualidade de um classificador.
- Neste tópico, veremos algumas outras métricas.

Matriz de confusão

$$\boldsymbol{C} = \begin{bmatrix} C_{11} & C_{12} & \cdots & C_{1Q} \\ C_{21} & C_{22} & & C_{2Q} \\ \vdots & & \ddots & \vdots \\ C_{Q1} & C_{Q2} & \cdots & C_{QQ} \end{bmatrix}$$

- O nome, ***matriz de confusão*** mostra o quanto um classificador está se ***confundindo***.
- A matriz permite verificar quais classes o ***classificador*** tem ***maior dificuldade em classificar***.
- A ***matriz de confusão*** ***contabiliza o número de classificações corretas e incorretas*** para cada uma das Q classes existentes.
- É uma ***matriz quadrada*** com dimensões $\mathbb{R}^{Q \times Q}$.

Matriz de confusão

Predições: classes a que os exemplos foram atribuídos

$$\begin{bmatrix} C_{11} & C_{12} & \cdots & C_{1Q} \\ C_{21} & C_{22} & & C_{2Q} \\ \vdots & & \ddots & \vdots \\ C_{Q1} & C_{Q2} & \cdots & C_{QQ} \end{bmatrix}$$

Rótulos: classes a que realmente pertencem os exemplos

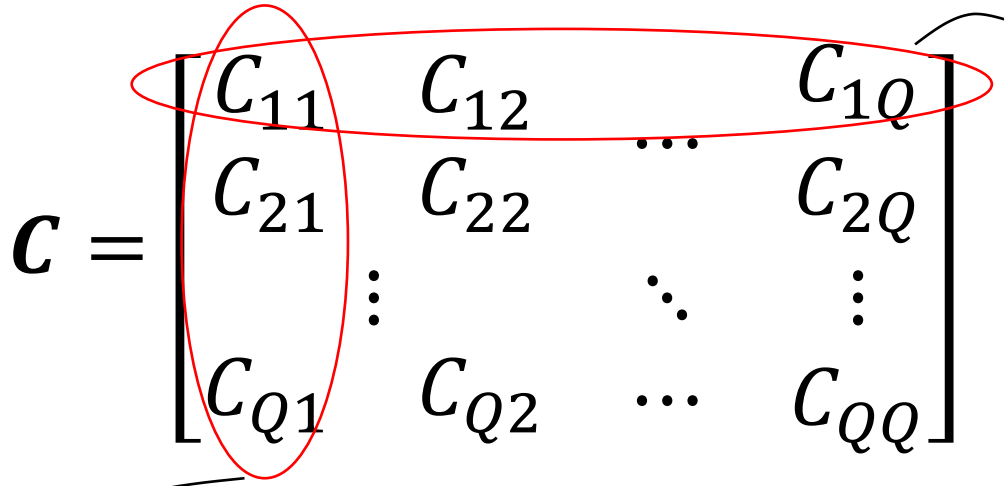
*em geral se apresenta a matriz desta forma, mas ela pode ser encontrada transposta também.

Matriz de confusão

$$\boldsymbol{C} = \begin{bmatrix} C_{11} & C_{12} & \cdots & C_{1Q} \\ C_{21} & C_{22} & & C_{2Q} \\ \vdots & & \ddots & \vdots \\ C_{Q1} & C_{Q2} & \cdots & C_{QQ} \end{bmatrix}$$

- A ***diagonal principal*** de $\boldsymbol{C}$ fornece o número de ***classificações corretas***.
- A q-ésima ***linha*** indica o ***total de exemplos que foram classificados como pertencentes a q-ésima classe***, incluindo exemplos que pertencem e não pertencem à classe.
- A q-ésima ***coluna*** indica o ***total de exemplos que realmente pertencem à q-ésima classe***, incluindo classificações corretas e incorretas.

Matriz de confusão

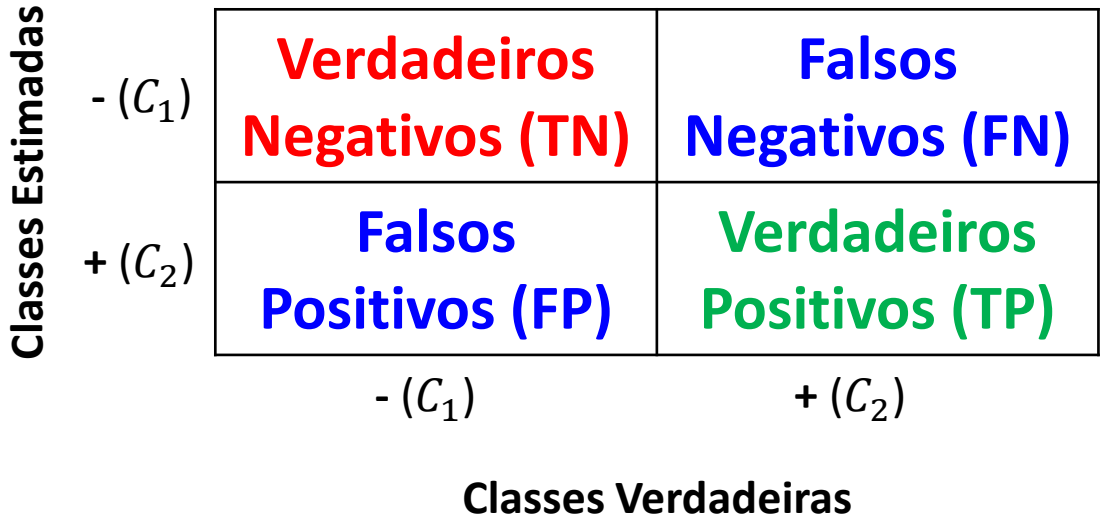


Exemplos classificados como pertencentes à classe 1.

Quantidade de exemplos realmente pertencentes à classe 1.

- C_{11} indica quantos exemplos da classe 1 foram corretamente atribuídos à classe 1.
- C_{12} indica quantos exemplos da classe 2 foram atribuídos à classe 1.

Matriz de confusão para caso binário ($Q = 2$)



- ***Verdadeiros Positivos*** (TP): número de exemplos da classe positiva, C_2, classificados corretamente.
- ***Verdadeiros Negativos*** (TN): número de exemplos da classe negativa, C_1, classificados corretamente.
- ***Falsos Positivos*** (FP): número de exemplos atribuídos à classe positiva, mas que pertencem à classe negativa.
- ***Falsos Negativos*** (FN): número de exemplos atribuídos à classe negativa, mas que pertencem à classe positiva.

Matriz de confusão para caso binário ($Q = 2$)

verdadeiro positivo



falso positivo





falso negativo



verdadeiro negativo

Nós podemos calcular algumas métricas de desempenho importantes a partir das informações contidas na matriz de confusão.

Acurácia

$$\text{acurácia} = \frac{\text{TP} + \text{TN}}{\text{TP + FN + FP + TN}}$$

- Acurácia mede a ***proporção de exemplos classificados corretamente em relação ao total de exemplos*** avaliados.
- Em outras palavras, a acurácia fornece uma indicação de quão bem o modelo está fazendo suas ***predições corretas em comparação com todas as previsões feitas***.
- É uma métrica útil para avaliar a ***performance geral do modelo***.

Acurácia

$$\text{acurácia} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FN} + \text{FP} + \text{TN}}$$

- A ***acurácia*** é, geralmente, a ***primeira escolha*** para medir a qualidade de um classificador.
- Entretanto, ela pode ser ***enganosa com problemas desbalanceados***.
  - Problemas onde uma ou algumas classes têm muito mais exemplos do que as demais.
- Nesses casos, ela pode nos levar a concluir que um ***classificador ruim é muito bom***.

Acurácia

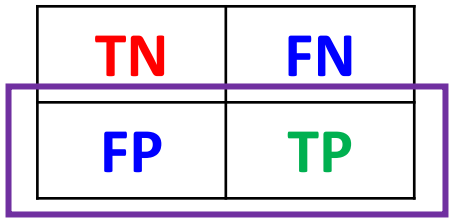
- Analisando a equação da acurácia, o que aconteceria se TP fosse muito maior do que TN, FN e FP?

$$\lim_{\mathrm{TP}\to\infty} \text{acurácia} = \lim_{\mathrm{TP}\to\infty} \left(\frac{\mathrm{TP} + \mathrm{TN}}{\mathrm{TP} + \mathrm{TN} + \mathrm{FN} + \mathrm{FP}} \right) = \frac{\mathrm{TP}}{\mathrm{TP}} = 1.$$

- O mesmo aconteceria se TN fosse muito maior do que TP, FN e FP
- Portanto, quando temos ***classes desbalanceadas, precisamos analisar outras métricas***.
- Ela também é enganosa quando os custos de falsos positivos e falsos negativos não são iguais.

Precisão

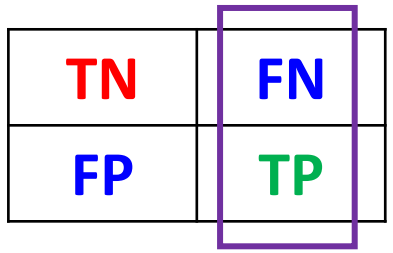
$$\text{precisão} = \frac{\text{TP}}{\text{FP} + \text{TP}}$$



- **Precisão** é a proporção de exemplos da classe positiva corretamente classificados (TP) em relação a todos os exemplos atribuídos à classe positiva (TP + FP).
- É uma ***boa medida*** para determinar a qualidade do classificador ***quando os custos de falsos positivos são altos***.
  - Por exemplo, na classificação de ***spams*** (***verdadeiro positivo***), um ***falso positivo*** significa que um ***ham*** (***verdadeiro negativo***) foi classificado como ***spam***. O usuário de *email* pode perder *emails* importantes se a ***precisão*** for baixa.

Recall

$$recall = \frac{\mathrm{TP}}{\mathrm{FN} + \mathrm{TP}}$$



- ***Recall*** ou sensibilidade é a proporção de exemplos da classe positiva corretamente classificados.
- O *recall* calcula quantos exemplos realmente positivos o classificador captura em relação a todos exemplos positivos.
- É uma ***boa medida*** para determinar a qualidade de um classificador ***quando os custos de falsos negativos são altos***.
  - Por exemplo, na classificação de doenças, se um paciente doente (***verdadeiro positivo***) for classificado como não doente (***falso negativo***). O custo associado ao ***falso negativo*** será extremamente alto se a doença for contagiosa.

OK, consigo calcular as métricas para problemas binários, mas e quando $Q > 2$?

Matriz de confusão para caso multiclasses ($Q > 2$)

Classe C_1 é a positiva.

Classes Preditas	+ (C_1)	Verdadeiro Positivo (TP)	Falso Positivo (FP)	Falso Positivo (FP)
	- (C_2)	Falso Negativo (FN)	Verdadeiro Negativo (TN)	Verdadeiro Negativo (TN)
	- (C_3)	Falso Negativo (FN)	Verdadeiro Negativo (TN)	Verdadeiro Negativo (TN)
		+ (C_1)	- (C_2)	- (C_3)
		Classes Verdadeiras		

- É possível calcular as métricas anteriores para o cenário multiclasses (i.e., $Q > 2$).
- Para isso, basta selecionar, uma vez, cada classe como sendo a classe positiva, enquanto todas as demais classes formam a classe negativa.
- Assim, obtém-se os valores das ***métricas para cada uma das Q classes***.
- Vejamos um exemplo para $Q = 3$, ou seja, C_1, C_2 e C_3.

Matriz de confusão para caso multiclasses ($Q > 2$)

Classe C_2 é a positiva.

Classes Preditas	- (C_1)	**Verdadeiro Negativo (TN)**	**Falso Negativo (FN)**	**Verdadeiro Negativo (TN)**
	+ (C_2)	**Falso Positivo (FP)**	**Verdadeiro Positivo (TP)**	**Falso Positivo (FP)**
	- (C_3)	**Verdadeiro Negativo (TN)**	**Falso Negativo (FN)**	**Verdadeiro Negativo (TN)**
		- (C_1)	+ (C_2)	- (C_3)
		Classes Verdadeiras		

Classe C_3 é a positiva.

Classes Preditas	- (C_1)	**Verdadeiro Negativo (TN)**	**Verdadeiro Negativo (TN)**	**Falso Negativo (FN)**
	- (C_2)	**Verdadeiro Negativo (TN)**	**Verdadeiro Negativo (TN)**	**Falso Negativo (FN)**
	+ (C_3)	**Falso Positivo (FP)**	**Falso Positivo (FP)**	**Verdadeiro Positivo (TP)**
		- (C_1)	- (C_2)	+ (C_3)
		Classes Verdadeiras		

Precisão versus *recall*

$$\text{precisão} = \frac{\text{TP}}{\text{FP} + \text{TP}}$$

$$recall = \frac{\text{TP}}{\text{FN} + \text{TP}}$$

- A ***precisão*** não fornece informações a respeito da quantidade de ***falsos negativos***.
- Por outro lado, o ***recall*** não fornece informações a respeito da quantidade de ***falsos positivos***.
- Mas e se os ***custos associados a falsos positivos e negativos são iguais***?
- Nesse caso, para analisarmos o desempenho de um classificador, precisamos de uma métrica que ***combine as duas***.

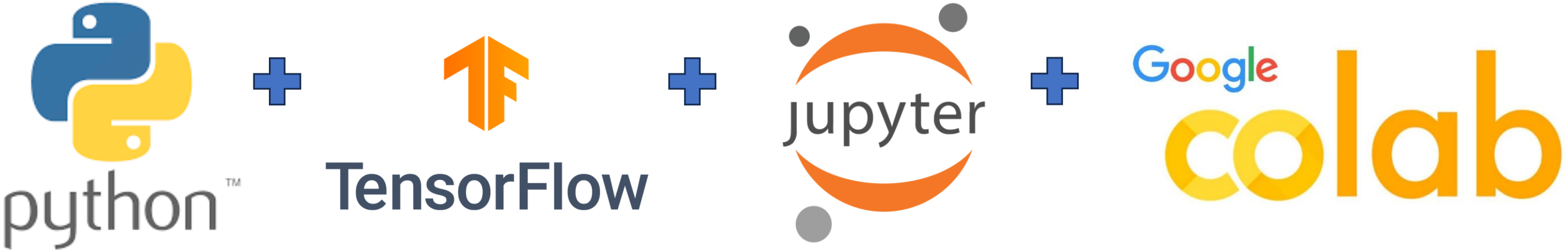
F1-*score*

- O F1-*score* combina as duas métricas em uma única.
- A métrica dá a mesma importância a ***precisão*** e para o ***recall***.
- Valores de F_1 próximos de 1 indicam que o ***classificador*** obteve ótimos resultados tanto de ***precisão*** quanto de ***recall***.
- Em outras palavras, $F_1 \approx 1$ significa que FN e $\mathrm{FP} \approx 0$.

$$F_1 = \frac{\mathrm{TP}}{\mathrm{TP} + \frac{\mathrm{FN} + \mathrm{FP}}{2}} = 2\,\frac{\mathrm{precisão} \times recall}{\mathrm{precisão} + recall}.$$

Exemplo

- Exemplo: Métricas de classificação

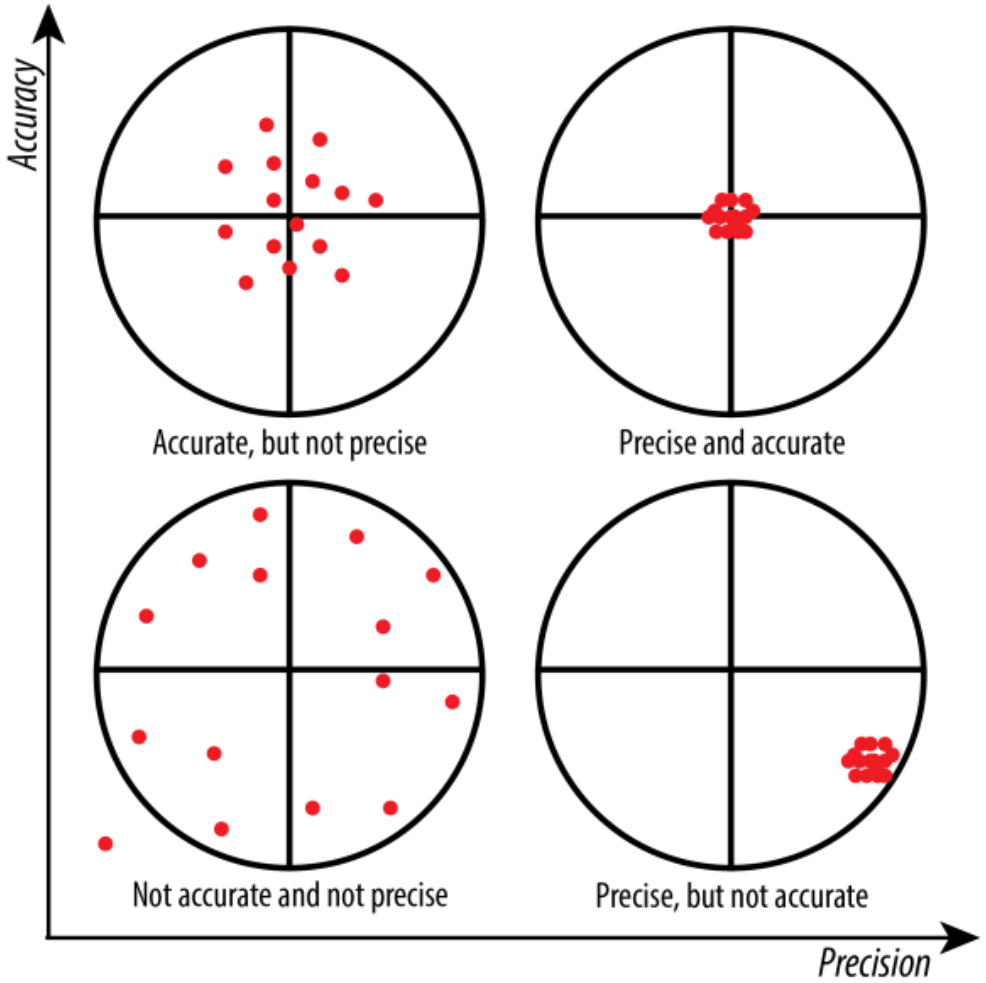


Atividades

- Quiz: “***TP557 – Métricas para análise de classificadores***”.
- Exercício: Métricas de classificação

Anexo I: Acurácia versus precisão

Acurácia versus precisão

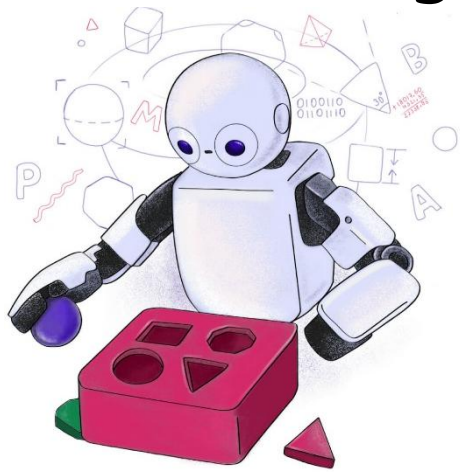


- Em um conjunto de predições:
  - Acurácia é a proximidade das predições em relação aos rótulos, ou seja, o quão perto estão do alvo.
  - Precisão é a proximidade das medidas entre si.
- A precisão é independente da acurácia.
- Isso significa que é possível ter um modelo muito preciso, mas não muito exato, e também é possível ser exato sem ser preciso.

Obrigado!

TP557 - Tópicos avançados em IoT e Machine Learning:

Introduzindo Convoluções



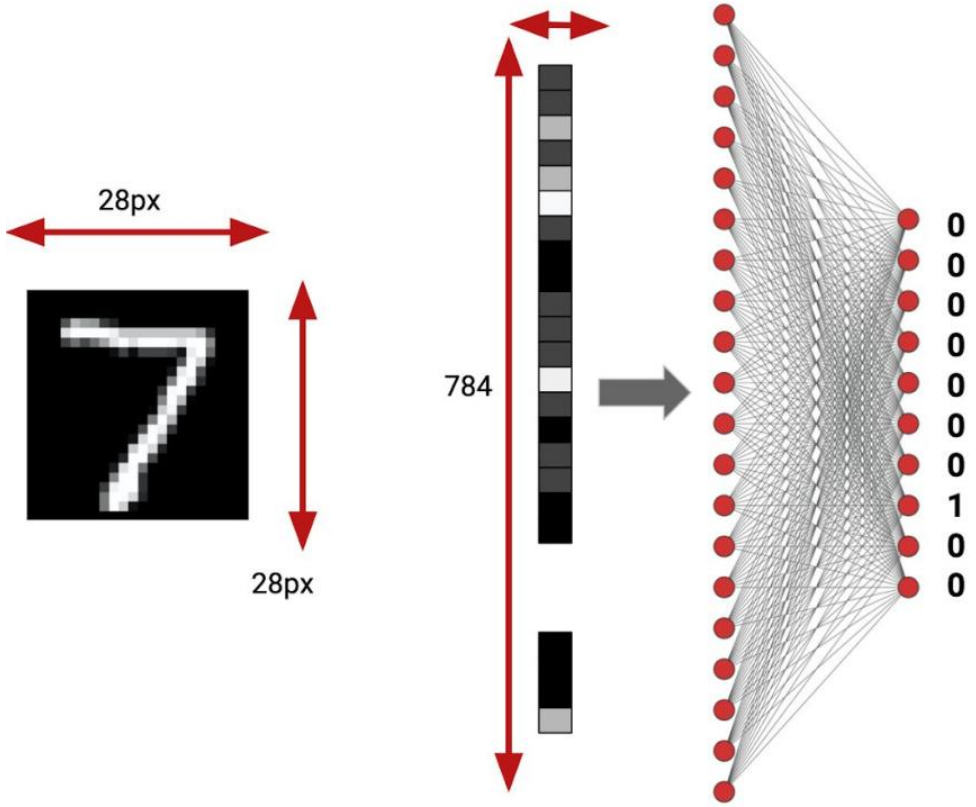
Inatel

Felipe Augusto Pereira de Figueiredo
felipe.figueiredo@inatel.br

O que vamos ver?

- Até agora, nossas redes neurais continham apenas dois tipos de camadas: ***densas*** e de ***achatamento***.
- Porém, um outro tipo muito importante são as ***camadas convolucionais***.
- Essas camadas formam as *Convolutional Neural Networks* (CNNs).
- A ***principal diferença*** para uma DNN é que ao invés de ***aprender os pesos*** das camadas densas, uma ***CNN aprende*** os valores de ***filtros de convolução*** (ou apenas ***filtros***, mas também chamados de ***kernels***).
  - Esses filtros são muito eficientes em "compreender" o conteúdo de imagens.
- CNNs são usadas em tarefas de ***visão computacional***, como, por exemplo, ***reconhecimento de objetos***, ***detecção de padrões***, ***segmentação de imagens***, ***rastreamento de objetos***, etc.

Dados tabulares



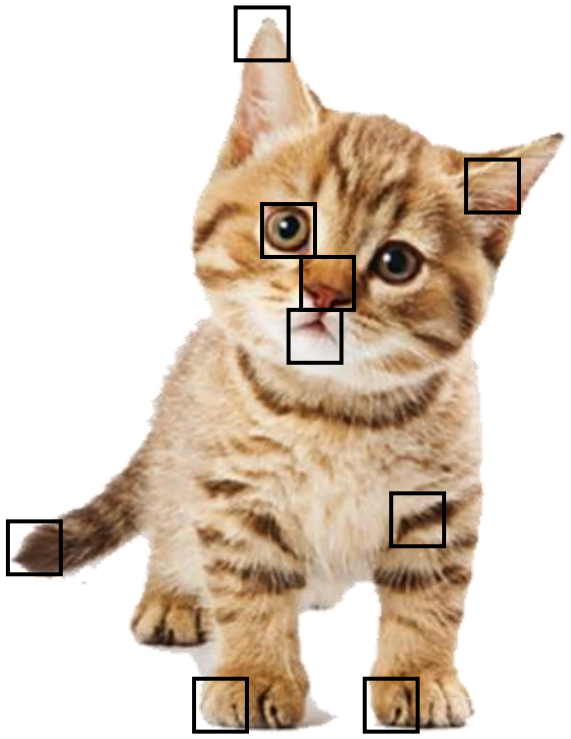
- DNNs são ***ideais para dados tabulares***, onde os ***exemplos são representados por linhas e os atributos por colunas***.
- Ao analisar esses dados, o objetivo de uma DNN é ***descobrir padrões que envolvem interações entre os atributos, sem presumir uma estrutura espacial*** (em termos de posicionamento físico) específica entre eles.
- Em contraste, ***imagens têm uma estrutura espacial*** que pode ser explorada por modelos de ML.

Padrões globais



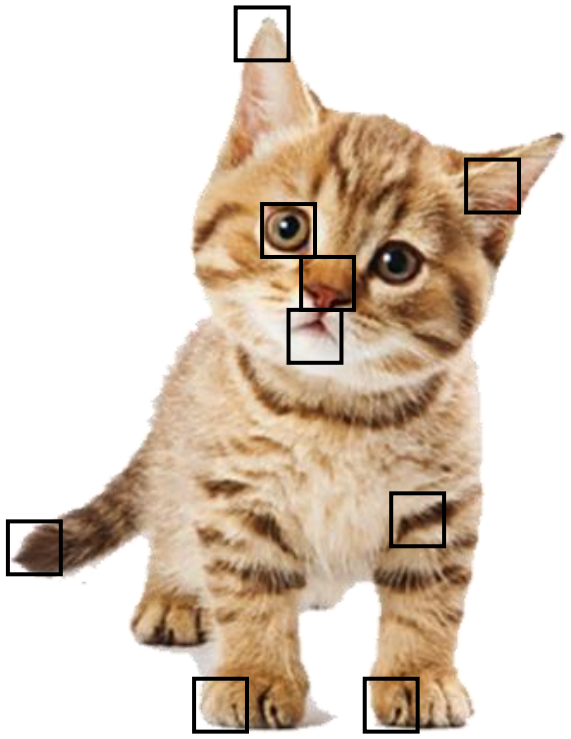
- DNNs ***não levam em consideração a estrutura espacial dos dados***.
- Elas ***tratam pixels de entrada distantes*** uns dos outros ***da mesma forma que pixels próximos***.
- Por exemplo, se embaralharmos os *pixels* das imagens de entrada de uma DNN treinada para classificar imagens de gatos e cachorros, ela ainda identificará os animais, mesmo as imagens não mais fazendo sentido visual algum.

Localidade de referência



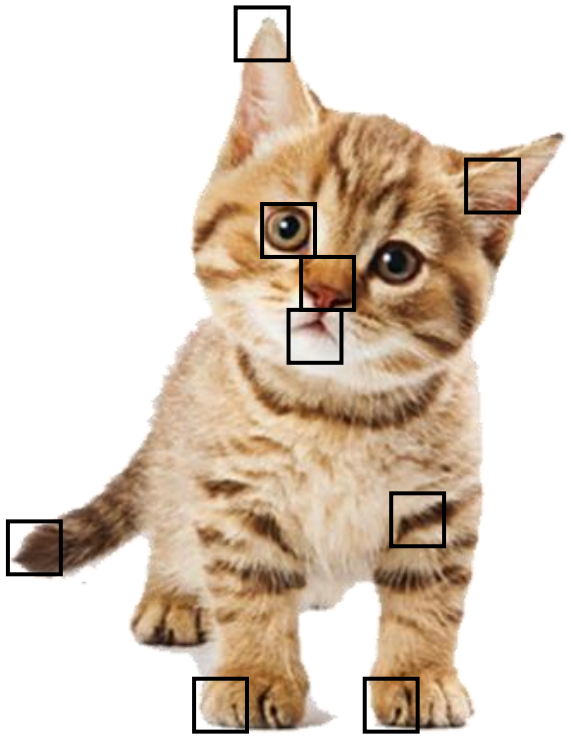
- DNNs ***ignoram a localidade de referência ou espacial*** em dados com topologia de grade (e.g., imagens), tanto computacional (eficiência) quanto semanticamente (significado de cada região).
- ***Localidade espacial***: em dados com topologia de grade, é a ***região*** onde ***informações próximas*** umas das outras ***estão relacionadas***.

Localidade de referência



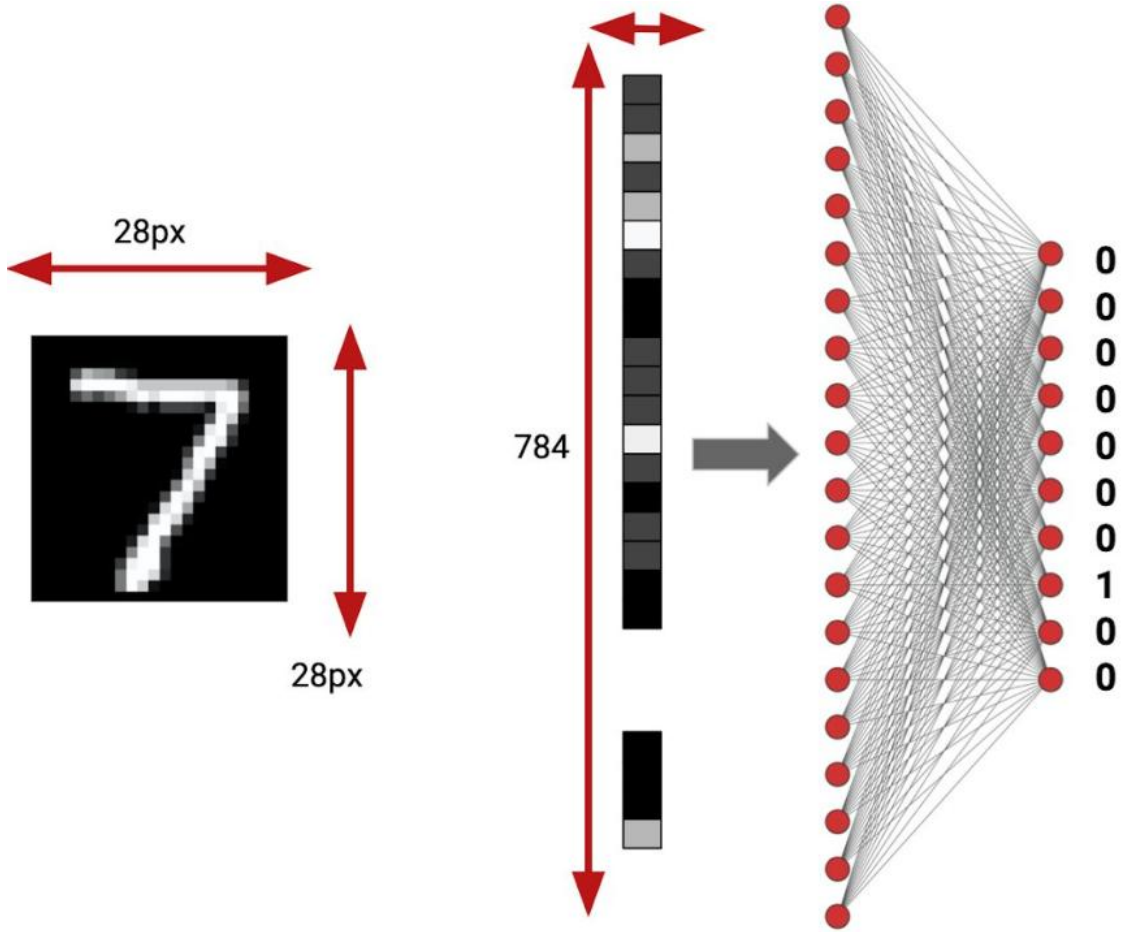
- Por exemplo, em uma imagem, *pixels* vizinhos tendem a conter informações semelhantes e relacionadas.
- A exploração da ***localidade espacial*** é ***benéfica*** para tarefas de ***reconhecimento de padrões***.

Estrutura espacial



- Assim, a ***conectividade total dos neurônios em uma DNN é um desperdício*** para propósitos como o reconhecimento de ***imagens*** que são ***dominados*** por ***padrões espacialmente locais***.
- Além disso, por serem densamente conectadas, as ***DNNs requerem um grande número de parâmetros, levando a um alto custo computacional***.

Imagens simples e “comportadas”



- Até o momento, as imagens que usamos nos problemas de classificação eram bem simples e “comportadas”.
- Eram ***imagens bidimensionais em tons de cinza***, com objetos ***centralizados***, ***sem*** muita ***variação*** em termos de ***rotação***, ***iluminação***, ***escala***, com um ***mesmo fundo***, ***sem oclusões*** (i.e., partes do objeto obstruídas), etc.

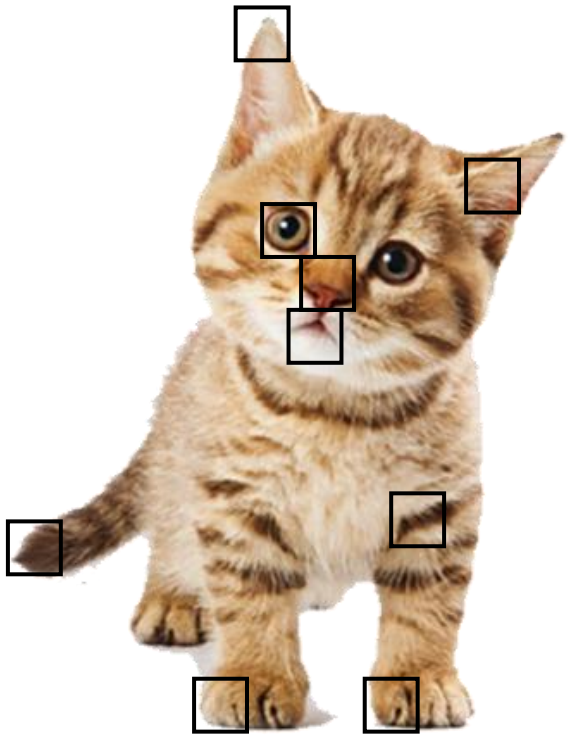
Imagens complexas





- Mas e quando as imagens são mais complexas?
- Com cores, objetos não centralizados, com variação em termos de rotação, iluminação, escala, diferentes fundos, oclusões, etc.
- Por exemplo, e se quiséssemos classificar imagens de pessoas e cavalos?

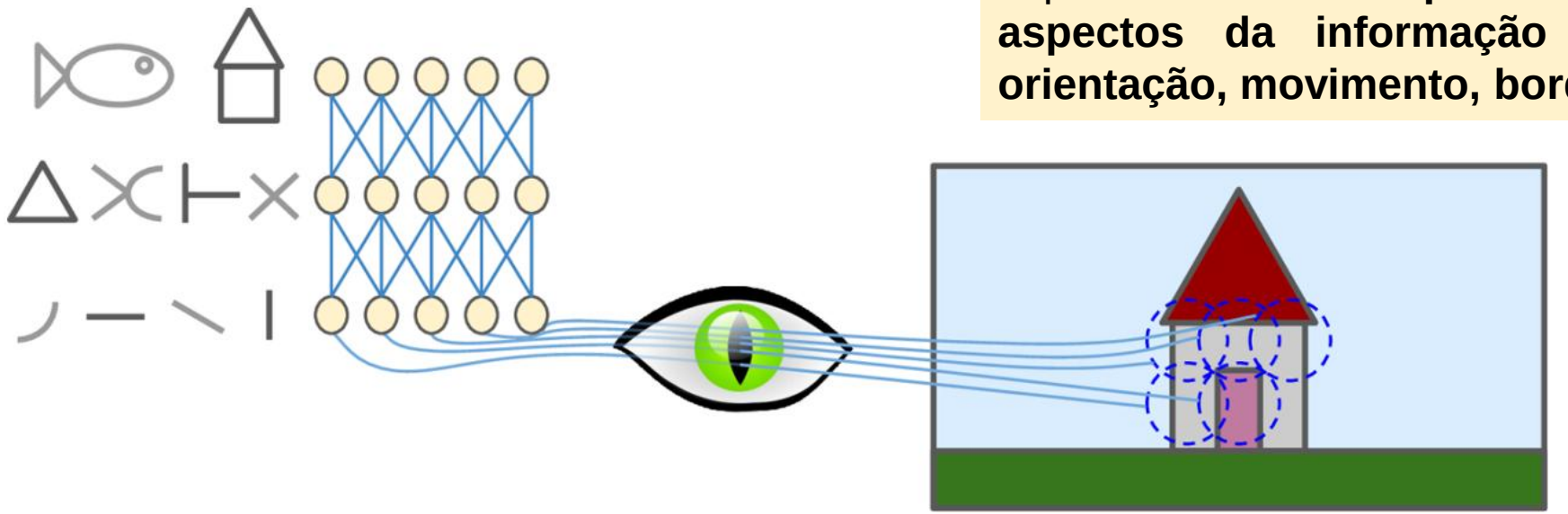
Imagens complexas



- Usar ***filtros de convolução*** pode nos ajudar a resolver esse problema.
- Os ***filtros*** podem ser ***treinados para detectar características locais*** que ***diferenciam pessoas de cavalos***.
- Essa ideia de usar ***filtros*** que ***detectam características*** que fazem um ***objeto ser diferente de outro*** é ***baseada*** em como os neurônios do ***córtex visual funcionam***.

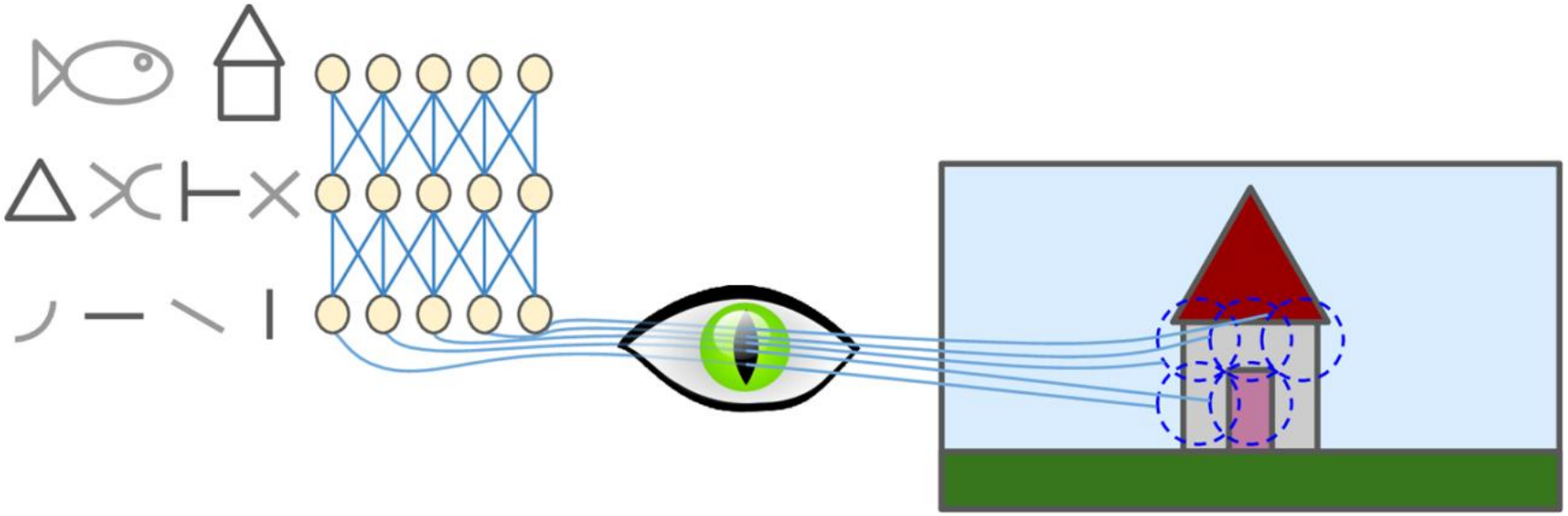
Neurônios biológicos

Os neurônios do córtex visual, localizados principalmente no lobo occipital, são especializados em **processar diferentes aspectos da informação visual, como orientação, movimento, bordas e formas.**



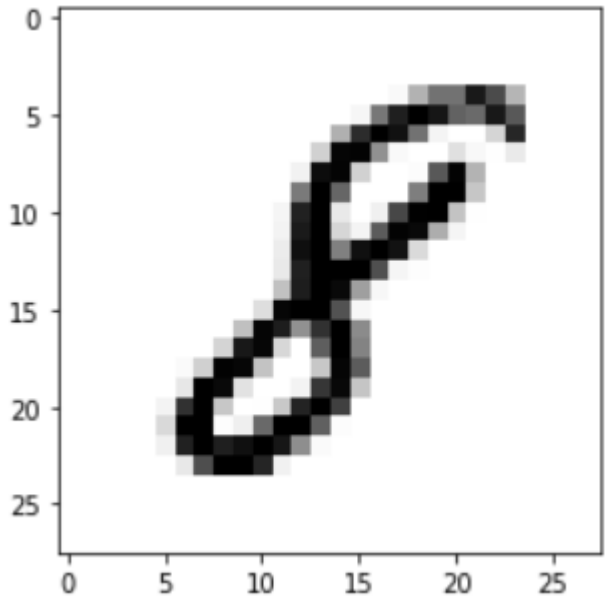
- Os ***neurônios*** biológicos no córtex visual ***respondem a padrões específicos*** em ***pequenas regiões do campo visual*** chamadas ***campos receptivos***.
- À medida que o sinal visual percorre as camadas do cérebro, os ***neurônios respondem*** a ***padrões mais complexos em campos receptivos maiores***.

Neurônios biológicos



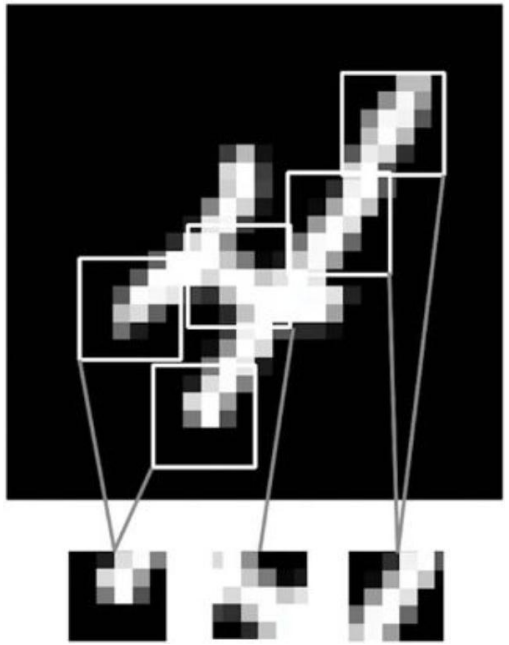
- Alguns ***neurônios reagem*** apenas a imagens de ***linhas horizontais***, enquanto ***outros reagem*** apenas a ***linhas com orientações diferentes***.
- Outros neurônios têm ***campos receptivos maiores*** e reagem a ***padrões mais complexos*** que são ***combinações de padrões de nível inferior*** (i.e., padrões mais simples).

Diferença entre camadas densas e convolucionais



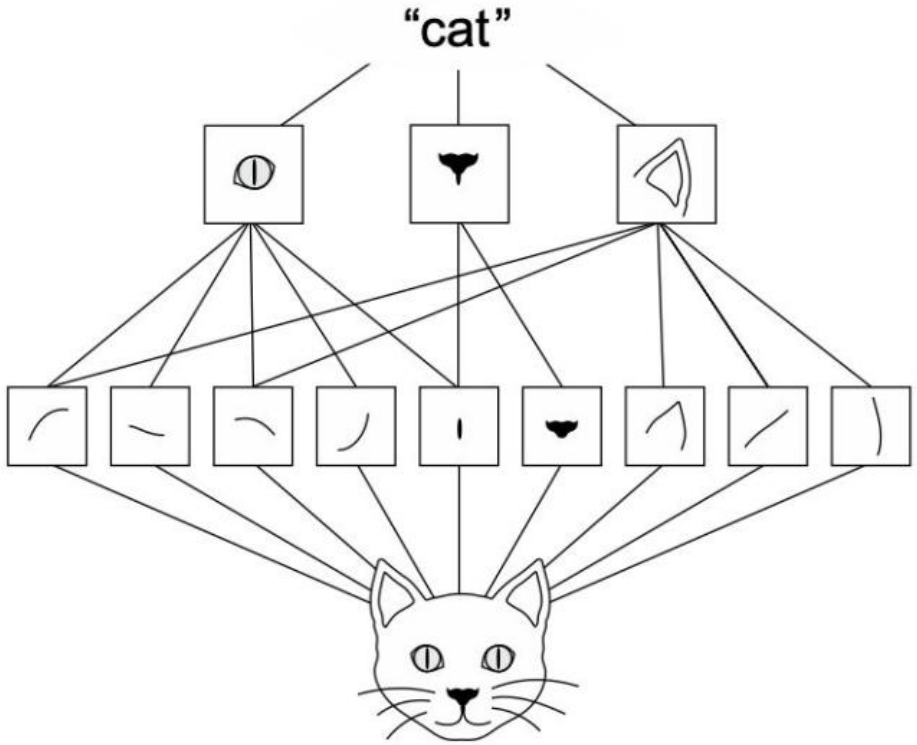
- Qual a ***diferença fundamental*** entre uma camada ***densamente conectada*** e uma de ***convolução***?
- As ***camadas densas aprendem padrões globais*** em seu espaço de atributos.
- Por exemplo, para um dígito da base de dados MNIST, as ***camadas densas aprendem padrões envolvendo todos os pixels***.

Diferença entre camadas densas e convolucionais



- As camadas de convolução ***aprendem padrões locais***.
- No caso de imagens, as camadas de convolução ***aprendem padrões encontrados em pequenas janelas*** (i.e., ***campos receptivos***) das entradas.
- Para aprender esses padrões, as camadas utilizam ***filtros de convolução***, também chamados de ***kernels***.

Diferença entre camadas densas e convolucionais



- Camadas convolucionais aprendem ***hierarquias espaciais*** de padrões.
- Uma ***primeira camada*** de convolução aprenderá ***pequenos padrões locais***, como bordas, uma ***segunda camada*** de convolução aprenderá ***padrões maiores criados a partir da combinação das características das primeiras camadas*** e assim por diante.

Cores básicas e canais





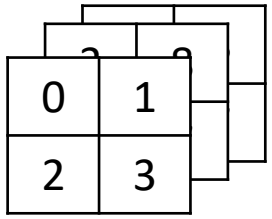
- Antes de falarmos sobre convolução, vamos falar sobre ***cores*** (ou ***canais***).
- Até agora, ignoramos que imagens, em geral, consistem em três canais: vermelho (R), verde (G) e azul (B).
- Imagens coloridas têm canais RGB para indicar a ***quantidade de vermelho, verde e azul***.
- Em suma, as ***imagens não são***, em geral, ***objetos bidimensionais***, mas sim ***tensores de três dimensões***, caracterizados por ***altura***, ***largura*** e ***canal***.

Filtros de convolução ou *kernels*

Kernel 2D

0	1
2	3

Kernel 3D



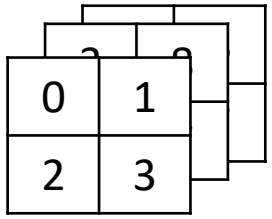
- Um ***kernel*** é um tensor (em geral em 3D) responsável por ***detectar características específicas em uma imagem***.
- Ele percorre uma imagem e realiza ***operações convolução*** entre seus valores e os dos *pixels* na região da imagem correspondente ao seu ***campo receptivo***.

Filtros de convolução ou *kernels*

Kernel 2D

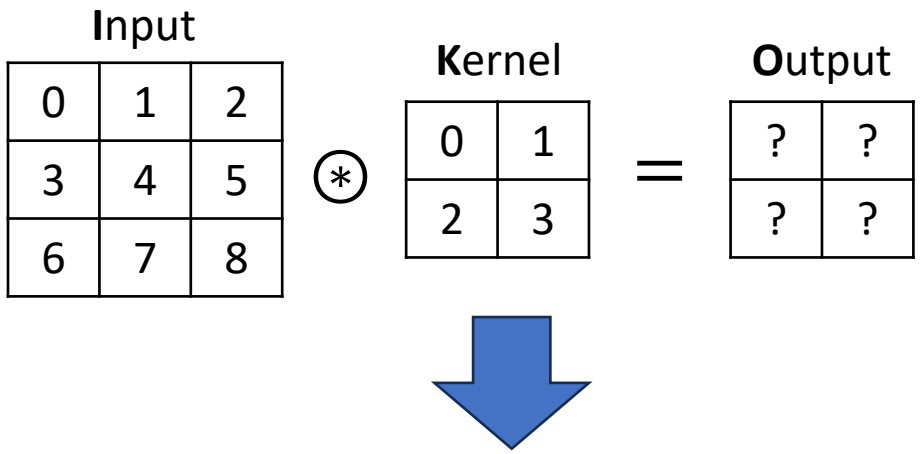
0	1
2	3

Kernel 3D



- Cada ***kernel*** ***detecta um tipo particular de característica***, como bordas, texturas, padrões ou partes específicas de objetos.
- Os ***kernels*** ***aprendem*** a ***detectar*** as características mais relevantes para a tarefa específica em mãos.
  - Eles ***aprendem***, a partir do ***conjunto de treinamento***, os valores dos ***elementos do tensor***, necessários para ***detectar um determinada característica***.

Operação de convolução com 1 canal

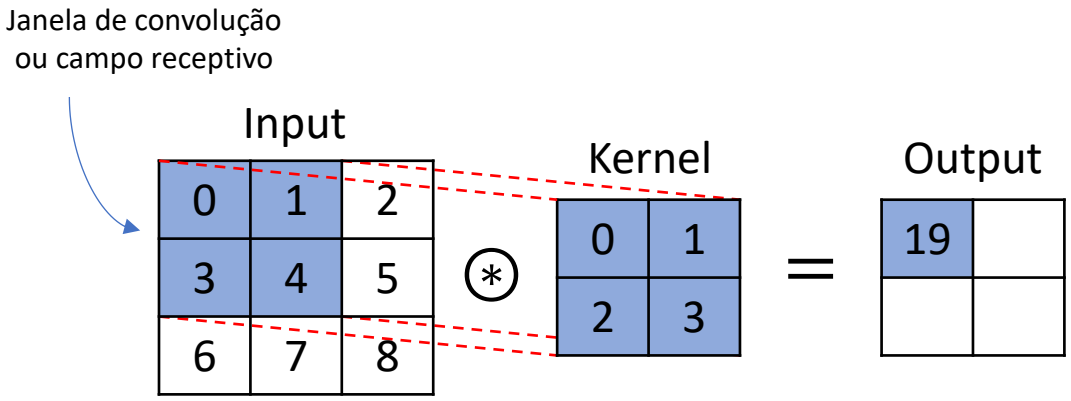


$$O(i,j) = \sum_m \sum_n I(i+m, j+n)\, K(m,n)$$

I : input
K: kernel
O: output

- Vamos ignorar múltiplos canais por enquanto e ver como uma operação de convolução funciona com dados bidimensionais.
- O símbolo ⊛ representa a operação de "convolução".
- A saída da operação de convolução é chamada de ***feature map*** ou ***mapa de características***.
- O operação é representada pela equação ao lado.

Operação de convolução com 1 canal



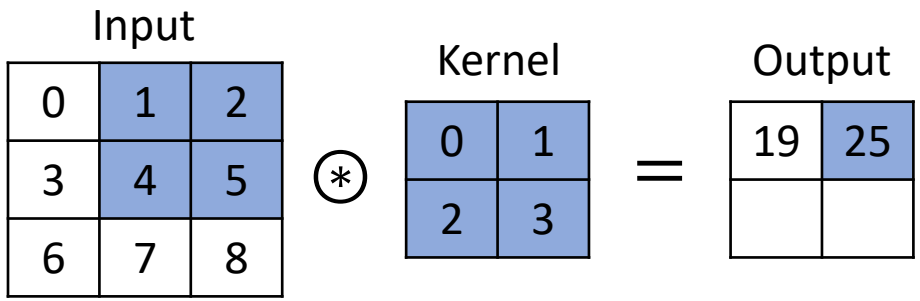
- Ao calcular a convolução, começamos com a ***janela de convolução*** no canto superior esquerdo do tensor de entrada.

$$O(i,j) = \sum_m \sum_n I(i+m, j+n)\, K(m,n)$$

$$= 0 * 0 + 1 * 1 + 3 * 2 + 4 * 3$$

$$= 19$$

Operação de convolução com 1 canal



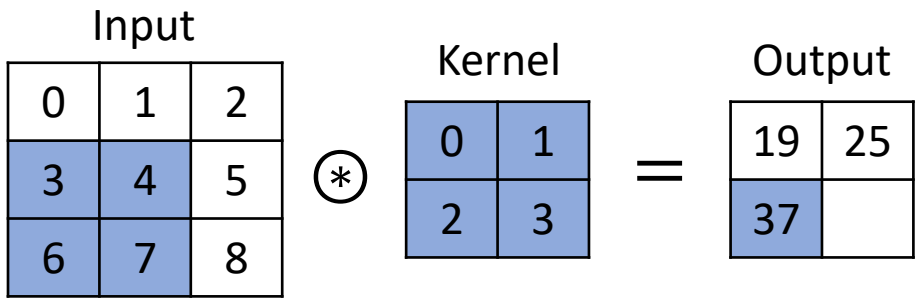
- Em seguida, deslizamos a janela, por exemplo, um elemento para a direita.

$$O(i,j) = \sum_m \sum_n I(i+m, j+n)\, K(m,n)$$

$$= 1 * 0 + 2 * 1 + 4 * 2 + 5 * 3$$

$$= 25$$

Operação de convolução com 1 canal



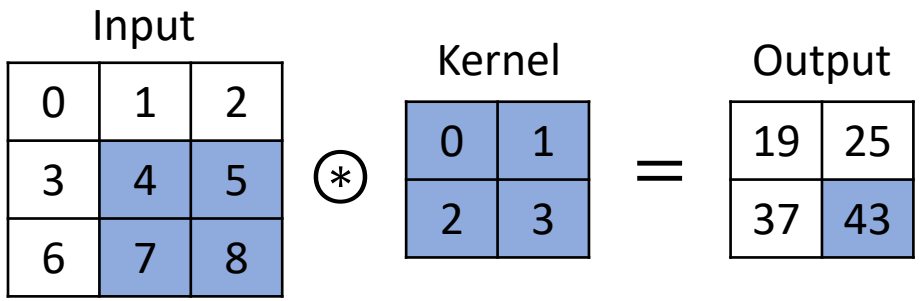
- Ao chegar-se ao final das colunas do tensor de entrada, volta-se ao seu início, deslizando a janela, por exemplo, um elemento para baixo, ou seja, uma linha.

$$O(i,j) = \sum_m \sum_n I(i+m, j+n)\, K(m,n)$$

$$= 3 * 0 + 4 * 1 + 6 * 2 + 7 * 3$$

$$= 37$$

Operação de convolução com 1 canal



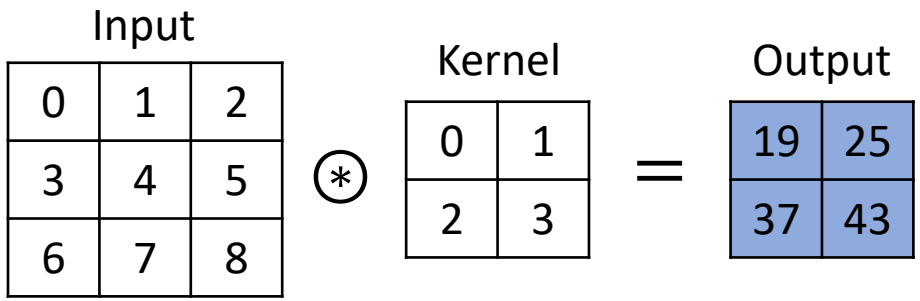
- Em seguida, deslizamos a janela um elemento para a direita.
- Esse processo se repete até que a janela de convolução tenha percorrido todo o tensor de entrada.

$$O(i,j) = \sum_m \sum_n I(i+m, j+n)\, K(m,n)$$

$$= 4 * 0 + 5 * 1 + 7 * 2 + 8 * 3$$

$$= 43$$

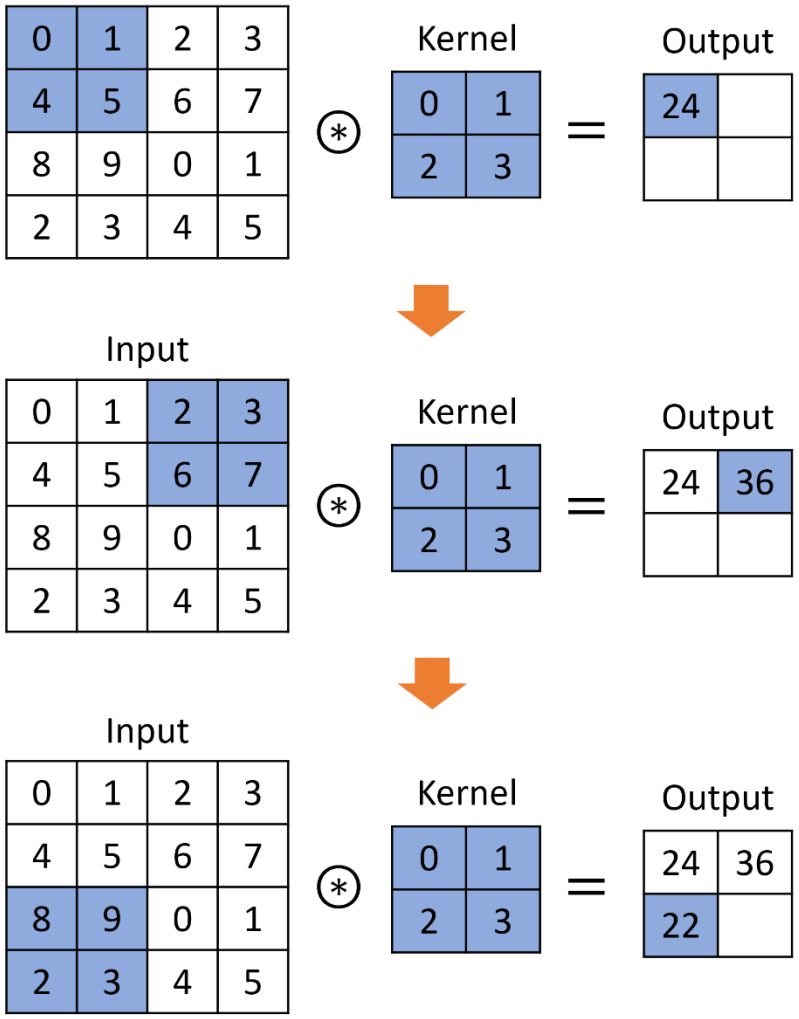
Mapa de características



$$O(i,j) = \sum_m \sum_n I(i+m, j+n)\, K(m,n)$$

- Lembrando que o ***objetivo*** dos ***kernels*** é ***extrair características***.
- Portanto, o resultado da operação de convolução é chamado de ***mapa de características ou de ativação.***
- É chamado assim, pois ***fornece as respostas/ativações desse filtro em cada posição espacial*** da imagem.
- Ele mostra ***onde*** e com que ***intensidade*** uma característica específica (detectada pelo filtro) está presente na entrada.

Stride



- No exemplo anterior, deslizamos a janela um elemento por vez.
- Porém, às vezes, seja por ***eficiência computacional*** ou porque desejamos ***reduzir a resolução***, movemos a janela mais de um elemento por vez.
- Esse parâmetro é chamado de *stride*.
- No exemplo ao lado, o *stride* é de 2 para deslizamentos ao longo das colunas e linhas.
  - Porém, ele pode ser diferente para deslocamentos ao longo das linhas e colunas.

Convolução ou correlação cruzada?

$$O(i,j) = K(i) \circledast I(j)$$
$$= \sum_m \sum_n K(i-m, j-n) I(m,n)$$

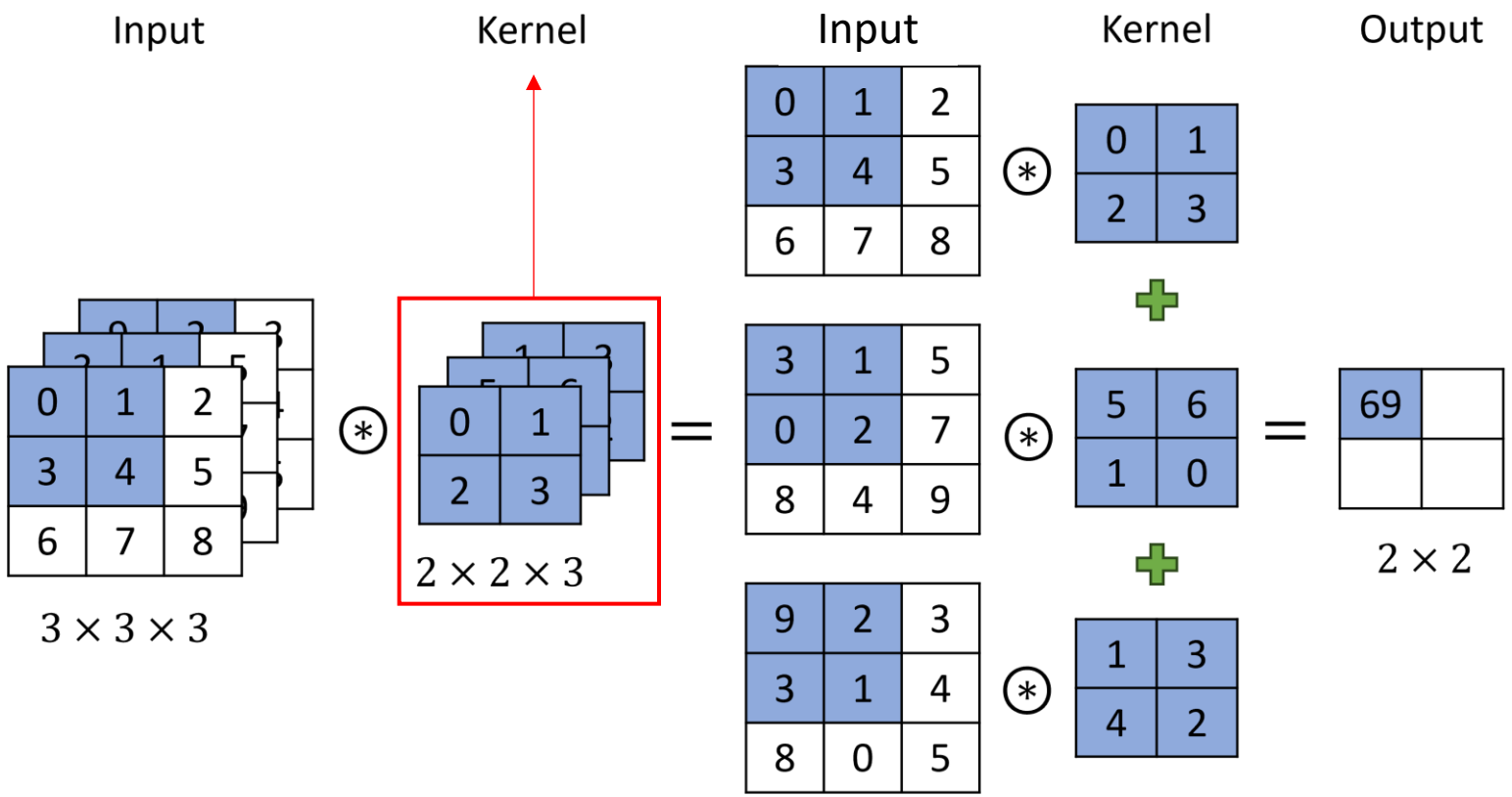
Convolução

$$O(i,j)$$
$$= \sum_m \sum_n K(i+m, j+n) I(m,n)$$

Correlação cruzada

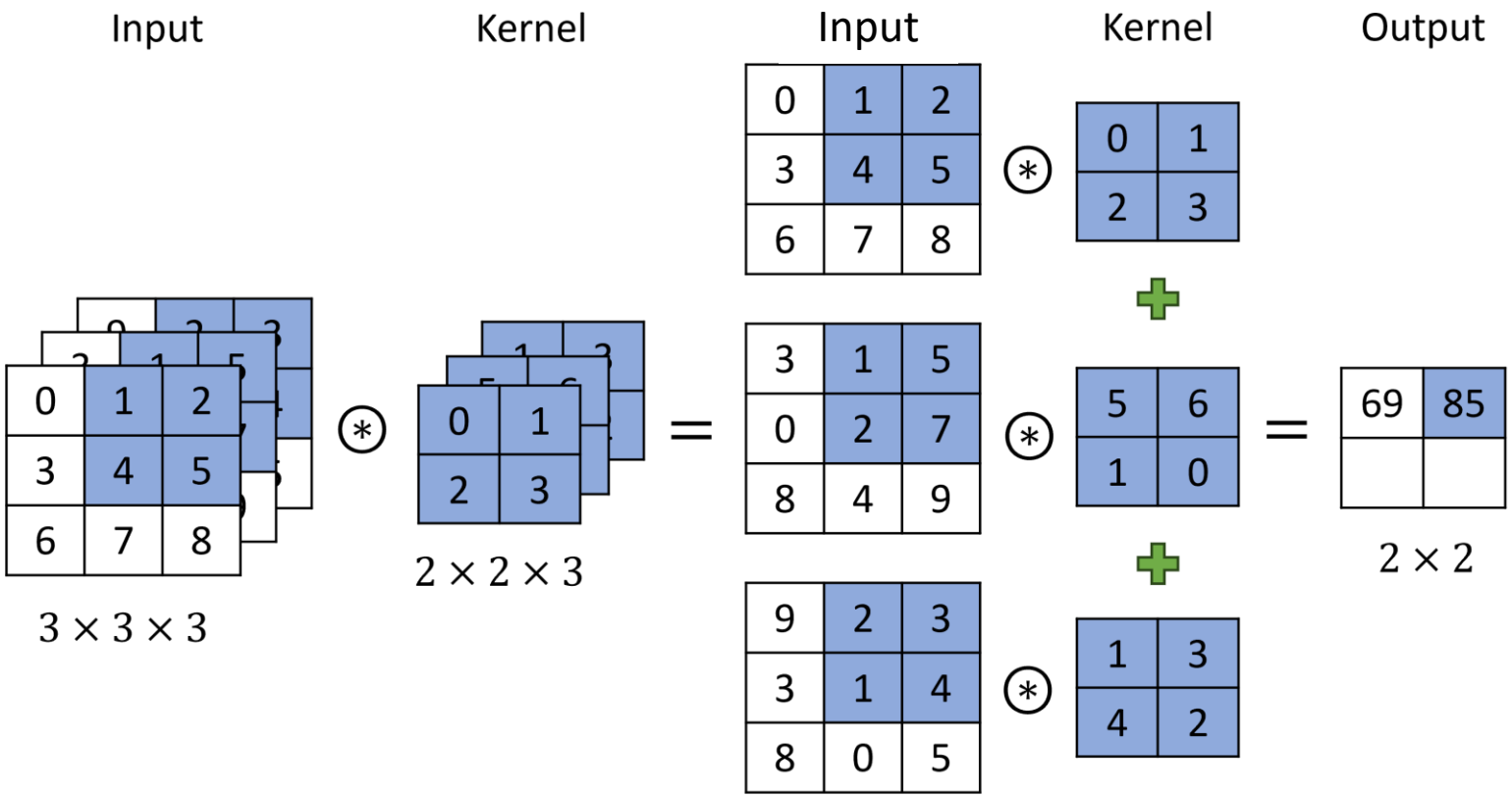
- As operações em uma CNN, embora sejam chamadas de convoluções, são implementadas como ***correlações cruzadas*** na maioria das bibliotecas.
  - Correlações são mais ***eficientes*** (sem inversão) ***e simples de serem implementadas***.
- Ao contrário da operação de convolução, as CNNs ***não invertem o kernel*** (ou o sinal de entrada).
- Porém, ***isso não importa***, pois os ***kernels*** são ***aprendidos*** e podem se adaptar tanto à correlação cruzada quanto à convolução.

Operação de convolução com 3 canais

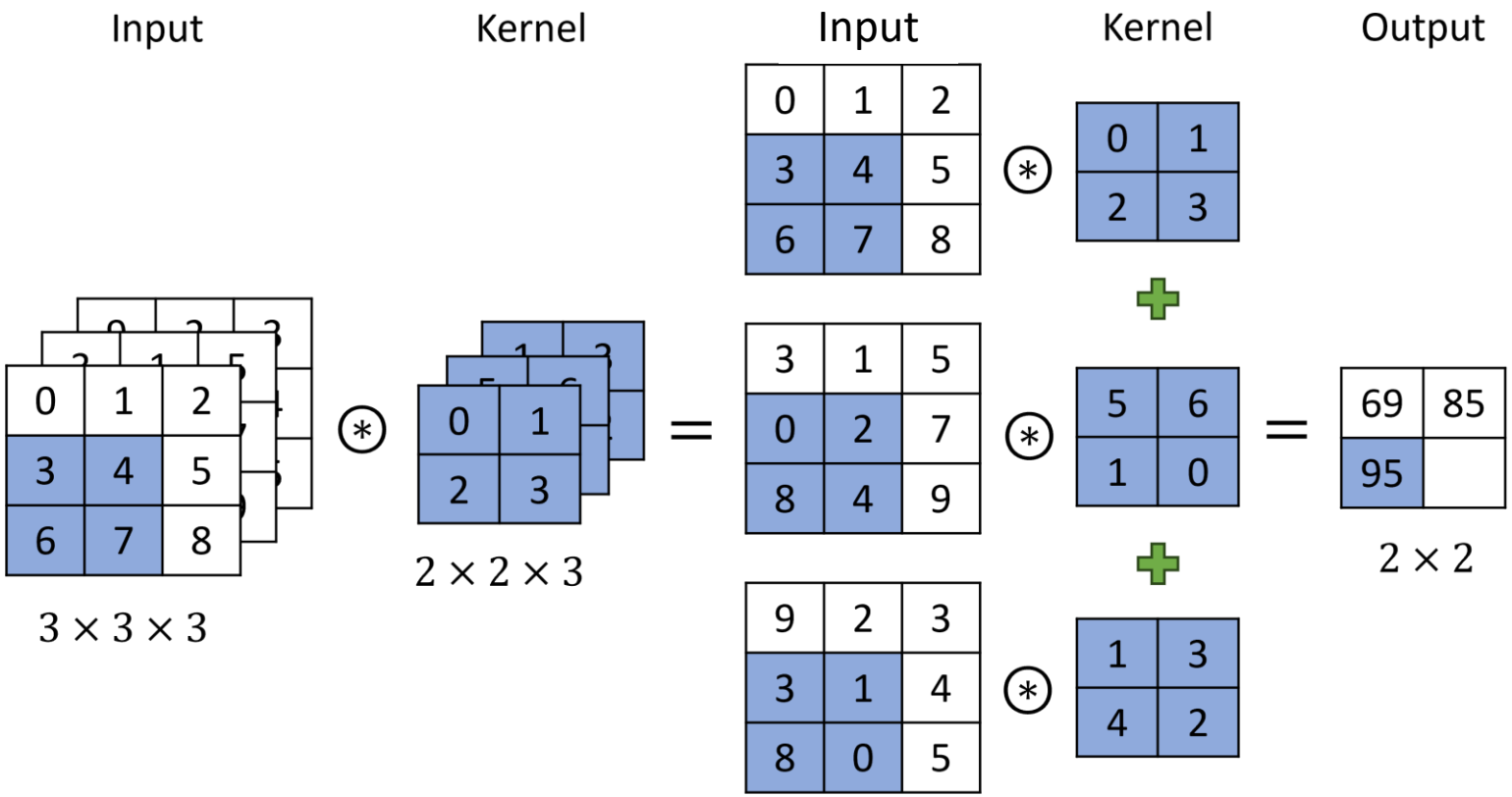


- Em geral, se a imagem tem 3 dimensões, o ***kernel*** também terá 3 dimensões.
- Para entender a operação, podemos ***dividi-la em 3 operações de convolução separadas*** que têm seus resultados somados ao final para gerar a saída.
- Usando um *stride* = 1, temos.

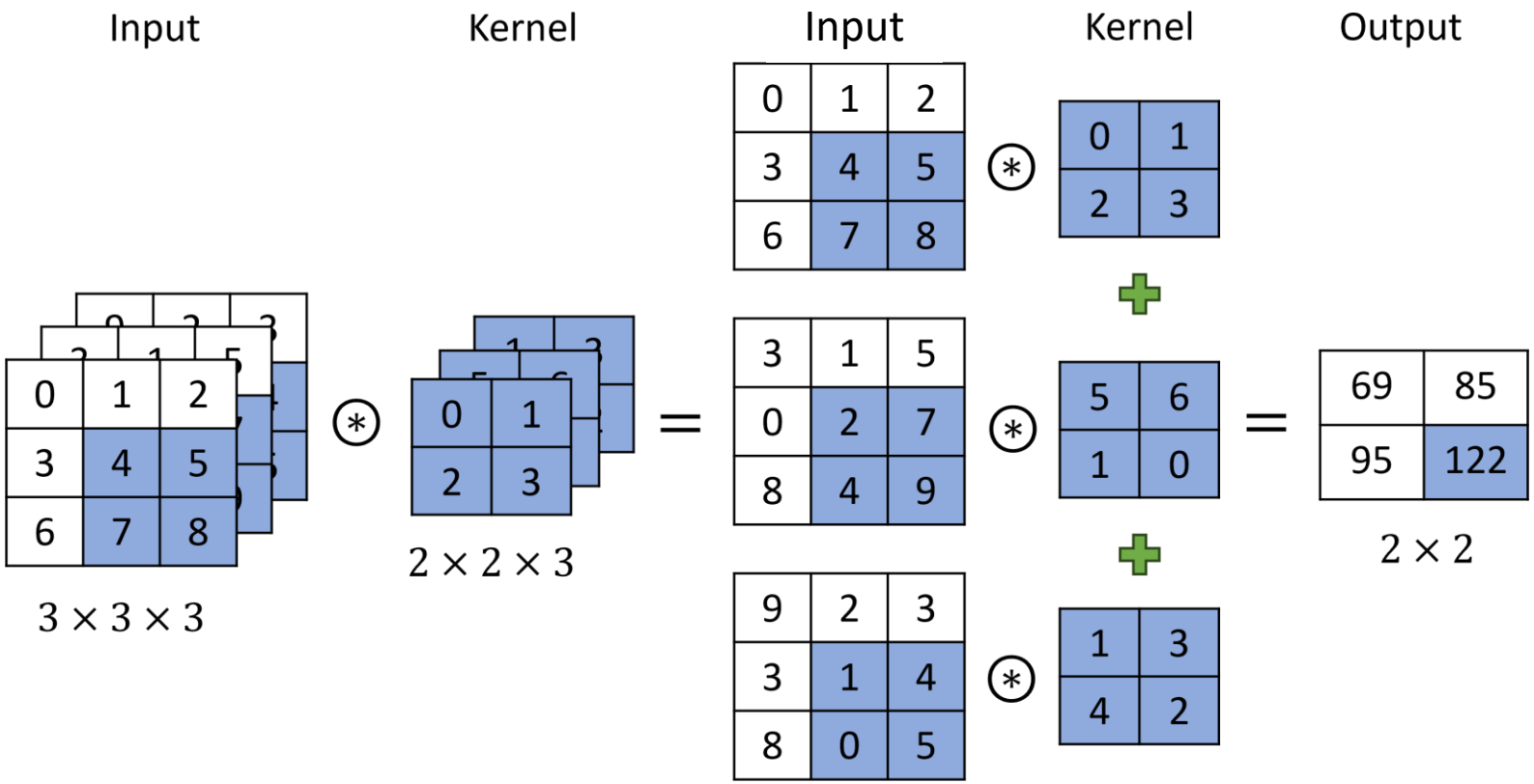
Operação de convolução com 3 canais



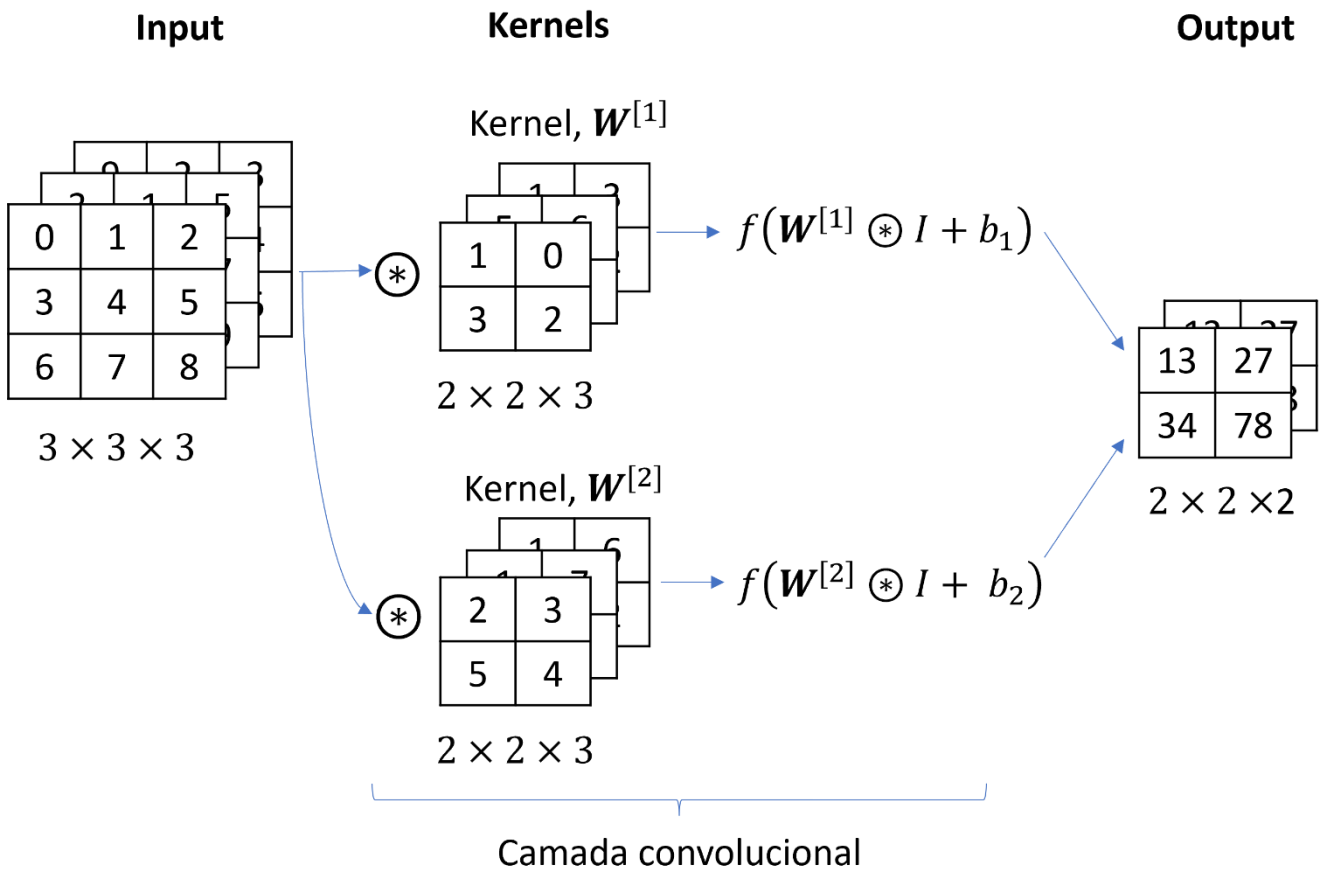
Operação de convolução com 3 canais



Operação de convolução com 3 canais

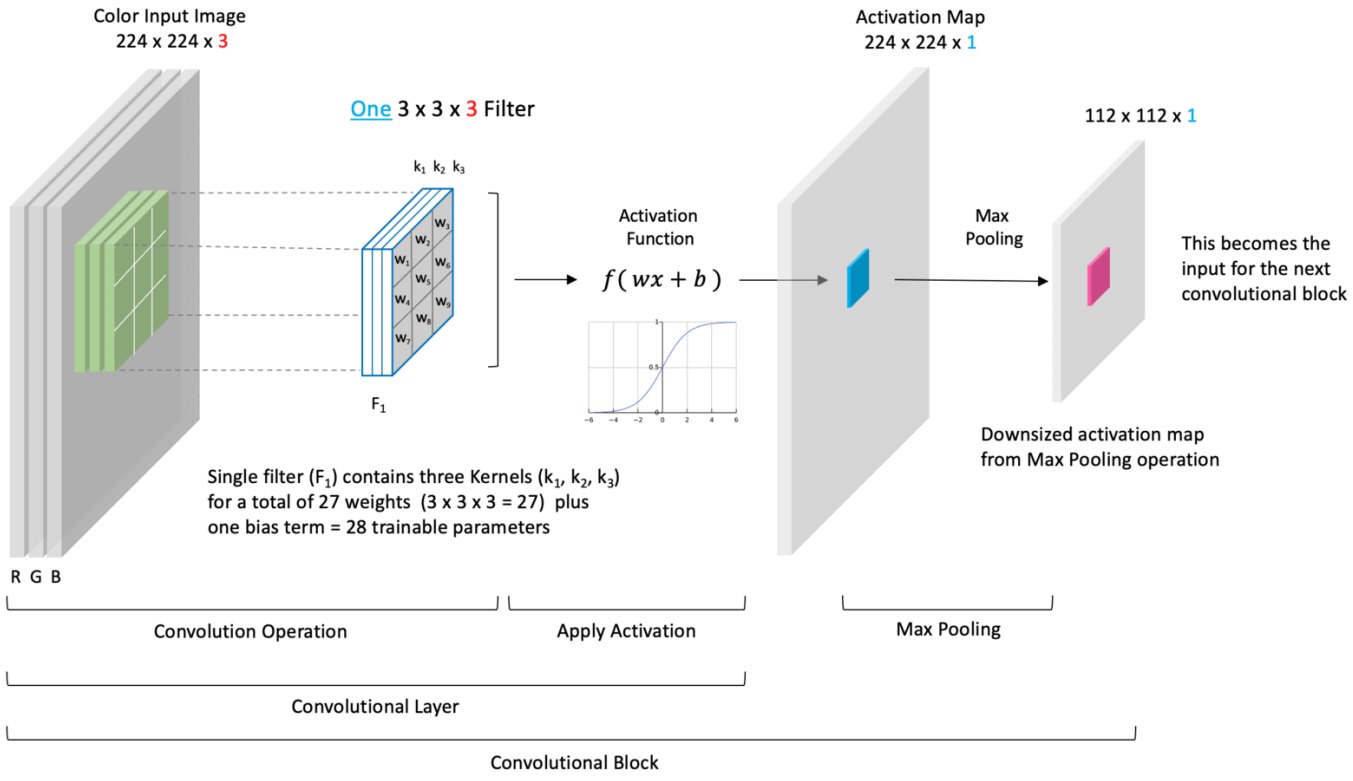


Kernels diferentes para características diferentes



- Em geral, cada ***camada convolucional possui vários kernels***.
- Cada **kernel detecta uma característica diferente**.
- O resultado da convolução tem um valor de *bias* somado a ele e o resultado é passado por uma função de ativação, $f(.)$, (e.g., ReLU).
- A saída da camada é o resultado do ***empilhamento de várias matrizes.***

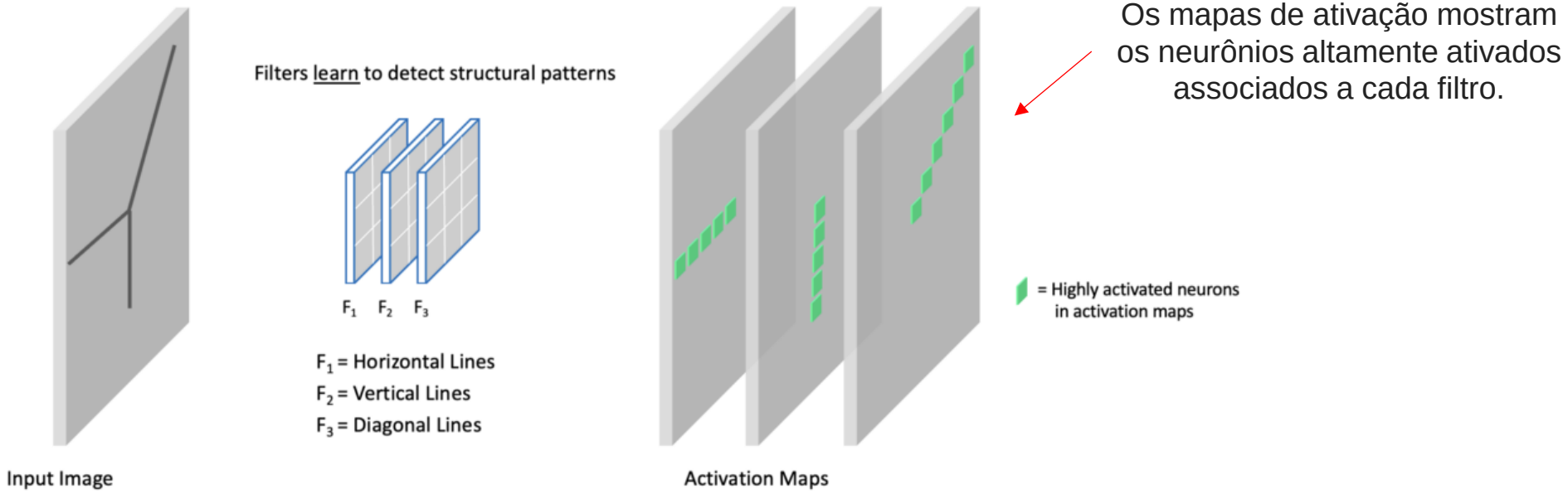
Kernels diferentes para características diferentes



OBS.: assume-se uma configuração de preenchimento que mantém o tamanho espacial dos dados em camadas convolucionais.

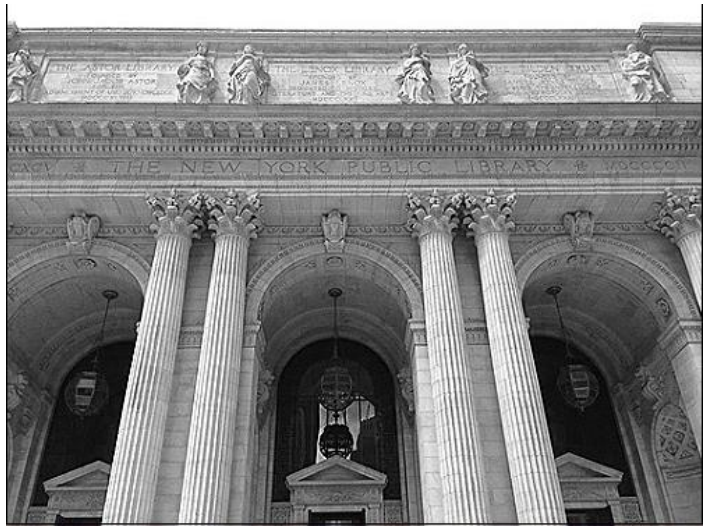
- A saída de uma camada de convolução é chamada de ***mapa de características (ou ativações)***.
- Ele é o resultado da aplicação das operações de convolução e ativação ***a cada trecho da entrada da camada*** (e.g., imagem).
- A saída do bloco é obtida aplicando-se uma operação de ***subamostragem*** ao mapa de características.

Filtros aprendem a detectar estruturas



- A imagem de entrada apresenta vários componentes lineares (bordas).
- Temos uma camada convolucional com três filtros.
- Cada filtro aprende a detectar diferentes elementos estruturais (i.e., linhas horizontais, linhas verticais e linhas diagonais).

Aplicando *kernels* a imagens



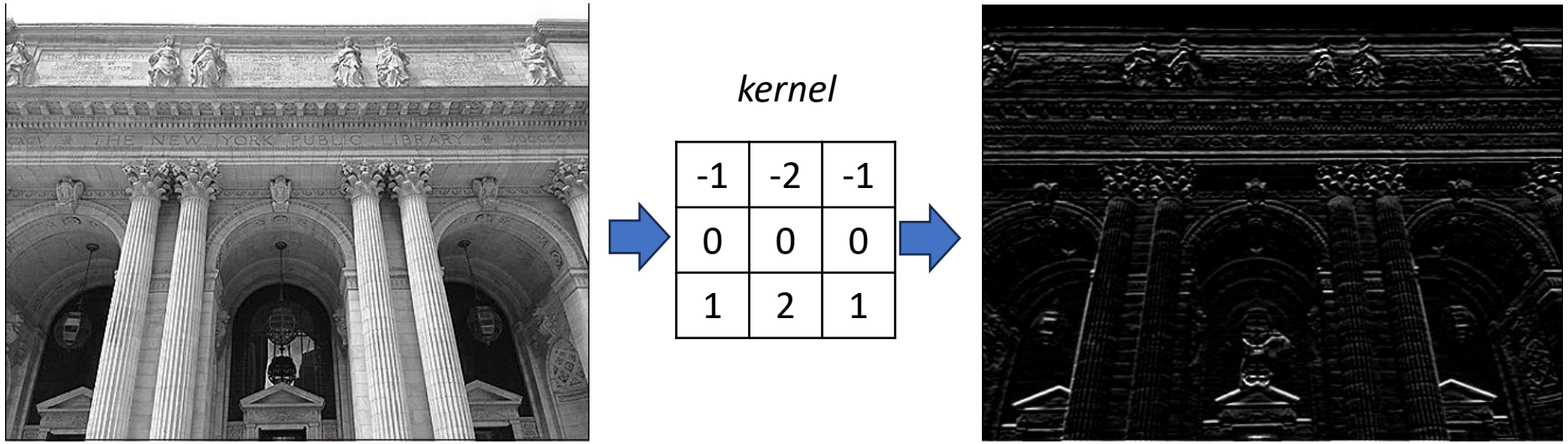
kernel

-1	0	1
-2	0	2
-1	0	1



- Considerem a imagem à esquerda.
- Se aplicarmos o *kernel* mostrado, obteremos os resultados à direita.
- Ele ***realça muito as linhas verticais e escurece todo o resto***.
- Portanto, podemos considerar este *kernel* como um ***detector de linhas verticais***.

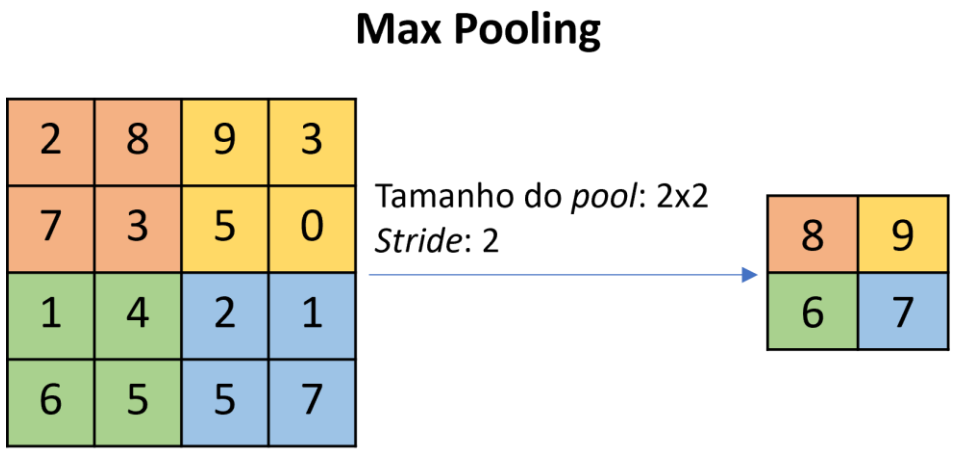
Aplicando *kernels* a imagens



- De forma similar, este outro *kernel* pode ***detectar linhas horizontais***, escurecendo quase tudo na imagem que não seja uma linha horizontal.
- Ao aplicar *kernels* como esses, podemos ***remover quase tudo***, ***exceto uma característica distinta***.
- Esse processo é chamado de ***extração de características***.
  - Processo que determina as partes mais importantes de uma imagem.

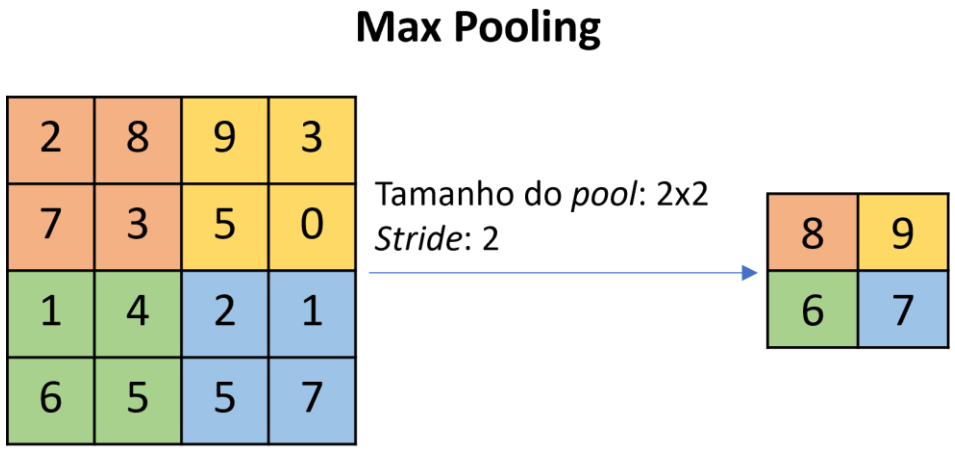
fonte: https://setosa.io/ev/image-kernels/

Camada de *Pooling* (ou subamostragem)



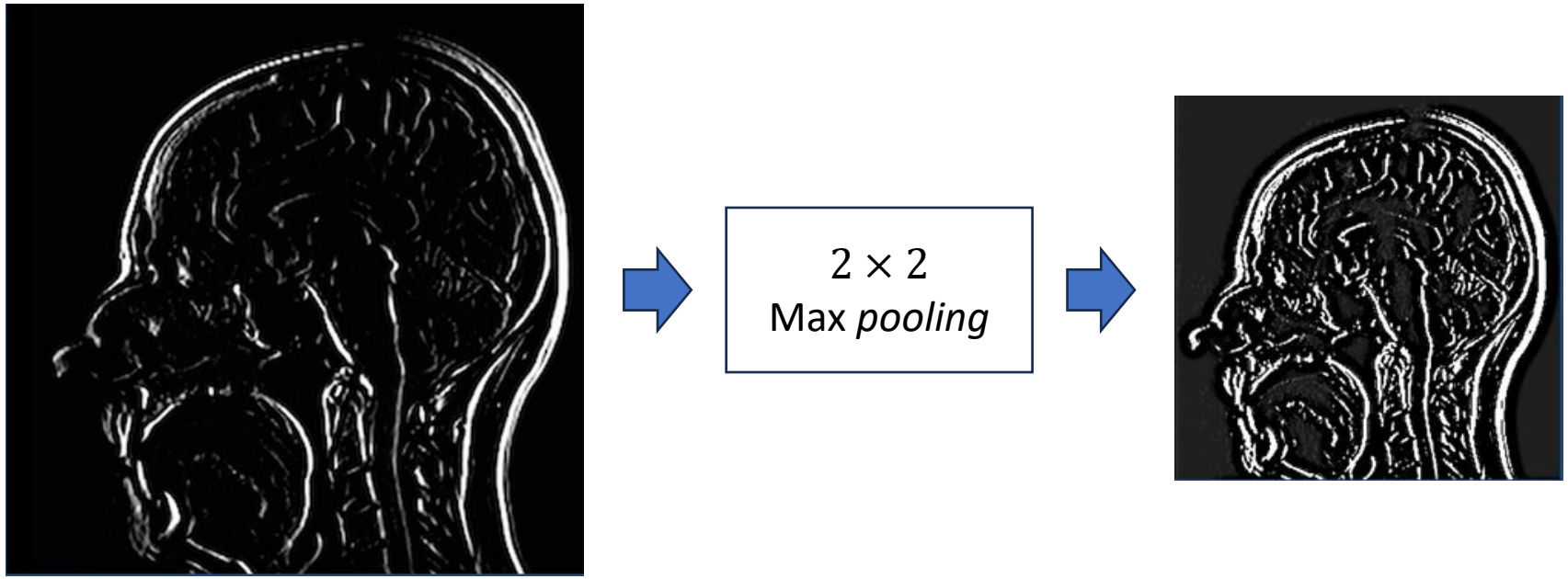
- Aplicada, em geral, ***após uma camada de convolução***.
- Ela ***subamostra*** sua entrada.
- O ***objetivo*** da subamostragem é ***reduzir a carga computacional***, o ***uso de memória*** e o ***número de parâmetros*** (limitando assim o risco de ***sobreajuste***).
- Além disso, ela ***ajuda a tornar a rede mais robusta a pequenas mudanças na posição das características***, o que é útil em tarefas de reconhecimento de objetos.

Camada de *Pooling* (ou subamostragem)



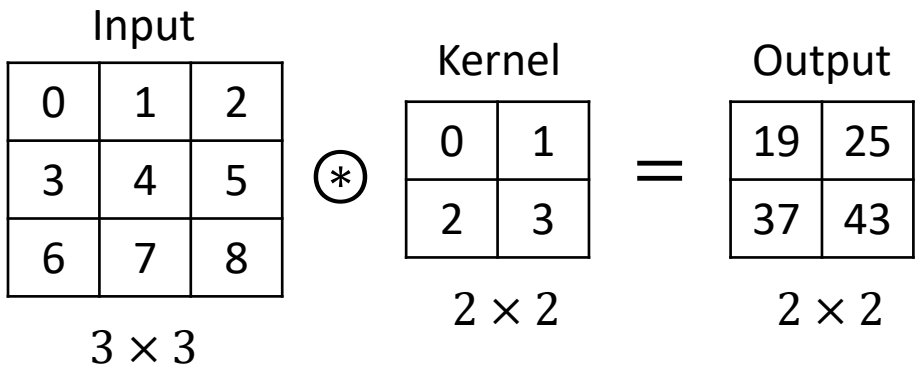
- A maneira ***mais comum*** de subamostrar é aplicar uma ***operação*** $\max(.)$ ao resultado de cada *kernel*.
- Somente o ***valor máximo de entrada em cada campo receptivo*** da camada de *pooling* ***passa para a próxima camada***, enquanto as ***outras entradas são descartadas***.
- O *pooling* é ***normalmente aplicado a cada canal de entrada de forma independente***, de forma que a ***profundidade de saída seja igual a de entrada***.

Camada de *Pooling* (ou subamostragem)



- Adicionalmente, as camadas de *pooling* ajudam a ***capturar e reter as características mais importantes***, ao mesmo tempo que ***descartam informações menos relevantes ou ruidosas***.
- Portanto, elas ***compactam*** os dados ***sem perder as características importantes***.

Padding ou preenchimento

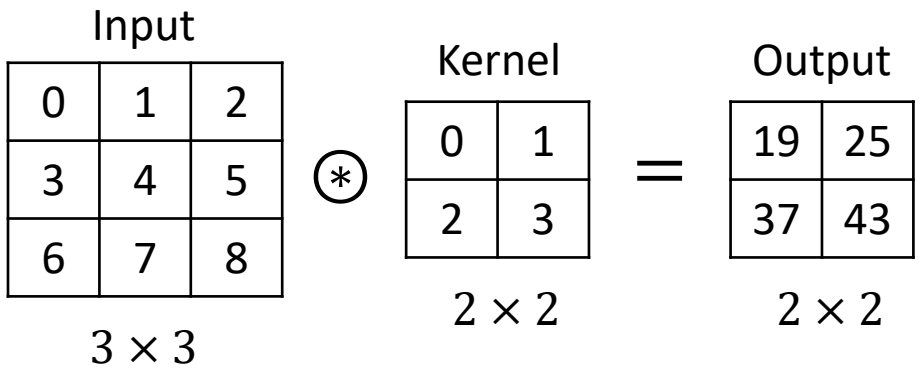


$$n_{out} = \left\lfloor \frac{n_{in} + 2p - k}{s} \right\rfloor + 1$$

n_{out} : dimensão da saída
n_{in} : dimensão da entrada
k : dimensão do kernel
p : quantidade camadas de *padding*
s : tamanho do *stride*
$\lfloor x \rfloor$: função piso retorna o maior inteiro menor ou igual a x.

- ***Depois de aplicar muitas convoluções*** sucessivas, as ***imagens tendem*** a se ***tornarem*** consideravelmente ***menores*** do que as da entrada.
- Se temos uma imagem de entrada com 240×240 *pixels*, após dez camadas de convolução 5×5 com *stride* igual a 1, ela é reduzida para 200×200 *pixels*.
- Isso reduz a imagem em ≈17%.

Padding ou preenchimento

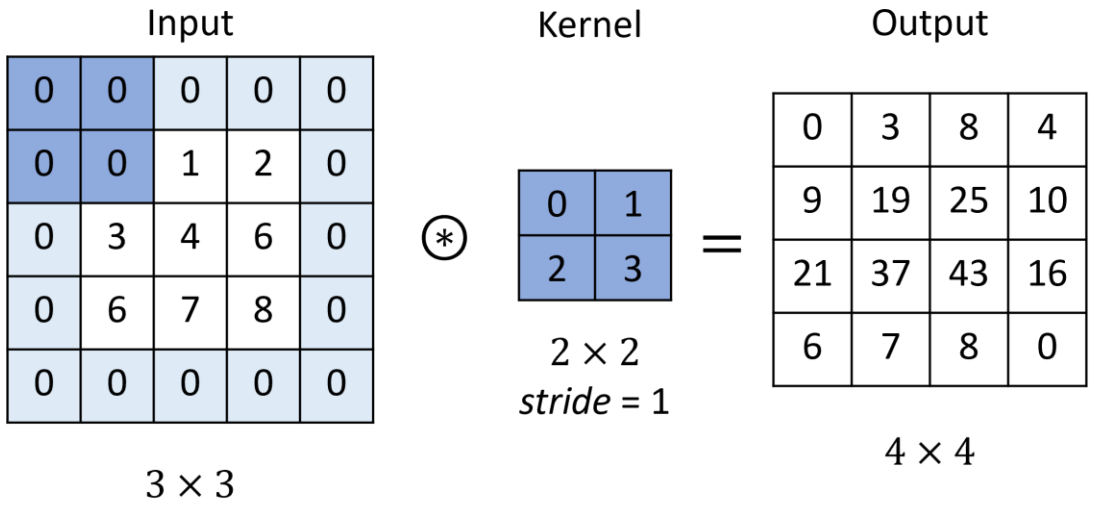


$$n_{out} = \left\lfloor \frac{n_{in} + 2p - k}{s} \right\rfloor + 1$$

n_{out} : dimensão da saída
n_{in} : dimensão da entrada
k : dimensão do kernel
p : quantidade camadas de *padding*
s : tamanho do *stride*
$\lfloor x \rfloor$: função piso retorna o maior inteiro menor ou igual a x.

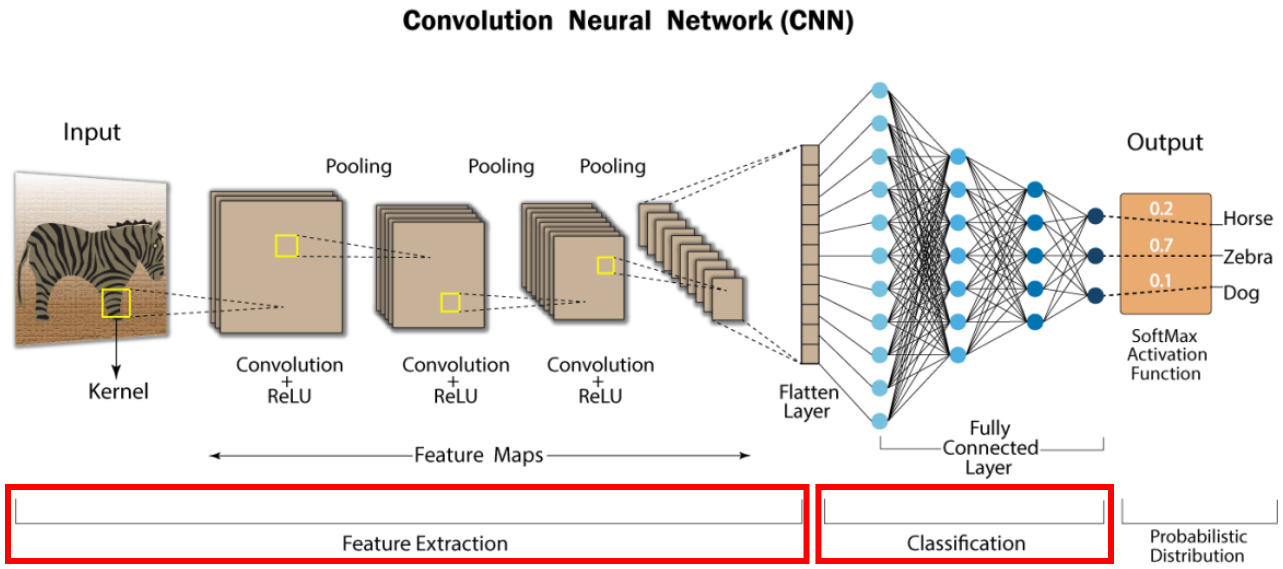
- Além da redução, a convolução faz com que qualquer ***informação interessante nas bordas da entrada (e.g., imagem) seja subutilizada***.
- Os *pixels* das bordas da entrada de uma camada são ***subutilizados***, i.e., participam de menos convoluções que os pixels centrais.
- O *padding* garante que todos os pixels contribuam igualmente para a extração de características.

Padding ou preenchimento



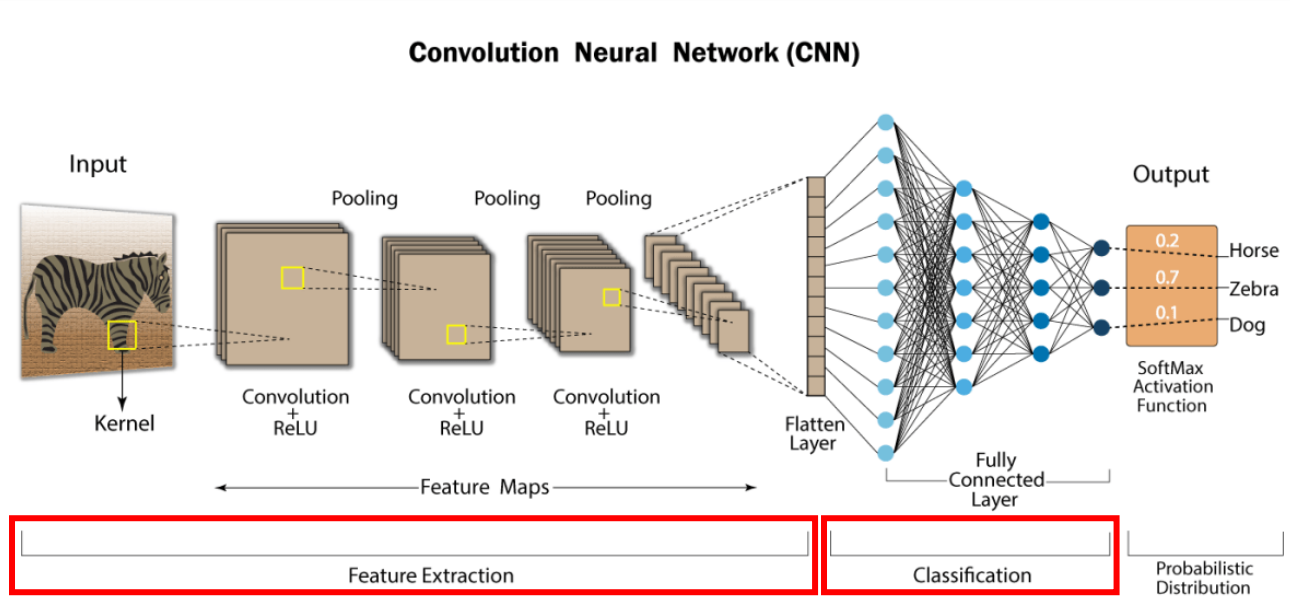
- O ***padding*** é usado para ***controlar o tamanho dos mapas de características*** após uma camada convolucional.
- ***Pixels de preenchimento são adicionados ao redor da borda da imagem*** de entrada, aumentando assim seu tamanho efetivo.
  - Normalmente, os ***pixels são feitos iguais a zero***.
- Em tarefas de ***classificação de imagens***, é comum aplicar *padding* nas ***camadas iniciais para preservar informações de borda***, enquanto ***camadas finais não o aplicam*** para reduzir a dimensionalidade.

Camadas densas



- Depois das camadas convolucionais, as CNN podem apresentar ***algumas camadas densas***.
- Elas são responsáveis por ***realizar a classificação (ou regressão)*** propriamente dita.
- As camadas densas ***recebem as características extraídas pelas camadas convolucionais e combinam essas informações para aprender padrões*** complexos e realizar a tarefa de classificação ou regressão.

Os filtros são aprendidos!



- Nós não precisamos definir os filtros manualmente.
- Em vez disso, durante o ***treinamento***, as ***camadas convolucionais aprenderão automaticamente*** os ***kernels*** mais úteis para sua tarefa, e as camadas densas aprenderão a combiná-los em padrões mais complexos.

Explorando CNNs

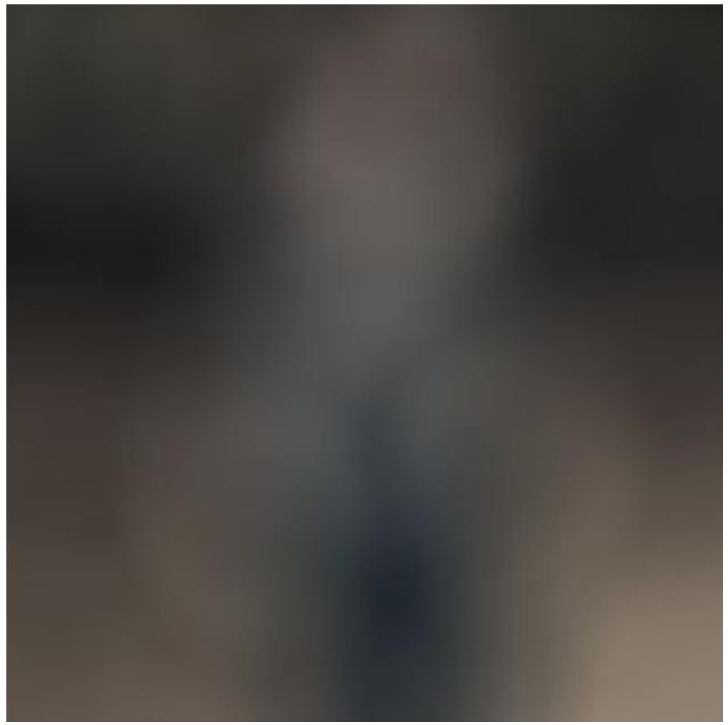
- CNN Explainer
  - https://poloclub.github.io/cnn-explainer/
- ConvNetJS MNIST demo
  - https://cs.stanford.edu/people/karpathy/convnetjs/demo/mnist.html
- ConvNetJS CIFAR-10 demo
  - https://cs.stanford.edu/people/karpathy/convnetjs/demo/cifar10.html

Atividades

- Exercício: Redes neurais convolucionais

Anexo I: Exemplo de classificação de imagens

Exemplo de classificação de imagens



- Para entender melhor o que uma camada de convolução faz, vamos considerar um exemplo simples.
- Vamos supor que ***não conhecemos o conteúdo da imagem*** ao lado.
- Isso foi simulado deixando-a embaçada.

Exemplo de classificação de imagens



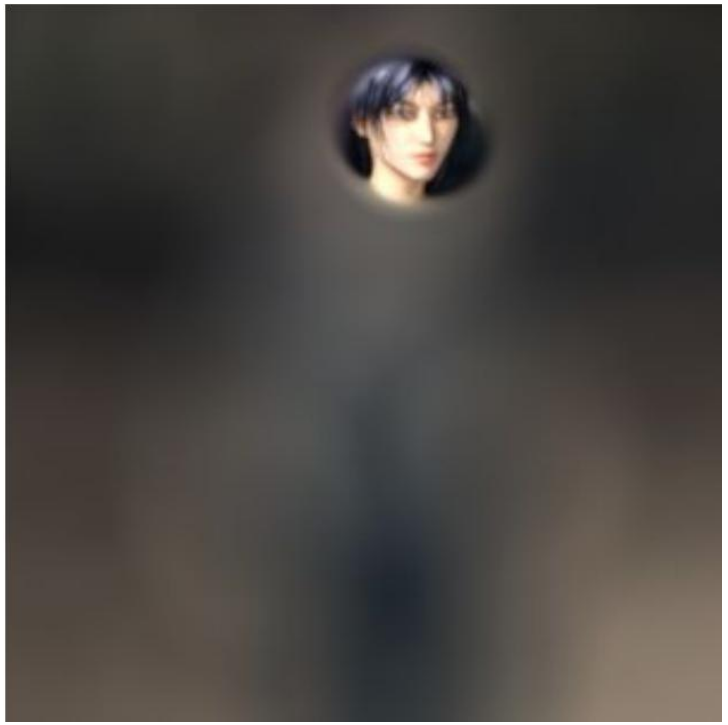
- Então, digamos que haja um ***filtro ou um conjunto de filtros*** que ao serem passados sobre a imagem ***extraem as características*** mostradas ao lado.
- Como podemos ver, ***são duas formas verticais***, que se parecem com pernas humanas.

Exemplo de classificação de imagens



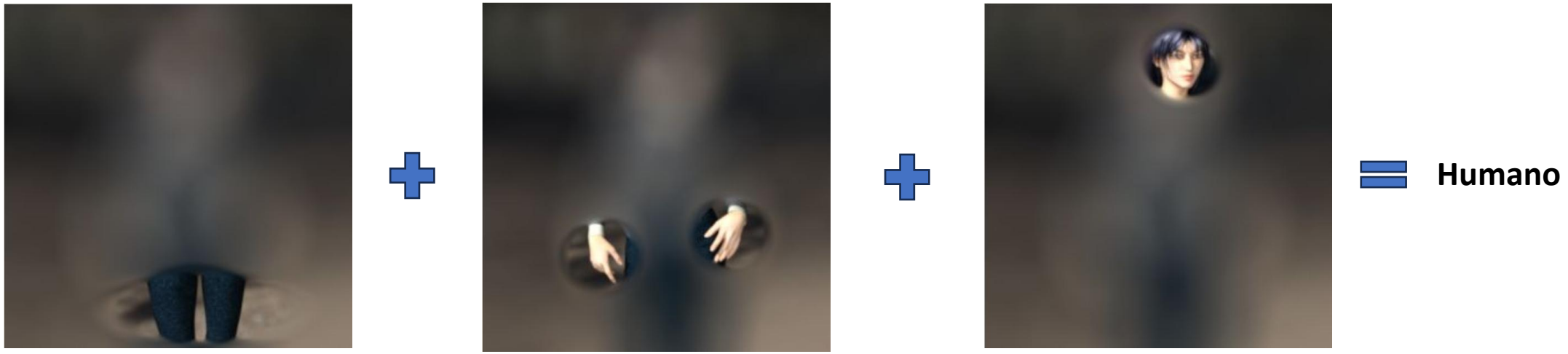
- Na sequência, ***outros filtros extraem as características*** mostradas ao lado.
- São pequenas formas cilíndricas que se projetam a partir de um cilindro maior.
- Nosso ***cérebro vê isso*** e ***instantaneamente classifica como mãos***.
- Porém uma rede neural em treinamento ainda não sabe disso.
- Ela apenas sabe que um filtro pode extrair essas características.

Exemplo de classificação de imagens



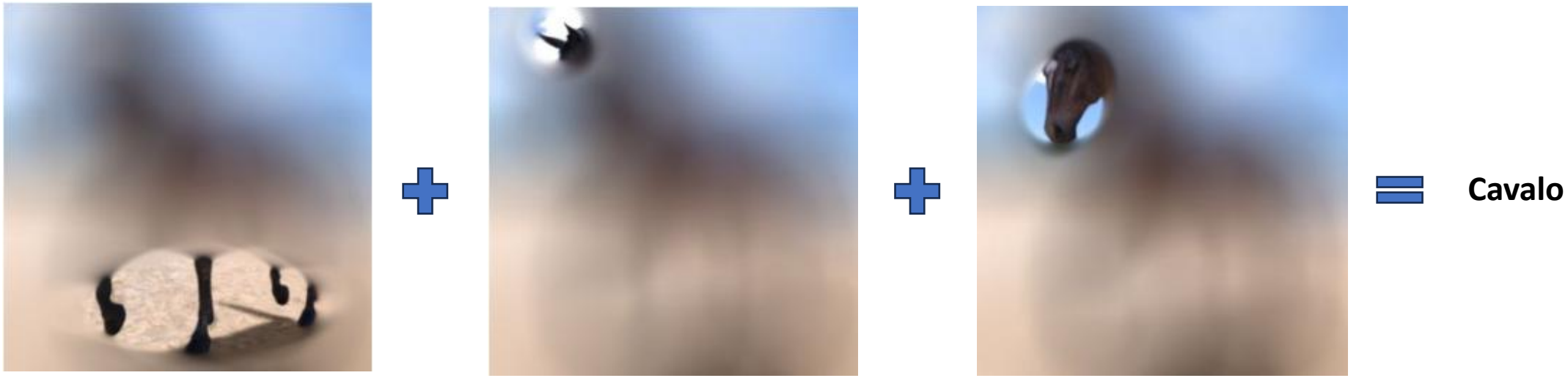
- Em seguida, ***outros filtros extraem um círculo com outros dois círculos menores, uma saliência e algumas formas paralelas***.
- Nosso cérebro reconhece esse conjunto de *pixels* como um rosto com olhos, boca e nariz.
- Mas, novamente, o modelo não tem contexto para decidir o que esses *pixels* são.
- Ele só sabe que um filtro específico pode extrair essas características.

Exemplo de classificação de imagens



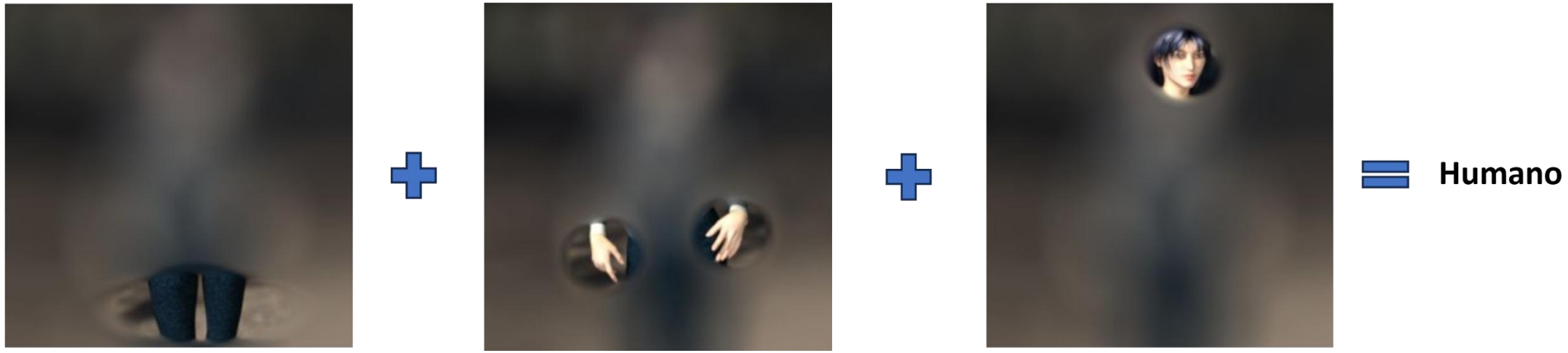
- Quando essas ***características estão presentes em uma imagem rotulada como humana***, a rede neural pode ser ***treinada para aprender filtros que extraem essas informações***.
- A rede ***pode combinar as características extraídas pelos três filtros*** (i.e., pernas, mãos e face) para detectar seres humanos em imagens inéditas.
  - A(s) camada(s) densa(s) faz(em) essa combinação.

Exemplo de classificação de imagens



- A ***mesma rede pode ser treinada para identificar*** diferentes ***características*** presentes nas imagens de ***cavalos***.
- Assim, ao ***aprender conjuntos de filtros que podem detectar humanos ou cavalos***, temos um ***modelo de visão computacional*** que pode ***lidar*** com ***imagens complexas*** e ***predizer o que há nelas***.

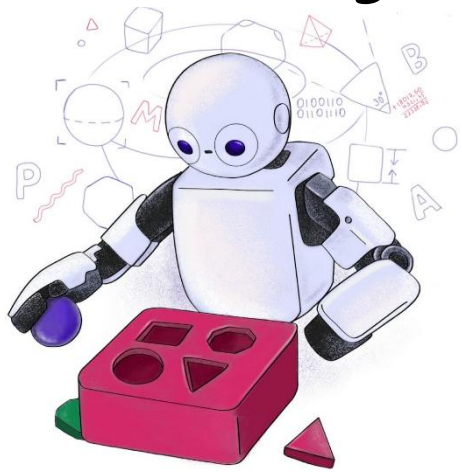
Observação



- Notem que usamos pernas, mãos e rostos como exemplos de características extraídas por um filtro.
- Essas são formas reconhecíveis pelo nosso cérebro, então as utilizamos para ilustrar o conceito.
- Porém, as redes convolucionais, muito provavelmente ***aprenderão características imperceptíveis*** a nós humanos.

TP557 - Tópicos avançados em IoT e Machine Learning:

Prevenindo o sobreajuste



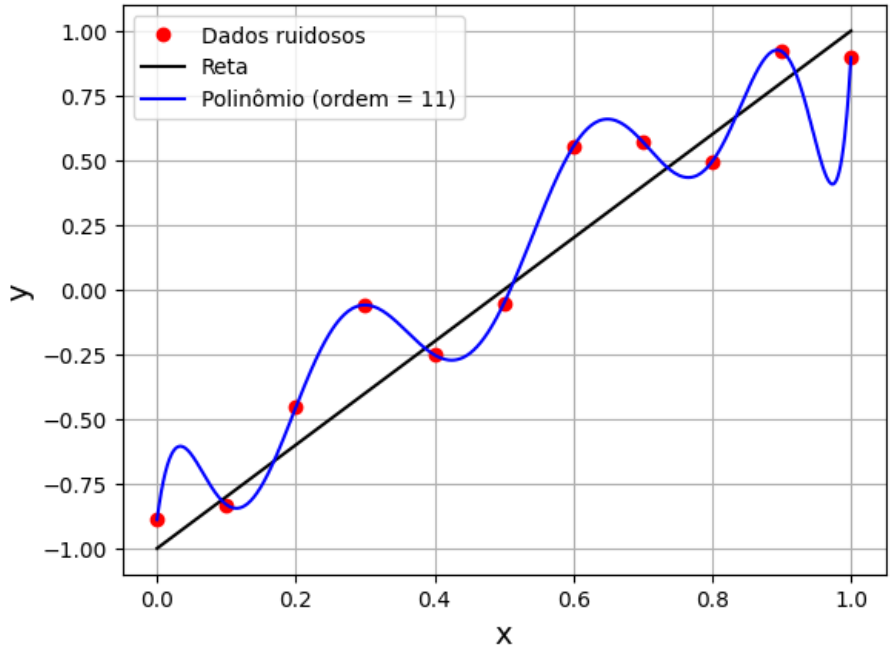
Inatel

Felipe Augusto Pereira de Figueiredo
felipe.figueiredo@inatel.br

O que vamos ver?

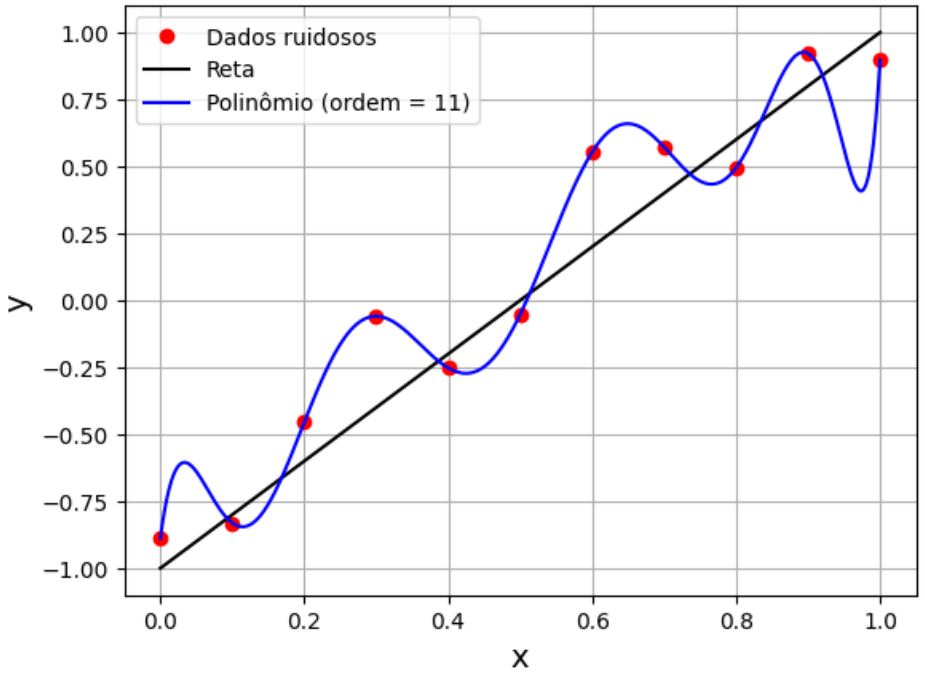
- Vimos anteriormente que o sobreajuste (*overfitting*) é um fenômeno indesejado que ocorre em modelos de aprendizado de máquina, no qual o modelo se ***ajusta excessivamente aos dados de treinamento***, em vez de generalizar bem para dados inéditos.
- Em outras palavras, o ***modelo "decora" os dados de treinamento capturando até mesmo a variação residual (e.g., ruído) presente nos dados*** em vez de aprender o padrão geral que pode ser aplicado a exemplos desconhecidos.
- Em geral, é ***mais comum um modelo*** se ***sobreajustar*** do que subajustar.
- Portanto, neste tópico, veremos algumas formas de se evitar o sobreajuste.

Sobreajuste



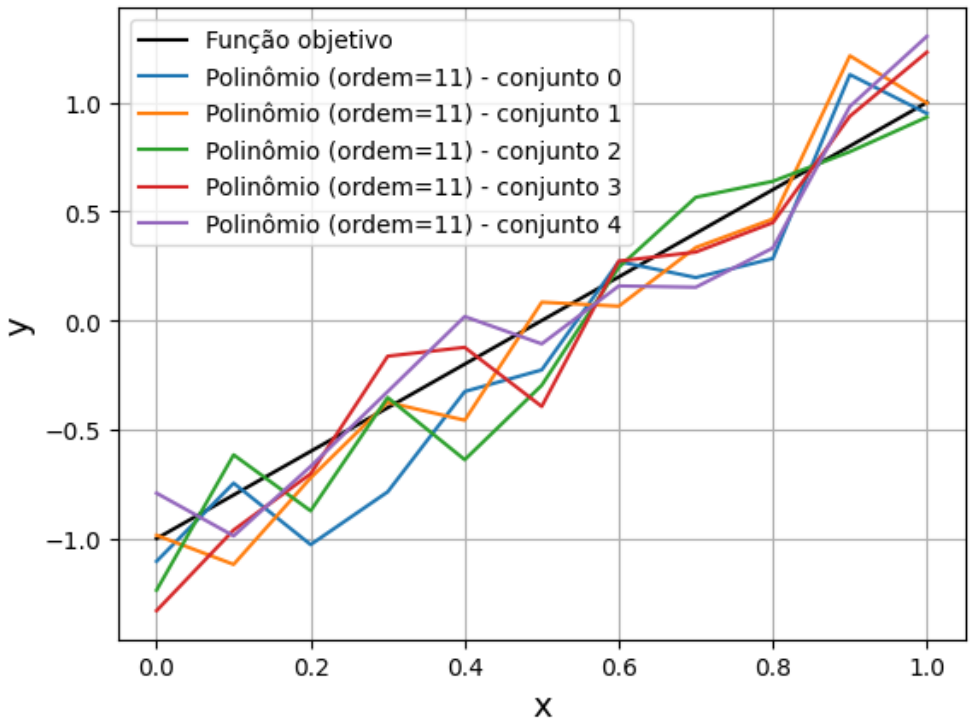
- A figura ao lado mostra 11 ***amostras ruidosas*** geradas a partir de uma reta, chamada de ***função objetivo***.
- Elas são aproximadas por uma (outra) ***reta*** e um ***polinômio de ordem 11***.
- Embora o ***polinômio se ajuste perfeitamente*** aos dados ruidosos, pode-se esperar que a ***reta generalize melhor***.
- Se as duas funções forem usadas para fazer predições além das 11 amostras, a ***reta*** deve obter ***melhores resultados no sentido da minimização do erro médio***.

Sensibilidade aos dados de treinamento



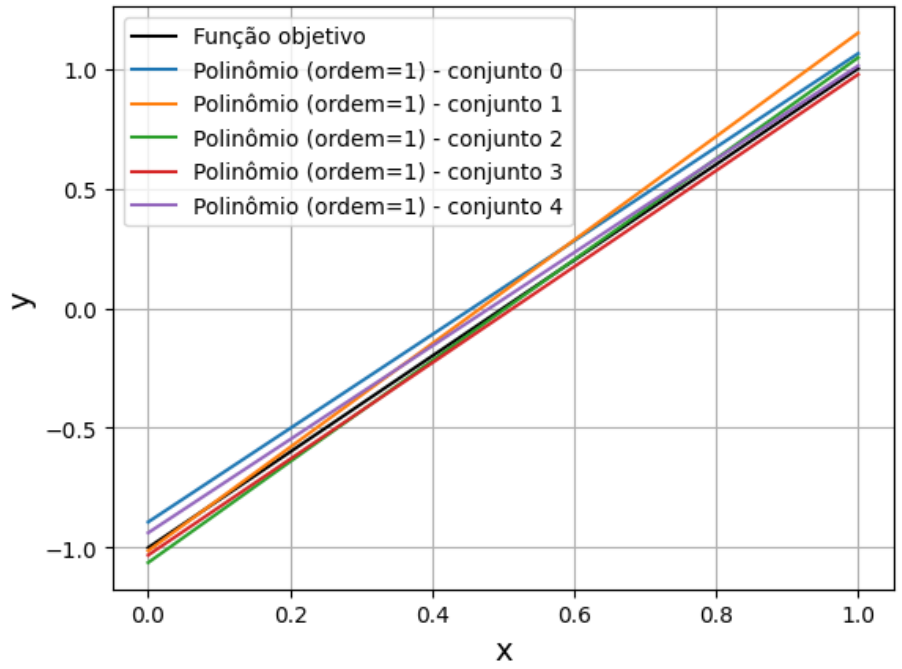
- Em essência, o ***sobreajuste*** faz com que um ***modelo extraia***, sem saber, parte da ***variação residual***, ***como se essa variação representasse o padrão geral por trás dos dados***.
  - ***Variação residual*** são ***padrões que não são significativos***, como por exemplo o ruído presente nas amostras.
- Um modelo ***muito flexível***, que se sobreajusta aos dados de treinamento, apresenta ***alta variância***.

Sensibilidade aos dados de treinamento



- A alta variância significa que o ***modelo*** é ***muito sensível às variações nos dados de treinamento***.
  - O ***modelo será diferente para cada conjunto de treinamento***.
  - Nesse caso, o modelo se ***distancia do padrão geral*** por trás dos dados.
- No exemplo ao lado ele irá se ***ajustar tão bem aos dados que vai aprender até o ruído presente nas amostras***.
  - Cada conjunto de treinamento ***corrompido com amostras ruidosas diferentes resultará em um modelo diferente***.

Complexidade (ou flexibilidade) ideal



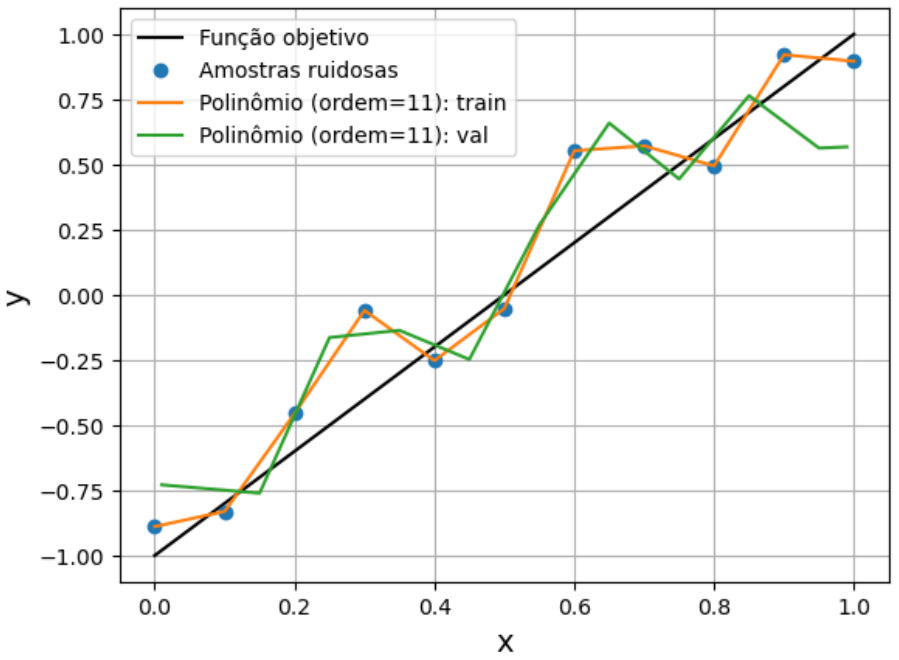
- O modelo mostrado na figura ao lado tem ordem apropriada (i.e., ordem 1).
- Percebam que ele ***não varia tanto***, como no exemplo anterior, ***quando se varia o ruído adicionado às amostras de treinamento***.
- Um modelo que apresente ***flexibilidade ideal*** terá ***baixa variância***, ou seja, ele sempre ***tenderá a capturar o padrão geral por trás das amostras***, ***mesmo que ruidosas***.

Exemplo: Overfitting.ipynb

Causas do sobreajuste

- **Modelo complexo:** Um modelo ***muito complexo***, por exemplo, com muitos parâmetros (i.e., pesos), tem uma ***alta capacidade de representação*** (ou flexibilidade) e pode ***se ajustar demais aos dados***.
- **Dados insuficientes:** Quando os ***dados de treinamento são limitados em quantidade***, o ***modelo pode não ter informações suficientes para generalizar*** adequadamente.
- **Ruído nos dados:** A presença de ***ruído ou outliers nos dados de treinamento*** pode fazer com que o ***modelo tente ajustar-se a essas variações aleatórias***.
  - O ruído é chamado de ***erro irredutível***, pois dificilmente nos livramos dele.

Como detectar o sobreajuste

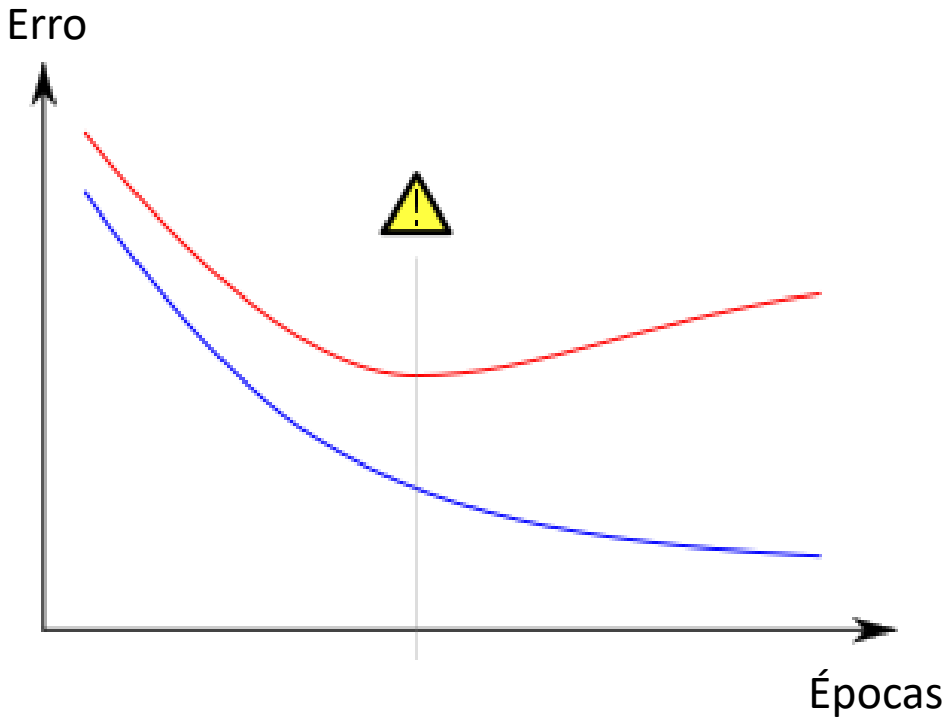


MSE train: 3.023264662910588e-24
MSE val: 0.18213360918420743

- **Conjunto de Validação:** ***Dividir o conjunto total*** de em um conjunto de treinamento e um de validação e ***avaliar o erro em ambos os conjuntos ao longo do treinamento***.
- Se o ***desempenho no conjunto de validação for significativamente pior*** do que no conjunto de treinamento, é um ***sinal de possível sobreajuste***.

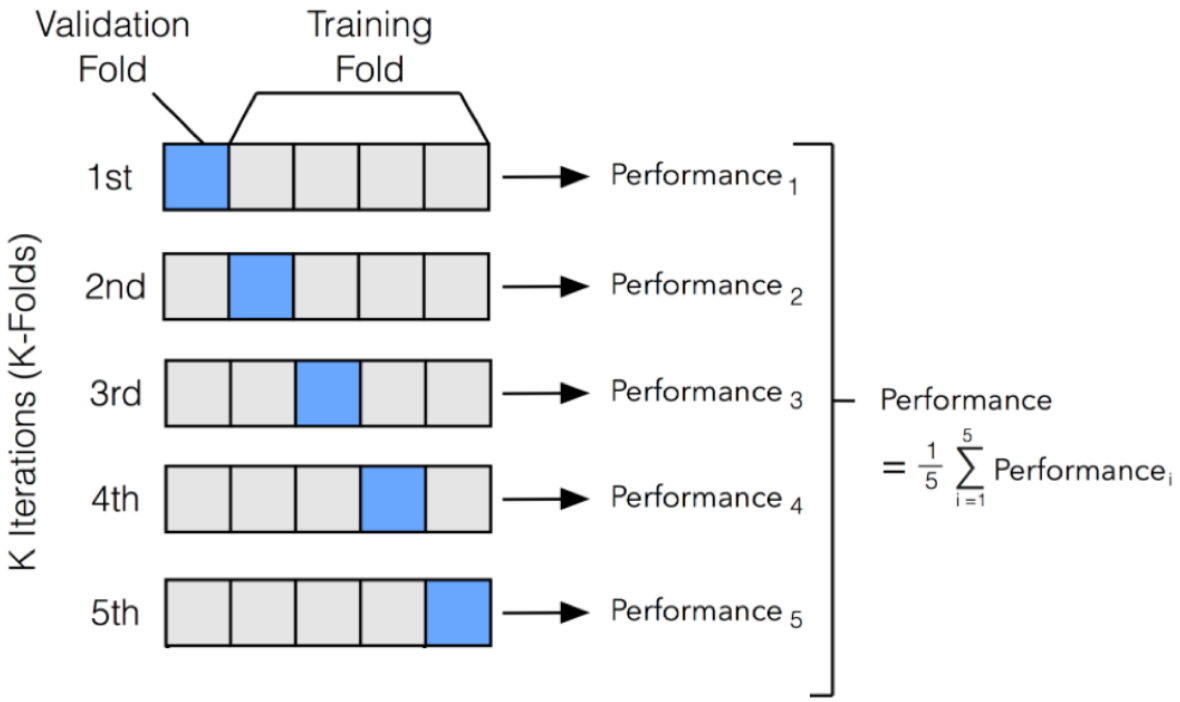
Exemplo: Overffiting.ipynb

Como detectar o sobreajuste



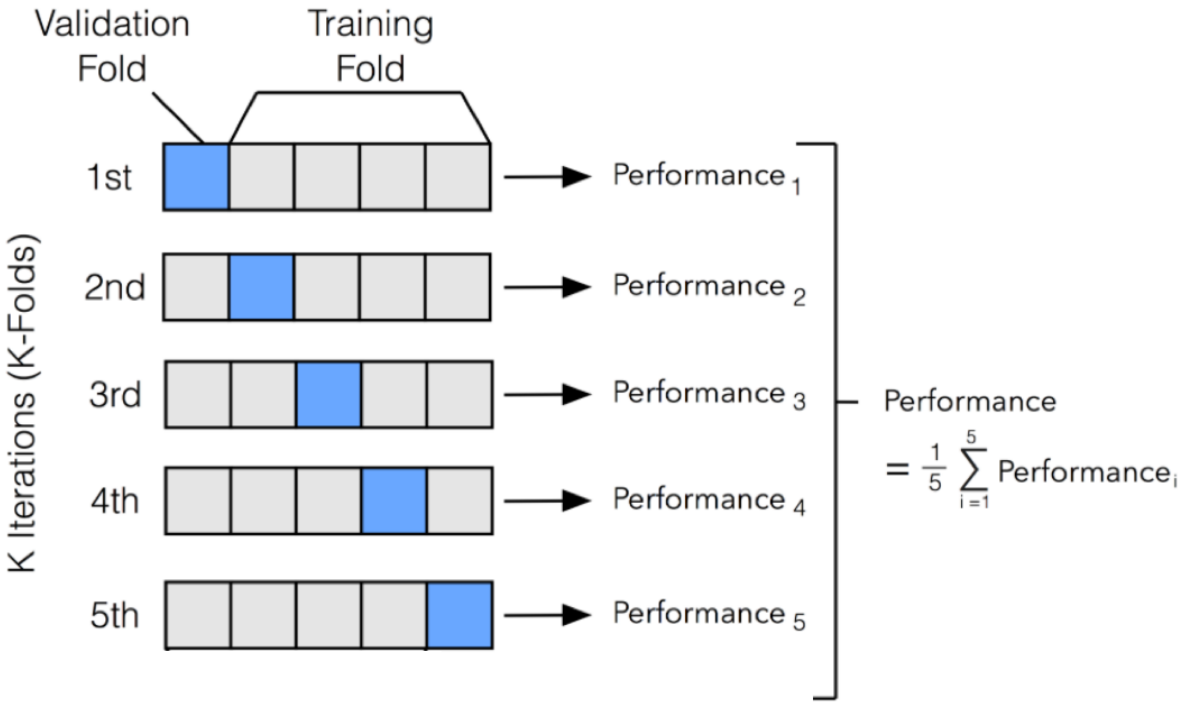
- **Conjunto de Validação:** No caso de ***modelos de aprendizado iterativo***, como redes neurais, podemos ***monitorar o desempenho do modelo em ambos os conjuntos*** durante o treinamento.
- Se o ***erro no conjunto de validação aumentar e no conjunto de treinamento diminuir*** ao longo das épocas de treinamento, isso é ***um claro sinal de sobreajuste***.
  - Como vimos antes, nesse caso, podemos usar a ***parada antecipada*** (*early stopping*).

Como detectar o sobreajuste



- Técnicas de ***validação cruzada avaliam o desempenho*** do modelo em ***várias divisões de dados de treinamento e validação***.
- Se o ***modelo mostrar variações significativas no desempenho*** (i.e., erro e variância) ***entre as divisões*** dos dados, isso pode ser um ***sinal de sobreajuste ou subajuste***.
- Técnicas de validação cruzada que podem ser usadas: *k-fold*, *leave-p-out*, *holdout*.
  - Na prática, o *k*-fold é a mais usada.

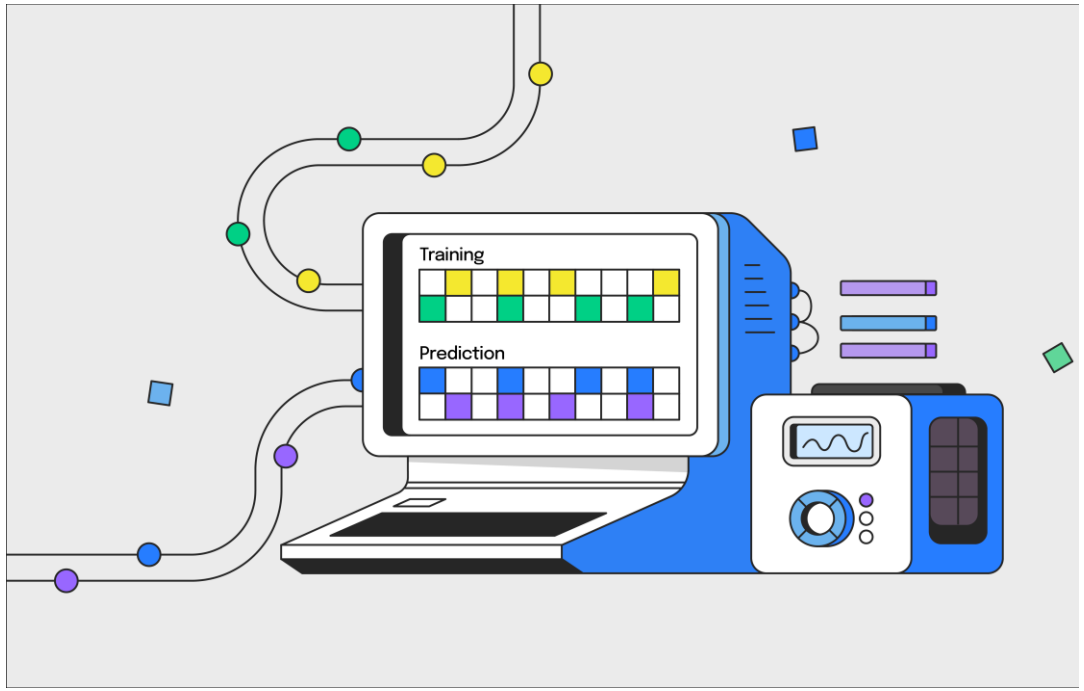
Como detectar o sobreajuste: *k-fold*



- A técnica ***divide o conjunto de dados em k subconjuntos***, chamados dobras (*folds*), ***treinando*** e ***avaliando o modelo k vezes*** (e.g., MSE ou acurácia), cada vez com um subconjunto diferente como conjunto de teste e os k-1 subconjuntos restantes como conjunto de treinamento.
- Ao final, ***calcula-se a média e a variância das k métricas*** individuais de desempenho para fornecer uma única ***métrica geral do modelo***.

Exemplo: validacao_cruzada_k_fold.ipynb

Como evitar o sobreajuste?



- **Coletar mais dados** de treinamento é uma das ***estratégias mais eficazes*** para ***melhorar a capacidade de generalização*** do modelo, consequentemente, reduzindo o sobreajuste.
- Porém, em algumas situações, a ***coleta de dados*** adicionais pode ser muito ***cara***, ***demorada*** ou ***simplesmente impossível***.
  - Por coleta, entende-se que a ***rotulagem também está inclusa***.

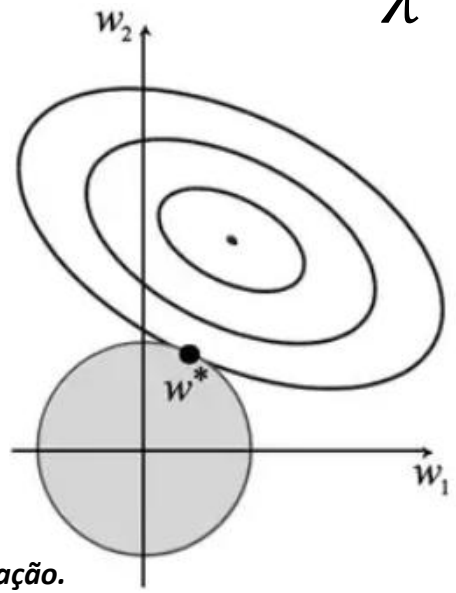
Como evitar o sobreajuste?

MSE Penalização L2

$$\min_{\boldsymbol{w}}(\|\boldsymbol{y} - \boldsymbol{h}(\boldsymbol{X};\boldsymbol{w})\|^2 + \lambda\|\boldsymbol{w}\|_2^2),$$

$$\min_{\boldsymbol{w}}\|\boldsymbol{y} - \boldsymbol{h}(\boldsymbol{X};\boldsymbol{w})\|^2$$

$$\text{s.}\,a.\,\|\boldsymbol{w}\|_2^2 \leq \frac{1}{\lambda}.$$



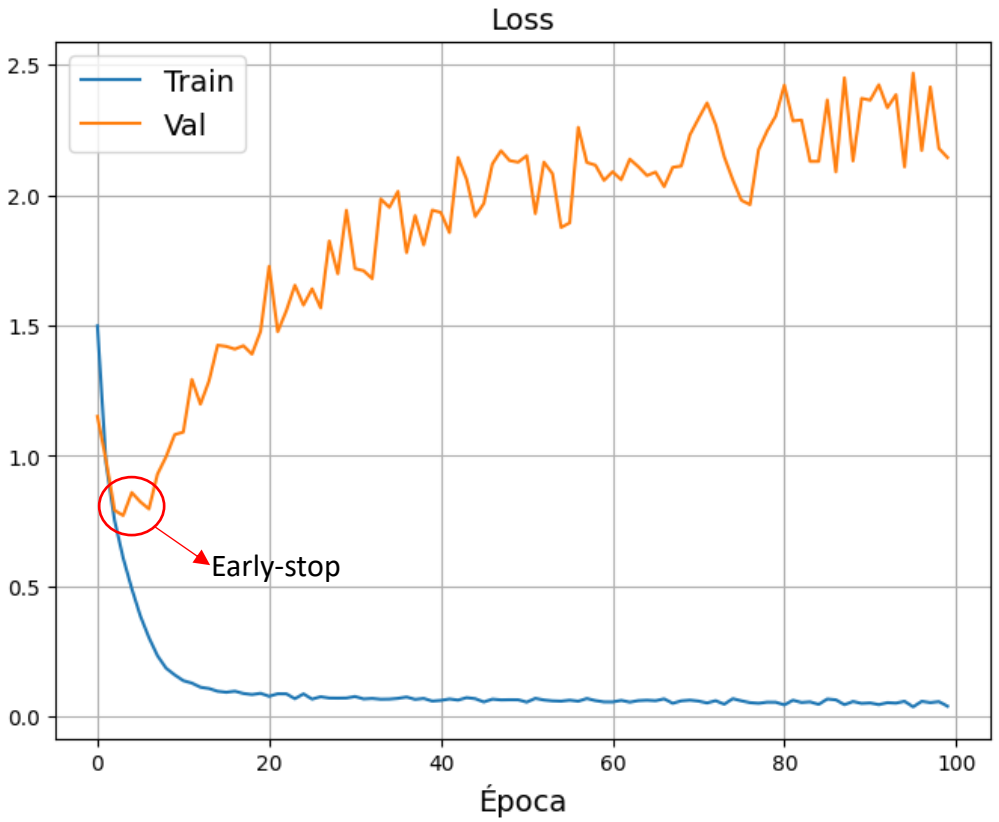
OBS.: $\lambda \geq 0$ é o ***fator de regularização.***

- **Regularização**: envolve a ***adição de um termo de penalização à função de perda durante o treinamento*** do modelo, com o objetivo de impor ***restrições sobre os pesos do modelo***.
- A regularização ***reduz a flexibilidade*** do modelo ***penalizando seus pesos***.
  - Modelos que sobreajustam têm ***pesos com valores absolutos muito altos***.
  - A regularização ***restringe*** o aumento dos pesos a uma ***região de possíveis valores***.
- As técnicas de regularização mais utilizadas são L1, L2 e *elastic-net*.

Como evitar o sobreajuste?

- Existem outras técnicas que ***não envolvem a adição de termos de penalização***, mas que evitam que o modelo se torne muito flexível e se sobreajuste.
- São técnicas de regularização, mas não no sentido tradicional.
- Duas técnicas não tradicionais muito utilizadas com redes neurais são o *early-stopping* e o *Dropout*.

Como evitar o sobreajuste?

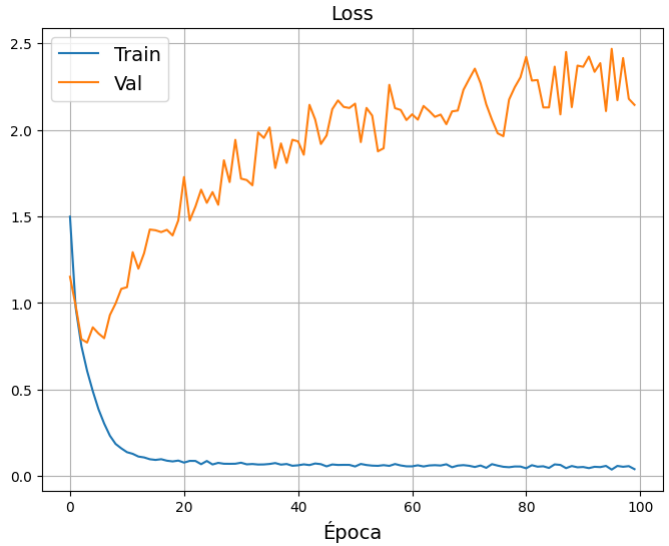


- ***Early-stopping*** (ou parada antecipada) é uma técnica de ***regularização temporal***.
- A ideia é ***interromper o treinamento do modelo assim que o desempenho no conjunto de validação começa a piorar***, em vez de continuar até que o modelo se ajuste demais aos dados de treinamento.
  - ***Vantagens***: ***economiza*** tempo e recursos computacionais.
  - ***Desvantagem***: se o treinamento for interrompido muito cedo, o modelo pode não ter tempo suficiente para aprender o padrão geral nos dados de treinamento, levando ao ***subajuste***.

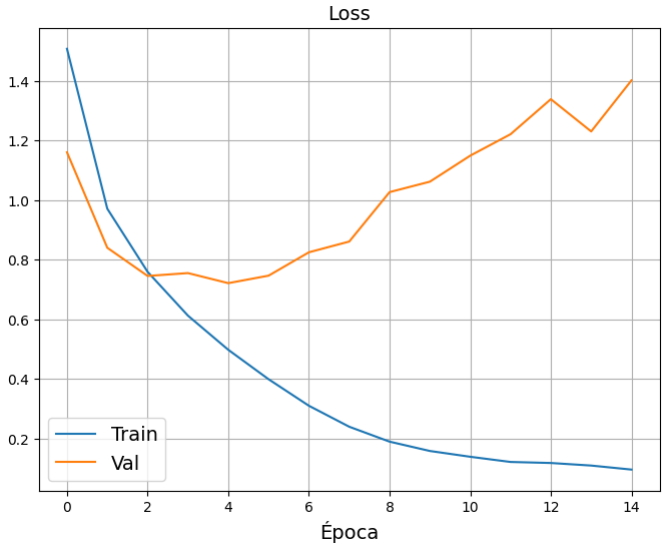
Como evitar o sobreajuste?

```
early_stop = EarlyStopping(monitor='val_loss',
                           mode='min',
                           patience=10,
                           restore_best_weights=True
)
```

```
history = model.fit(train_images,
                    train_labels,
                    batch_size=64,
                    validation_data=(test_images, test_labels),
                    epochs=100
)
```

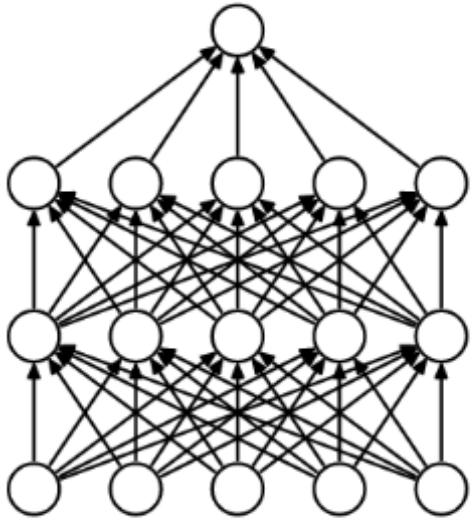



```
history = model.fit(train_images,
                    train_labels,
                    batch_size=64,
                    validation_data=(test_images, test_labels),
                    epochs=100,
                    callbacks=[early_stop]
)
```

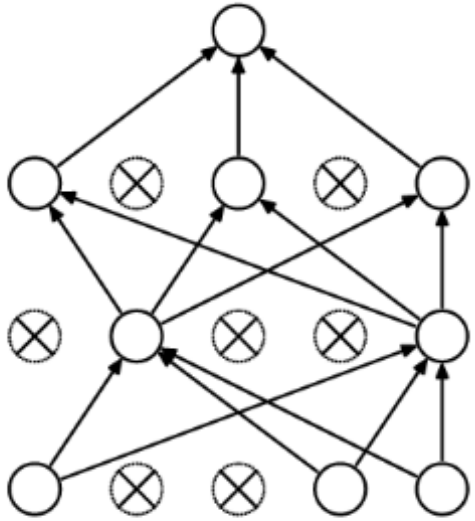


Exemplo: Sobreajuste_e_early_stop.ipynb

Como evitar o sobreajuste?



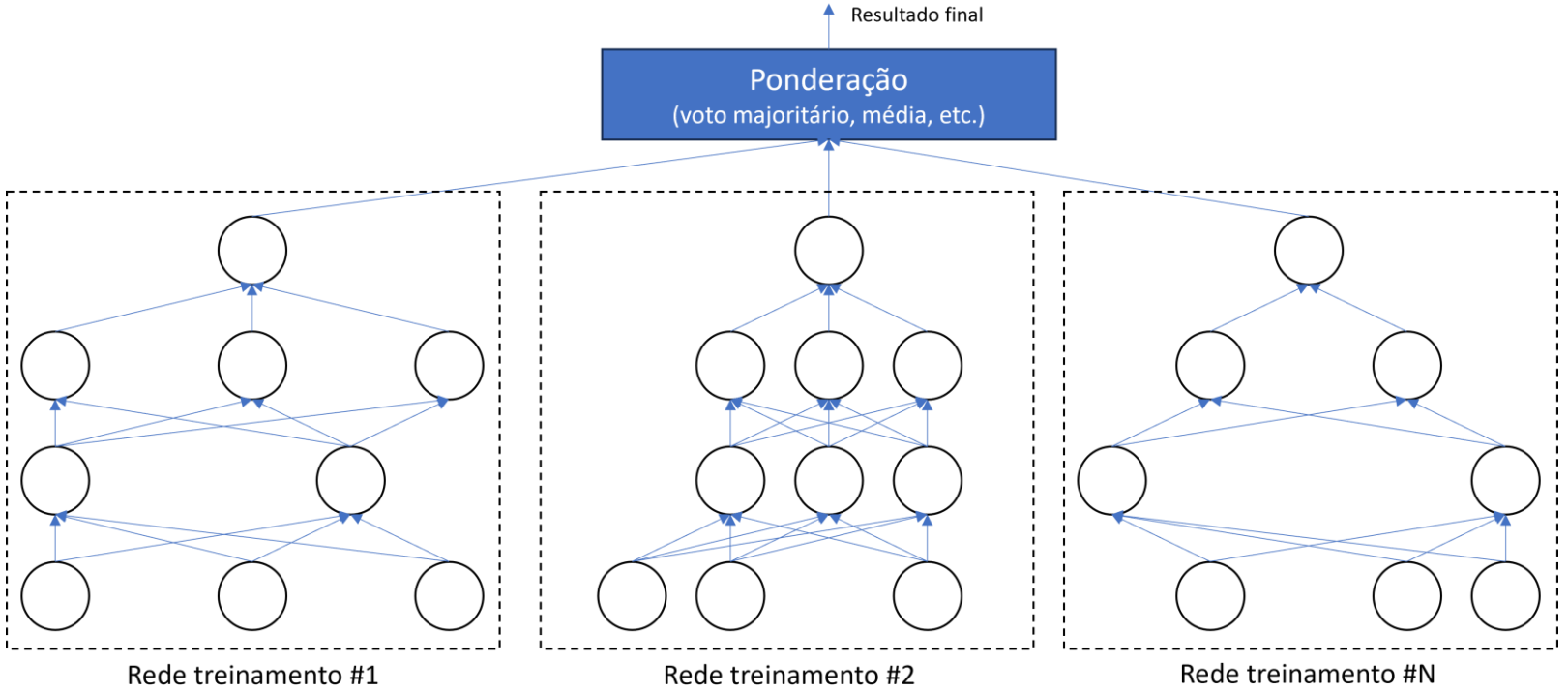
(a) Standard Neural Net



(b) After applying dropout.

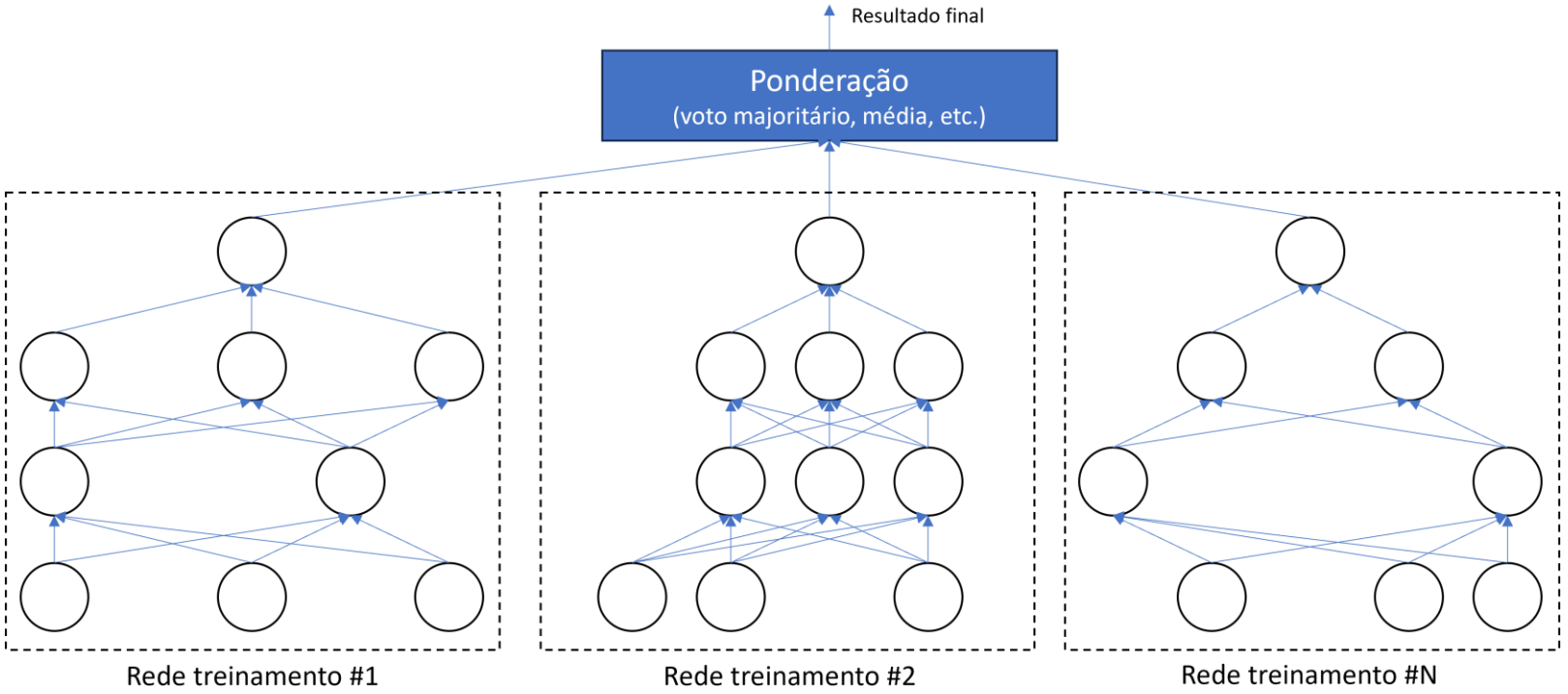
- O ***dropout*** envolve ***aleatoriamente "desligar" ou "descartar" neurônios de uma camada durante o treinamento*** da rede neural.
- Isso significa que esses neurônios não contribuirão para o cálculo das saídas da rede durante essa iteração de treinamento.
- Desta forma, o ***modelo*** fica ***menos complexo***, ***diminuindo*** as chances de ***sobreajuste***.

Como evitar o sobreajuste?



- ***Heuristicamente***, quando ***desligamos*** diferentes conjuntos de ***neurônios***, é como se ***estivéssemos treinando várias redes*** neurais diferentes.

Como evitar o sobreajuste?



- Durante a ***inferência***, os ***resultados dessas redes são ponderados*** (e.g., média ou voto majoritário) para produzir a saída final da rede.
- As ***diferentes redes se adaptarão de diferentes maneiras***, e assim, esperançosamente, o efeito do *dropout* será ***reduzir o sobreajuste***.

Como evitar o sobreajuste?

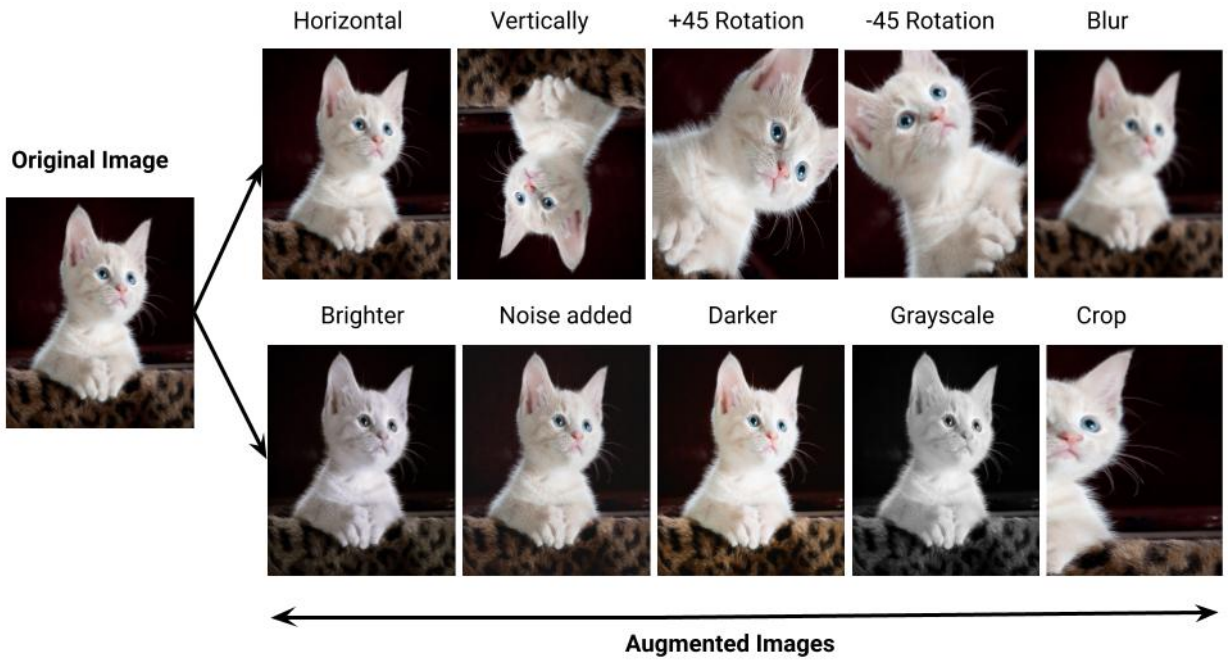
```
model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(filters=32, kernel_size=(3, 3), activation='relu', padding='same', input_shape=(32, 32, 3)),
    tf.keras.layers.Conv2D(filters=32, kernel_size=(3, 3), activation='relu', padding='same'),
    tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Conv2D(filters=64, kernel_size=(3, 3), activation='relu', padding='same'),
    tf.keras.layers.Conv2D(filters=64, kernel_size=(3, 3), activation='relu', padding='same'),
    tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Conv2D(filters=128, kernel_size=(3, 3), activation='relu', padding='same'),
    tf.keras.layers.Conv2D(filters=128,kernel_size=(3, 3), activation='relu', padding='same'),
    tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(units=1024, activation='relu', kernel_initializer='he_uniform'),
    tf.keras.layers.Dense(units=512, activation='relu', kernel_initializer='he_uniform'),
    tf.keras.layers.Dense(units=10, activation='softmax')
])
```

Probabilidade de descarte dos nós ou conexões da camada.

Podem ser usadas com qualquer tipo de camada: densa, convolucional, etc.

Exemplo: Sobreajuste_e_dropout.ipynb

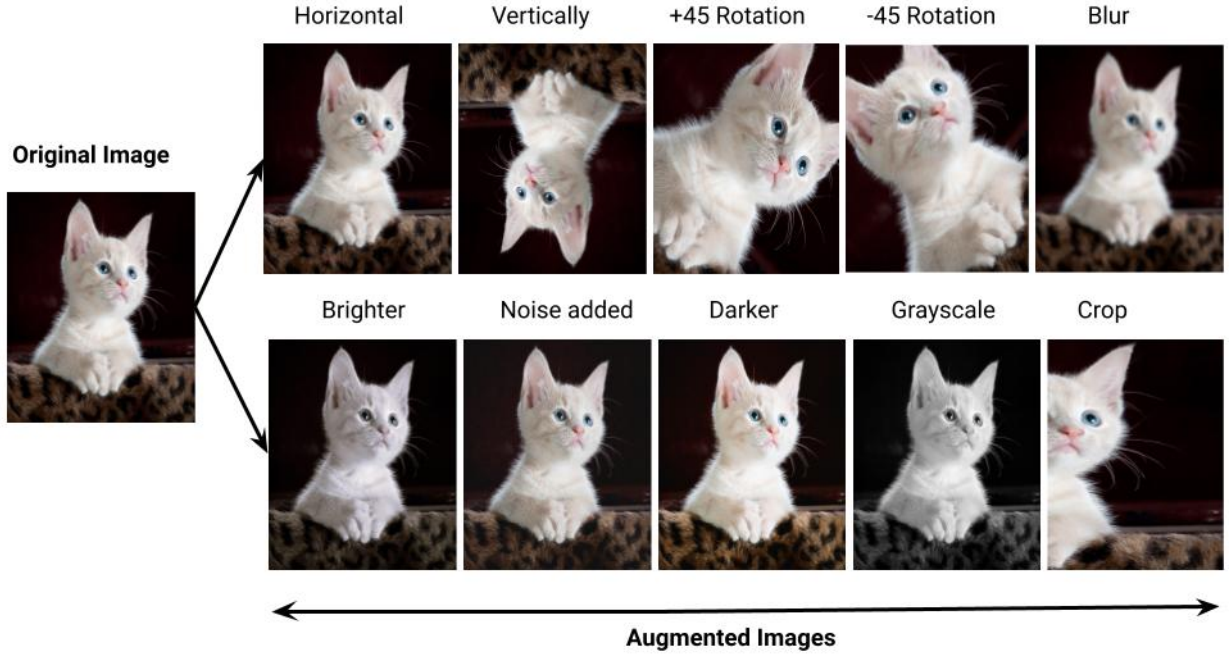
Como evitar o sobreajuste?



- **Aumentar artificialmente os dados (*Data Augmentation*):** O objetivo é ***aumentar*** a ***variedade*** e a ***diversidade*** dos dados de treinamento ***criando novos exemplos a partir dos dados existentes***.
- Essa técnica ajuda a melhorar o desempenho do modelo, tornando-o mais ***robusto*** e capaz de ***generalizar melhor***.

Exemplo: Treinamento_com_e_sem_data_augmentation.ipynb

Como evitar o sobreajuste?



- Quando o problema envolve ***imagens***, podemos aplicar ***transformações***, como ***rotação***, ***espelhamento*** e ***corte***, modificação do ***brilho***, adição de ***ruído***, ***desfoque***, etc.
- O ***data augmentation torna CNNs robustas à rotação das imagens***, pois a rede é treinada com a ***mesma imagem com diferentes rotações***.
- Assim a rede aprende a ***reconhecer padrões independentemente da orientação***.

Data augmentation com Tensorflow

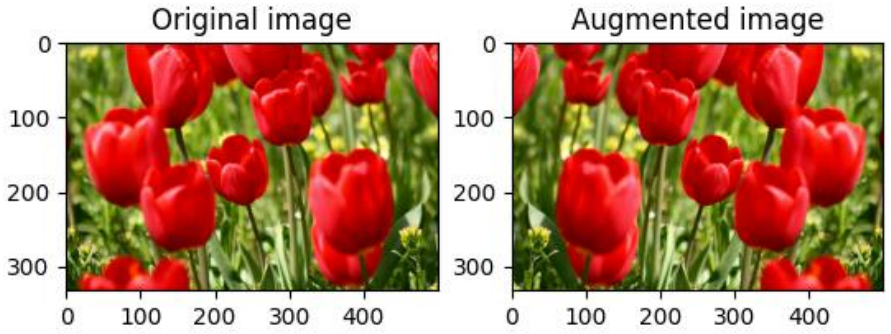
```
data_augmentation = keras.Sequential(
  [
    layers.RandomFlip("horizontal", input_shape=(img_height, img_width, 3)),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),
  ]
)
```



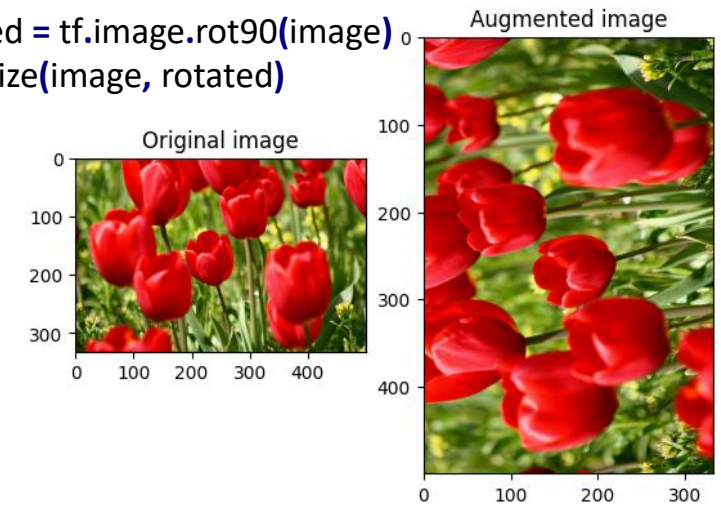
- O Tensorflow possui várias ***camadas*** que podem ser usadas para criar novas imagens a partir das existentes:
  - tf.keras.layers.RandomCrop(),
  - tf.keras.layers.RandomZoom(),
  - tf.keras.layers.RandomFlip(),
  - tf.keras.layers.RandomRotation(), etc.
- Essas camadas podem ser ***adicionadas como camadas iniciais em um modelo*** e o TensorFlow cuidará automaticamente do pré-processamento durante o treinamento e as ***desligará*** durante a inferência.

Data augmentation com Tensorflow

```
flipped = tf.image.flip_left_right(image)
visualize(image, flipped)
```



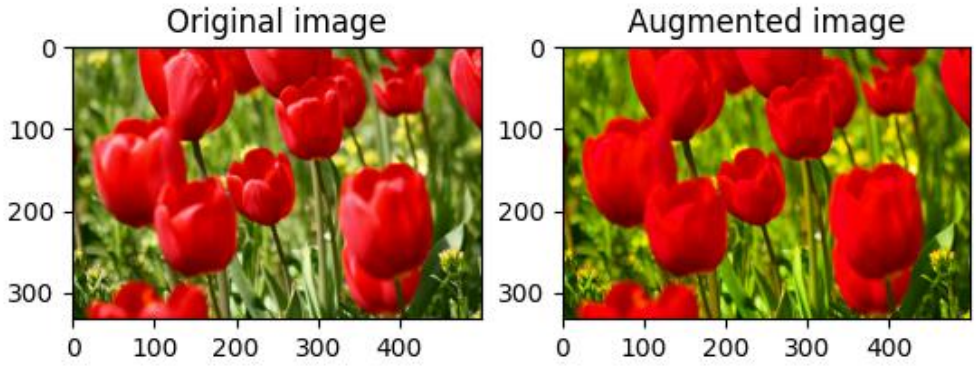

```
rotated = tf.image.rot90(image)
visualize(image, rotated)
```



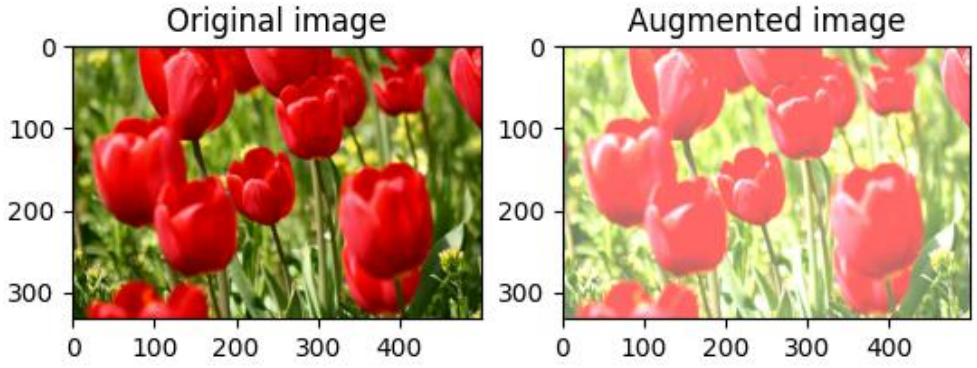
- Além das camadas, o Tensorflow possui um módulo de pré-processamento de imagens chamado ***tf.image*** que contém ***funções*** que também podem ser usadas para criar novas imagens.
- Elas são usadas para um ***controle mais preciso*** do pré-processamento.

Data augmentation com Tensorflow

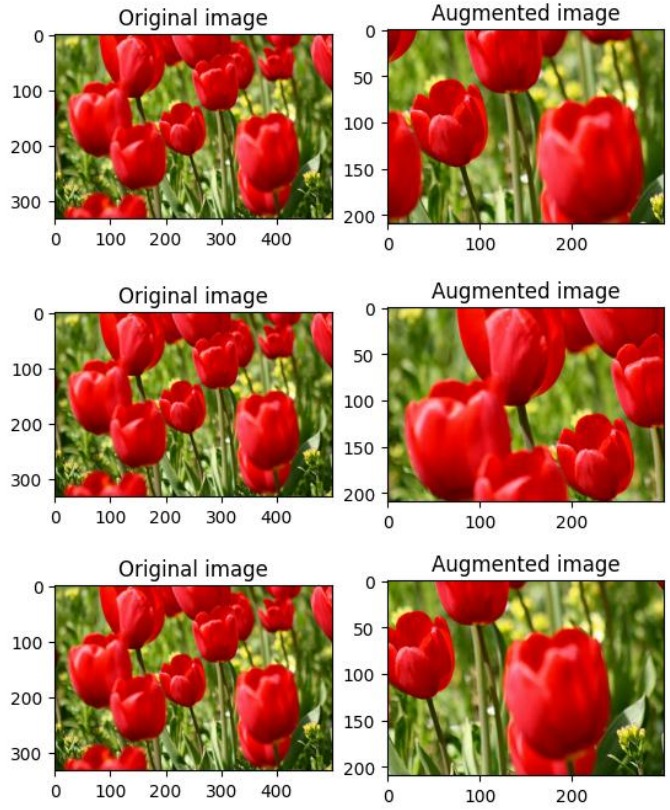
```
saturated = tf.image.adjust_saturation(image, 3)
visualize(image, saturated)
```




```
bright = tf.image.adjust_brightness(image, 0.4)
visualize(image, bright)
```

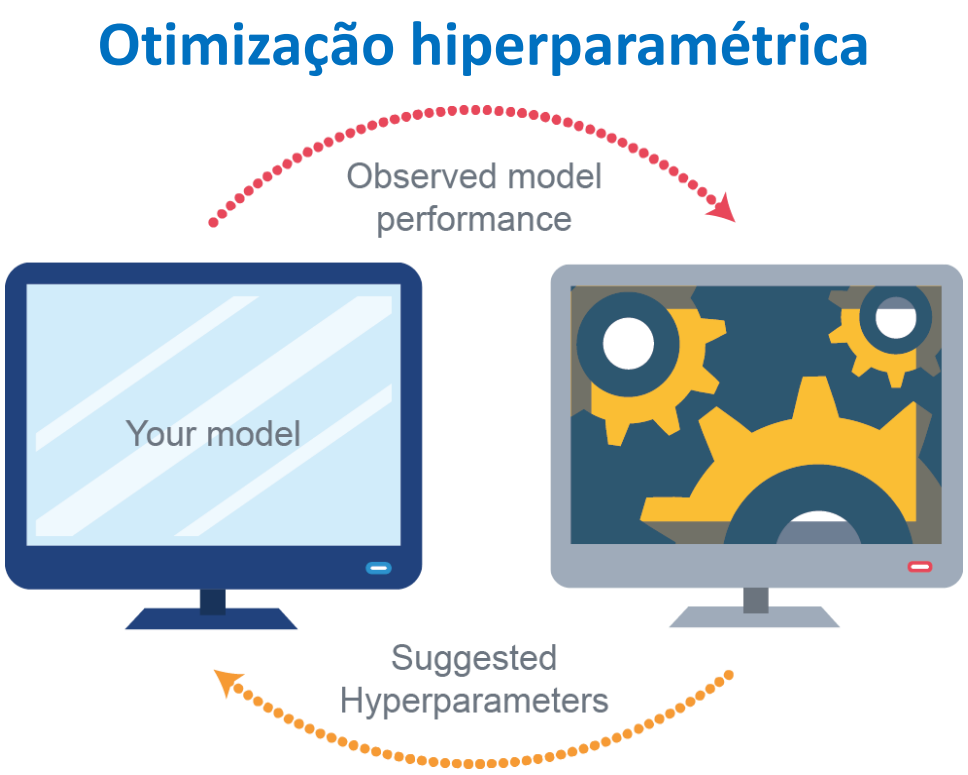



```
for i in range(3):
  seed = (i, 0)  # tuple of size (2,)
  stateless_random_crop = tf.image.stateless_random_crop(
     image, size=[210, 300, 3], seed=seed)
  visualize(image, stateless_random_crop)
```



Exemplo: data_augmentation.ipynb

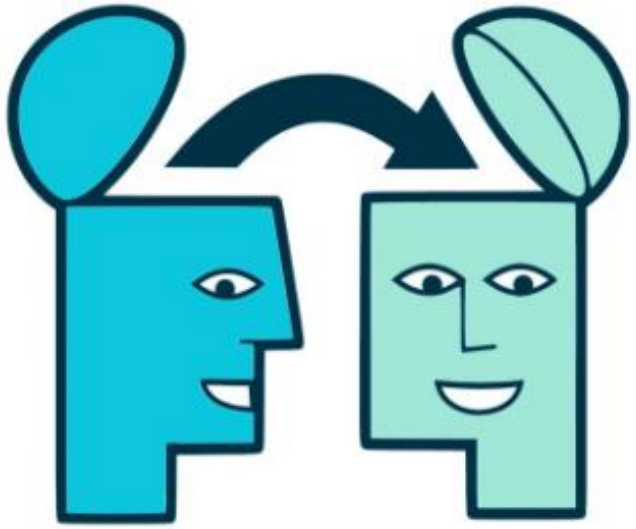
Como evitar o sobreajuste?



- Um ***modelo excessivamente complexo*** tem grandes chances de se ***sobreajustar***.
- Portanto, ***modelos menos complexos*** (i.e., mais simples), com menos parâmetros, por exemplo, ***tendem a não se sobreajustar e a generalizar melhor***.
- O objetivo é encontrar um modelo com ***complexidade suficiente para capturar o padrão geral por trás dos dados***.
- ***Otimização hiperparamétrica*** pode ajustar a complexidade do modelo e ***encontrar a configuração que oferece um bom equilíbrio entre viés e variância***.

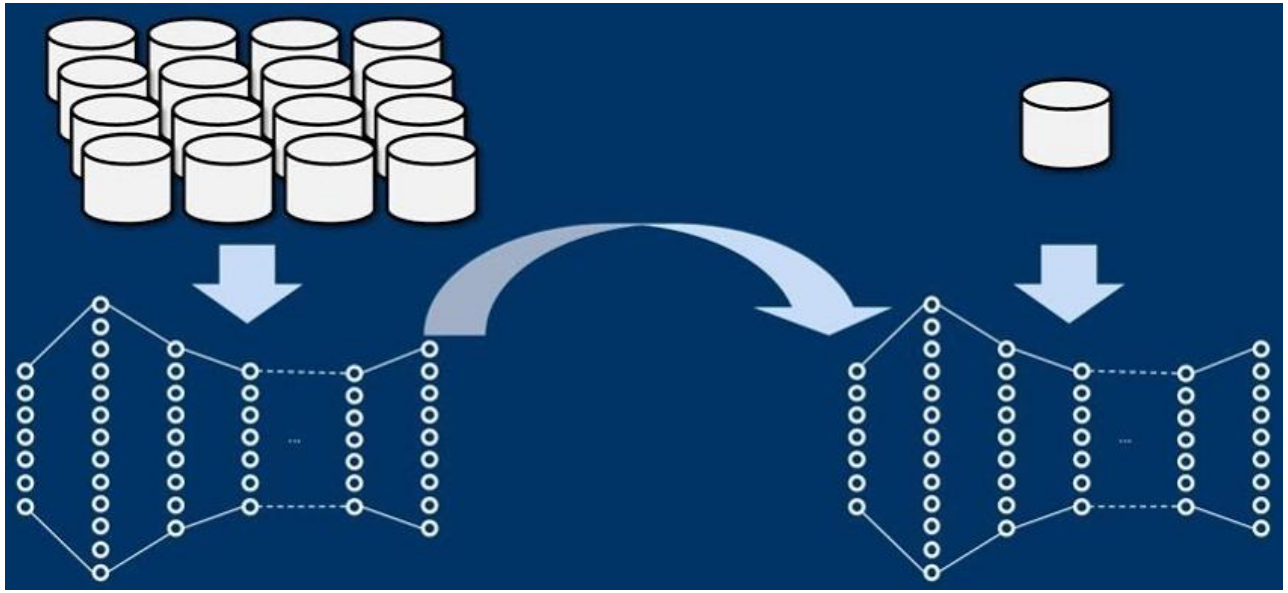
Exemplo: Sobreajuste_e_otimização_hiperparamétrica.ipynb

Como evitar o sobreajuste?



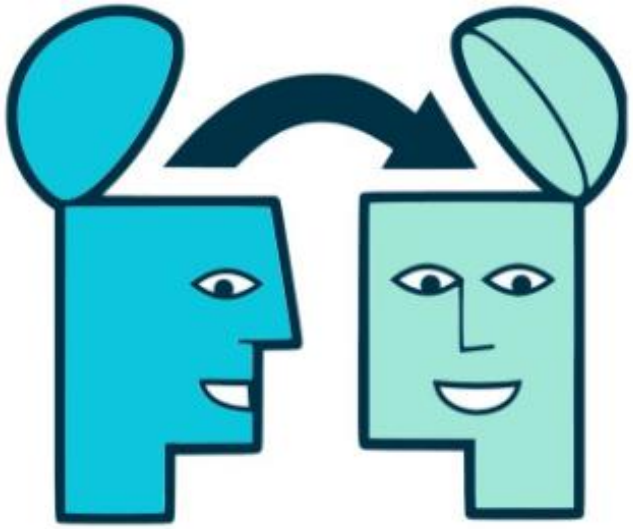
- ***Transfer learning*** ou ***transferência de aprendizado*** é uma abordagem que envolve o uso de um ***modelo pré-treinado em uma tarefa*** (ou domínio) relacionada ***como ponto de partida para uma nova tarefa***.

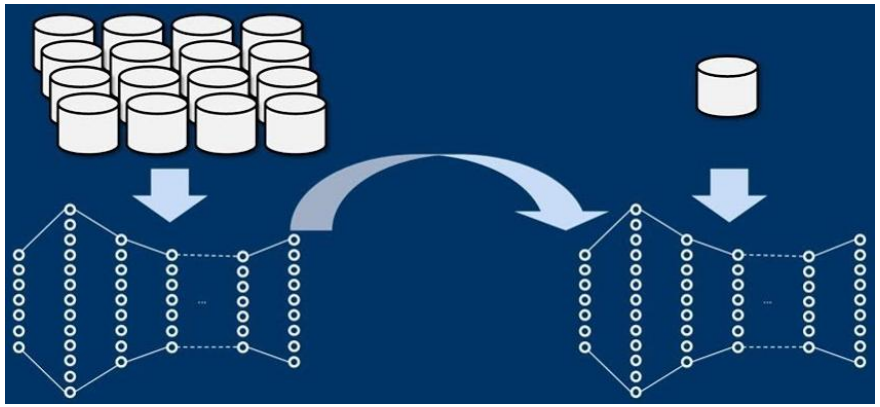
Como evitar o sobreajuste?



- Usamos o ***conhecimento adquirido*** por um ***modelo treinando em uma grande base de dados*** como ***ponto de partida*** para treinar um modelo com uma ***base de treinamento menor***, ***mas que seja relacionada à primeira***.

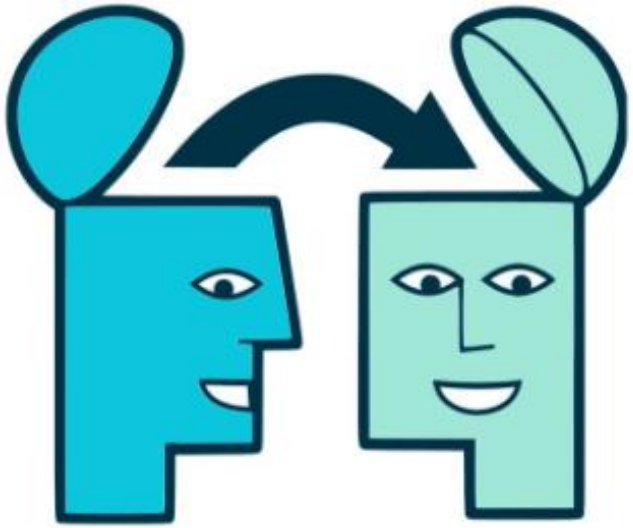
Transfer learning

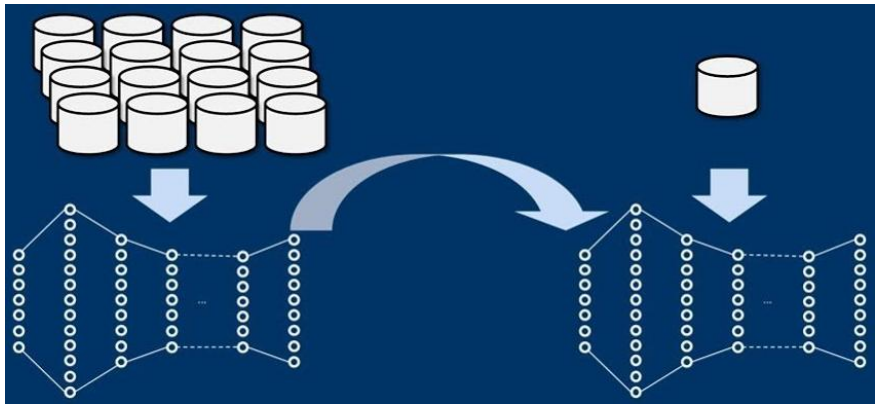




- Suponham que temos uma ***rede pré-treinada*** para classificar imagens em 100 categorias diferentes, incluindo ***animais***, ***plantas***, ***veículos***, etc. e queremos treinar uma ***nova rede para classificar tipos específicos de veículos***.
- Por essas tarefas serem semelhantes, ***podemos reutilizar partes da primeira rede***.

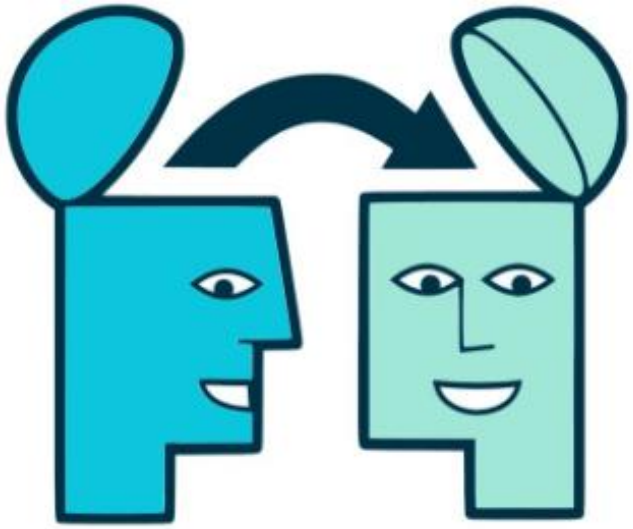
Como evitar o sobreajuste?

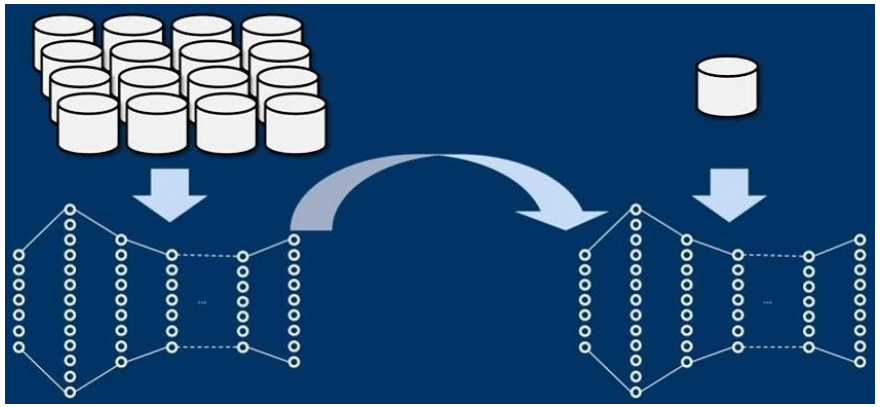




- Modelos pré-treinados em grandes conjuntos de dados ***extraem características relevantes dos dados***.
- Essas ***características são transferidas para o novo modelo***, que pode se beneficiar de uma ***inicialização mais informada***.

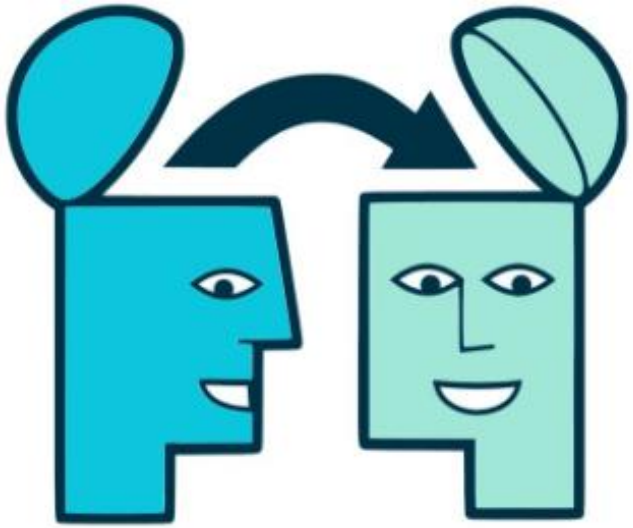
Transfer learning

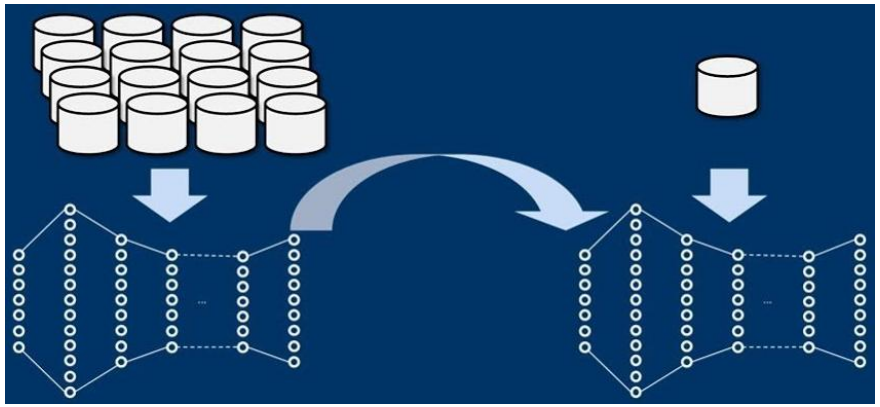




- O *transfer learning* ***pode evitar*** que o ***novo modelo sobreajuste***, pois ele ***já começa com características relevantes***.
- Como o ***modelo pré-treinado já adquiriu um bom entendimento de características genéricas*** em uma tarefa relacionada, o ***treinamento do novo modelo pode exigir menos dados*** para alcançar um bom desempenho.

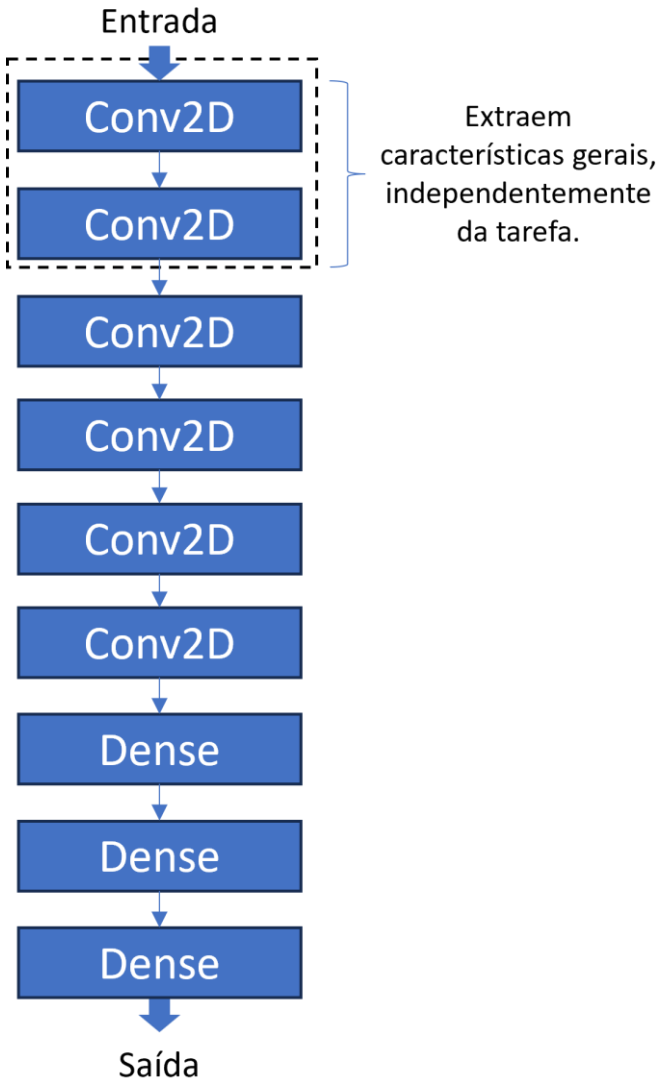
Transfer learning





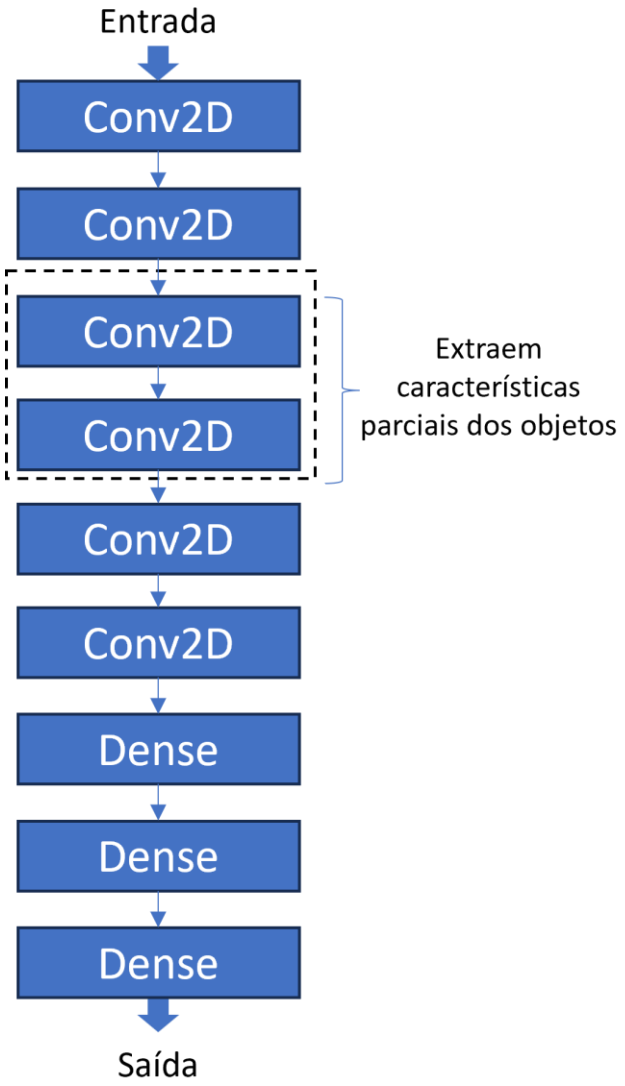
- O *transfer learning* é particularmente ***útil quando os dados de treinamento são escassos***, o que é uma situação ***propensa ao sobreajuste***.
- Além de requerer menos dados, o *transfer learning* ***acelera consideravelmente o treinamento***.

Transfer learning em uma rede convolucional



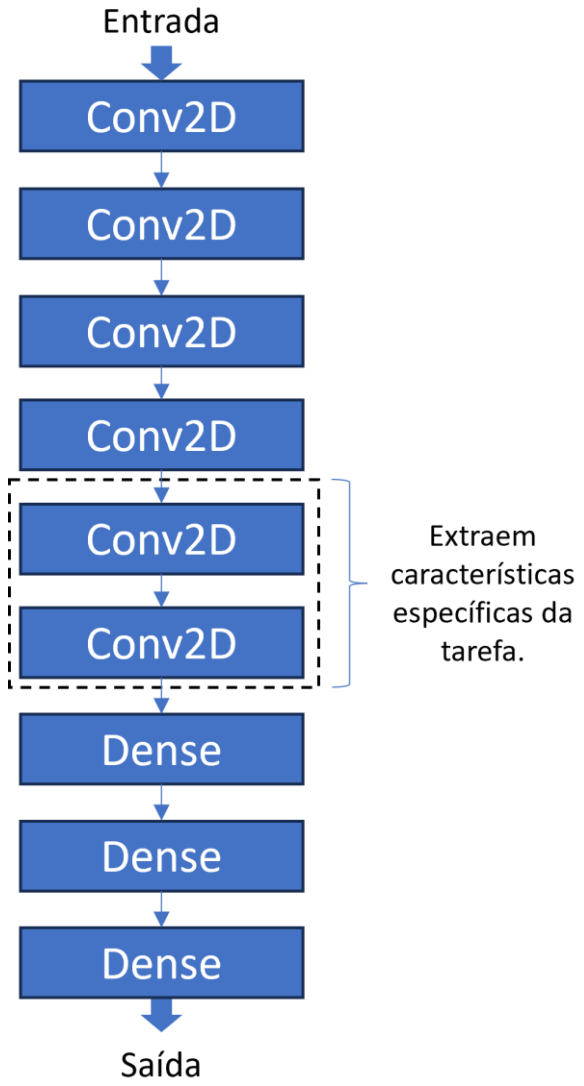
- Para entender como o *transfer learning*, funciona, vamos usar uma ***rede convolucional*** como exemplo.
- Nestas redes, as ***camadas convolucionais iniciais***, ***aprendem*** a ***detectar características simples*** (i.e., de baixo nível), como bordas (verticais e horizontais), linhas, cantos e texturas.
  - Características gerais, ***independentes da tarefa***.

Transfer learning em uma rede convolucional



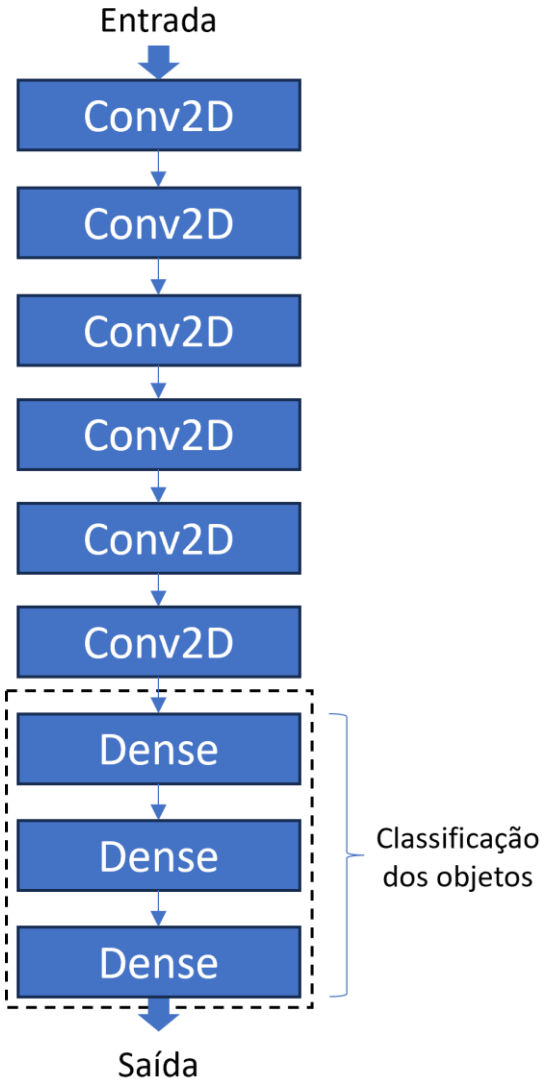
- As ***camadas intermediárias***, ***baseando-se nas características aprendidas pelas camadas anteriores***, ***aprendem a detectar características mais complexas***, como formas geométricas mais elaboradas, partes de objetos e texturas mais abstratas.
- Estas camadas ***capturaram características parciais de objetos***, como, por exemplo, olhos, narizes e partes do corpo.
- No caso de carros, lanternas, retrovisores, para-choques, vidros, pneus, etc.

Transfer learning em uma rede convolucional



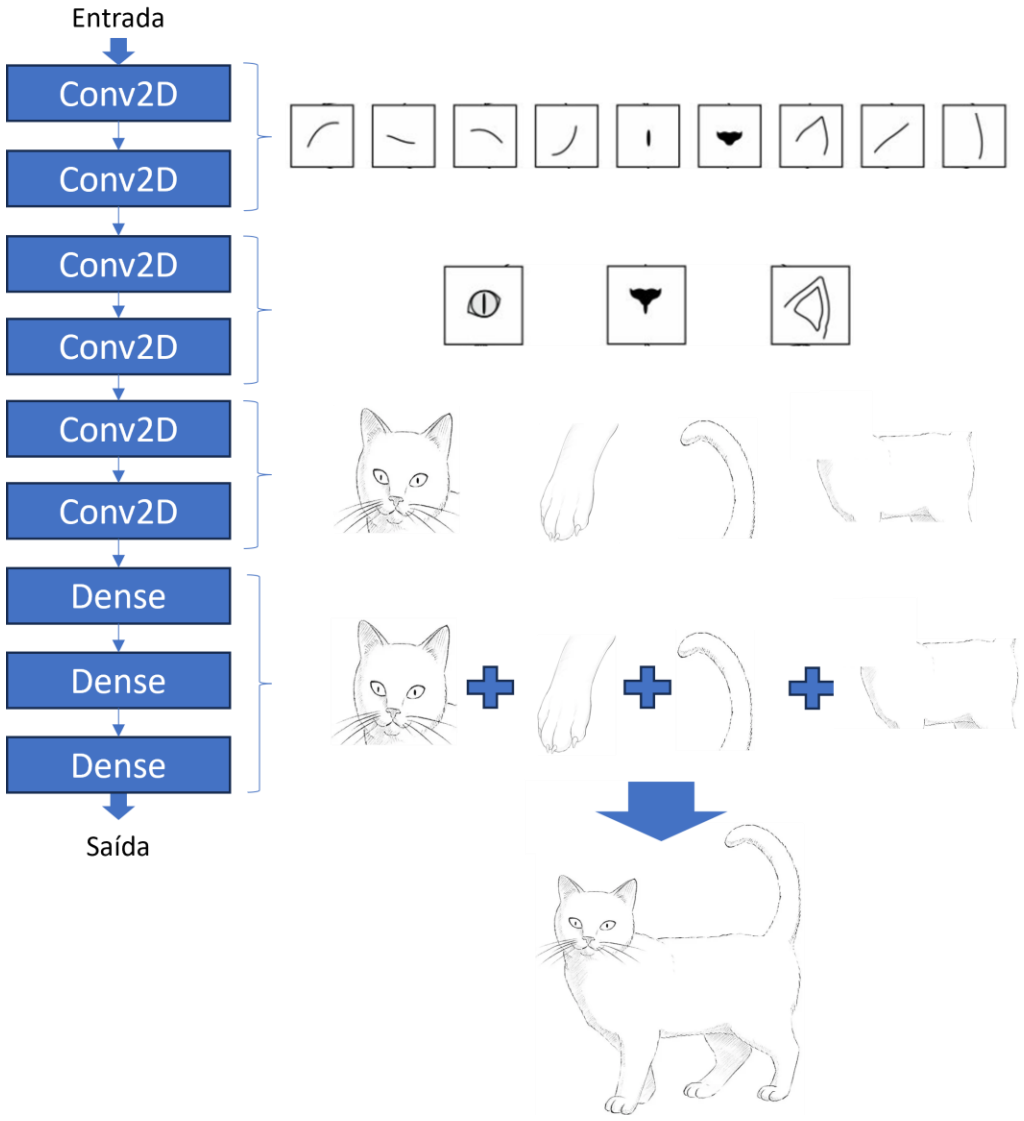
- Nas ***camadas mais profundas, combinando as características detectadas pelas camadas anteriores***, a ***rede convolucional tende a aprender características de alto nível***, como partes completas de objetos, objetos inteiros e até mesmo conceitos semânticos mais abstratos, como "rosto humano" ou "carro".

Transfer learning em uma rede convolucional



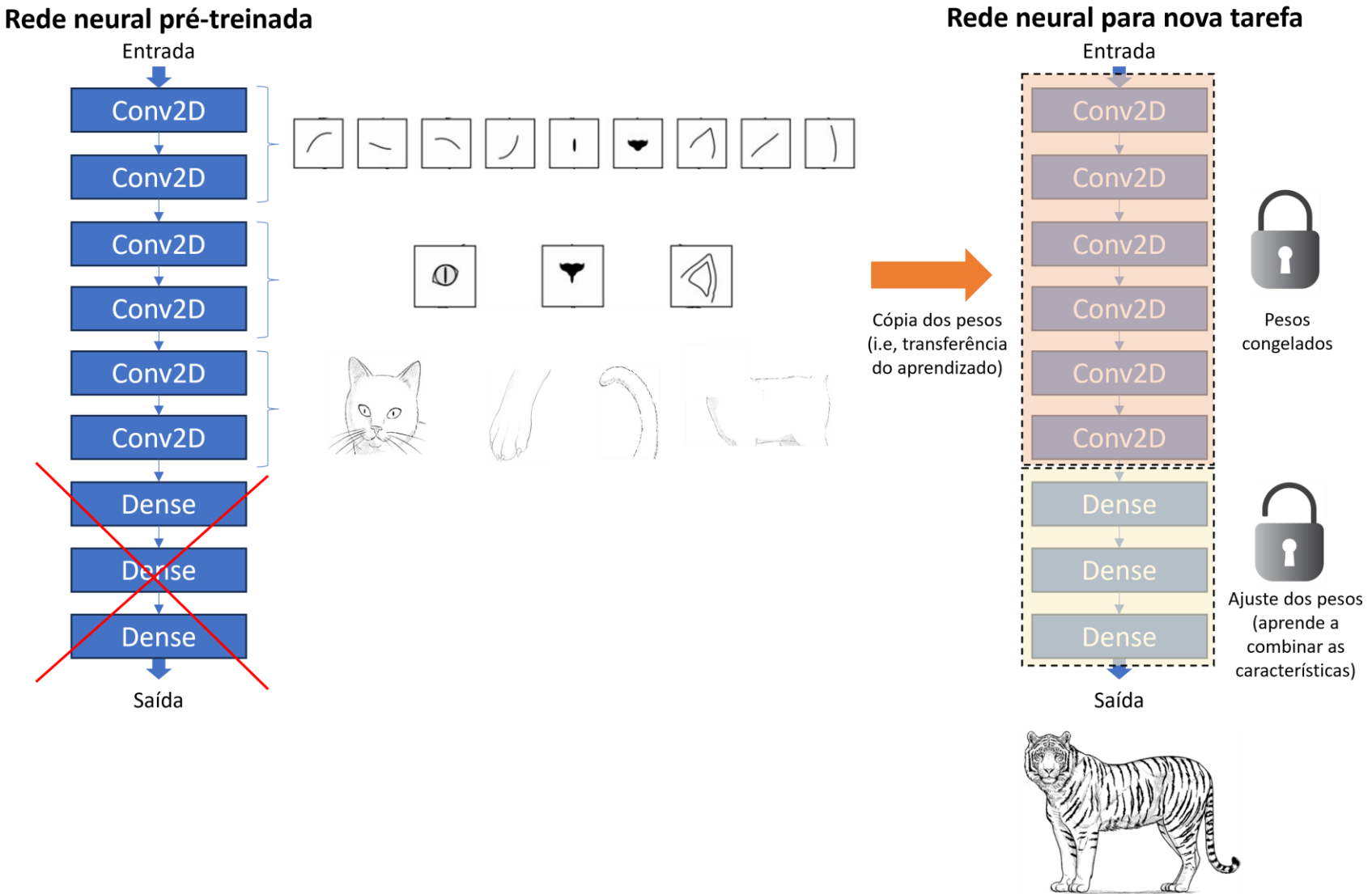
- As ***camadas densas recebem as características extraídas*** pelas camadas convolucionais e ***combinam essas informações*** para ***aprender padrões complexos*** e realizar a tarefa de ***classificação*** ou ***regressão***.
- Em tarefas de classificação, elas mapeiam as características para as classes alvo, enquanto em tarefas de regressão, produzem uma saída contínua.

Transfer learning em uma rede convolucional



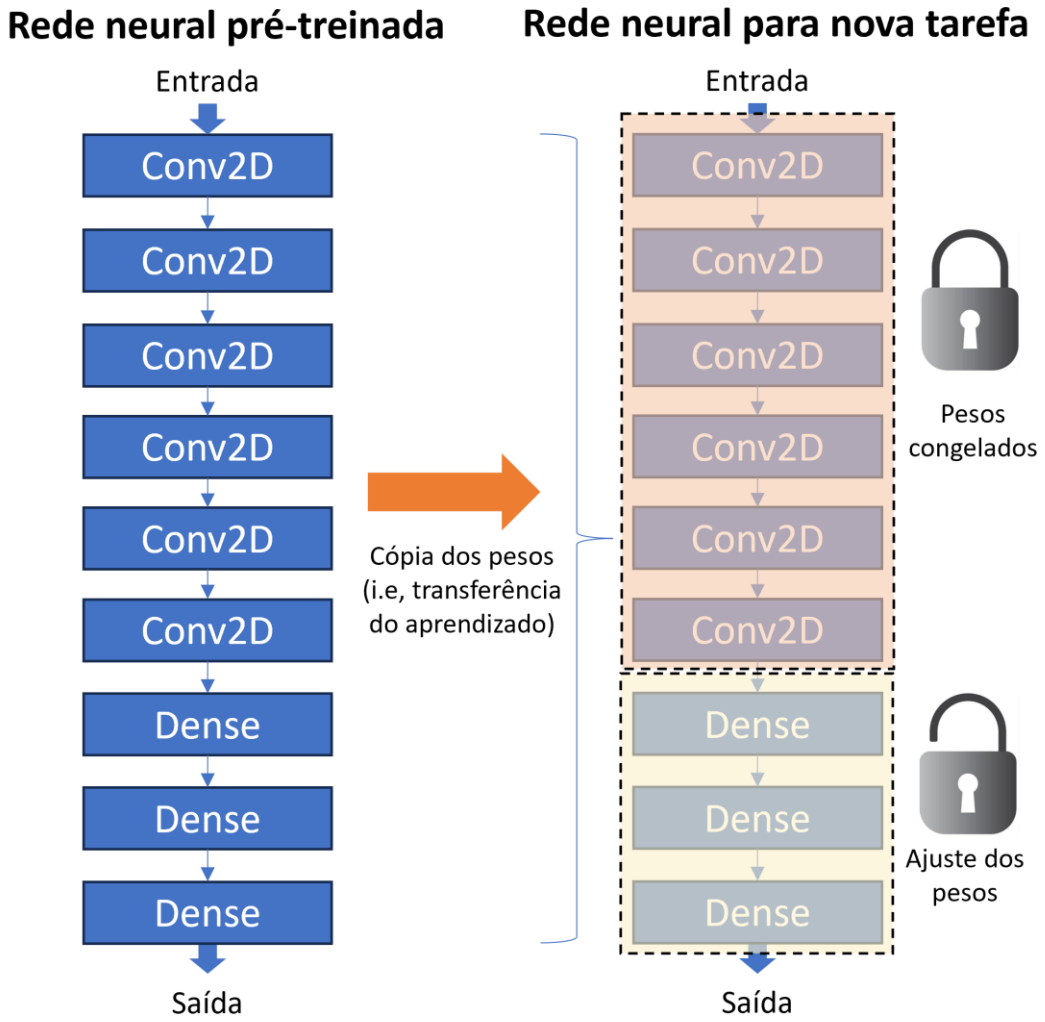
- ***Combinando conceitos simples***, como cantos e contornos, ***com conceitos mais complexos***, como partes de um objeto, ***uma rede convolucional aprende a detectar gatos***, por exemplo.

Transfer learning em uma rede convolucional



- Usando *transfer learning,* podemos usar as características aprendidas sobre gatos e usá-las para identificar tigres, por exemplo.

Transfer learning em uma rede convolucional



- Portanto, o que nós fazemos no *transfer learning* é ***congelar*** os pesos das camadas iniciais (i.e., ***características gerais, independentes da tarefa***) e apenas aplicar ***atualizações*** (i.e., gradiente descendente) ***aos pesos das camadas finais***.
- Se a tarefa da rede neural pré-treinada for ***muito similar à nova tarefa***, pode-se ***treinar apenas as camadas densas***.

Procedimento para aplicar o *Transfer learning*

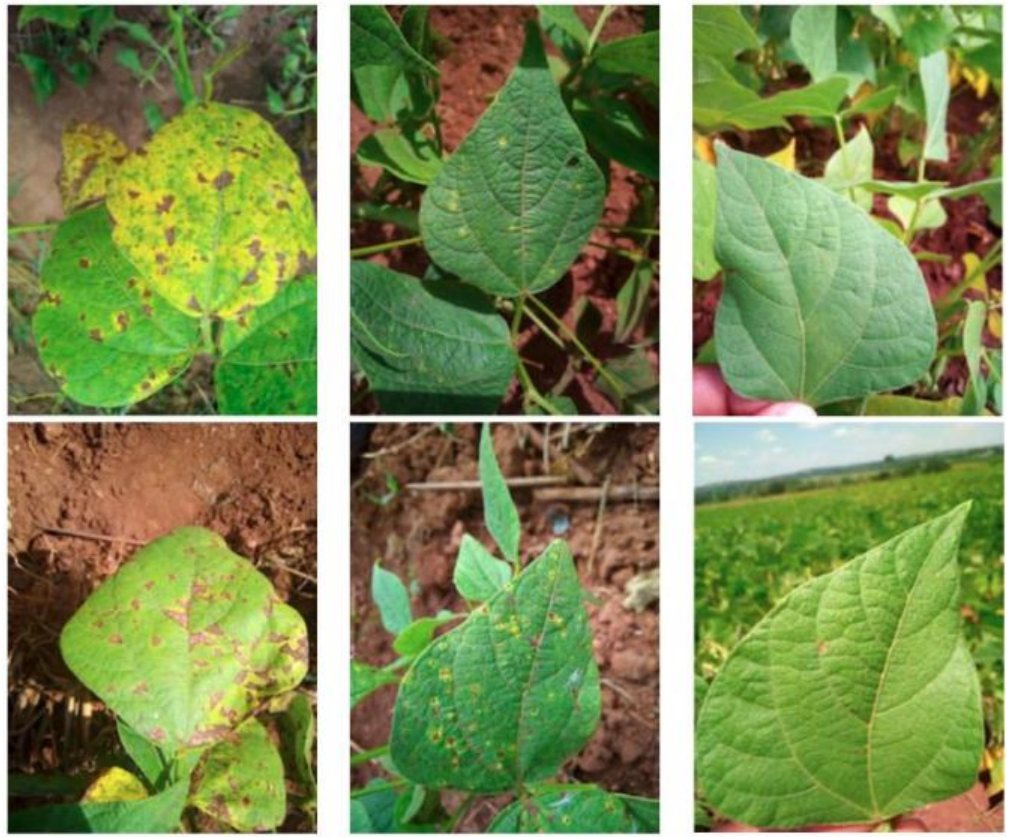
- Em geral, ***começamos com todas as camadas congeladas***, com ***exceção da camada densa de saída*** (i.e., a camada com função de ativação *softmax*).
- ***Treinamos*** o modelo por um número de épocas pequeno e ***verificamos seu desempenho***.
- Na sequência, ***descongelamos uma ou duas camadas*** ocultas do topo da rede e ***treinamos*** o modelo para ver se seu ***desempenho melhora***.
- Esse ***processo*** pode ser ***repetido até que não se obtenha mais melhorias*** no desempenho.
- ***Quanto mais dados de treinamento*** tivermos, ***mais camadas podemos descongelar***.
- Também é útil ***reduzir a taxa de aprendizado ao descongelar camadas***, pois isso evitará destruir os pesos já ajustados.

Exemplo: transfer_learning.ipynb

Atividades

- Exercício: Transfer learning.

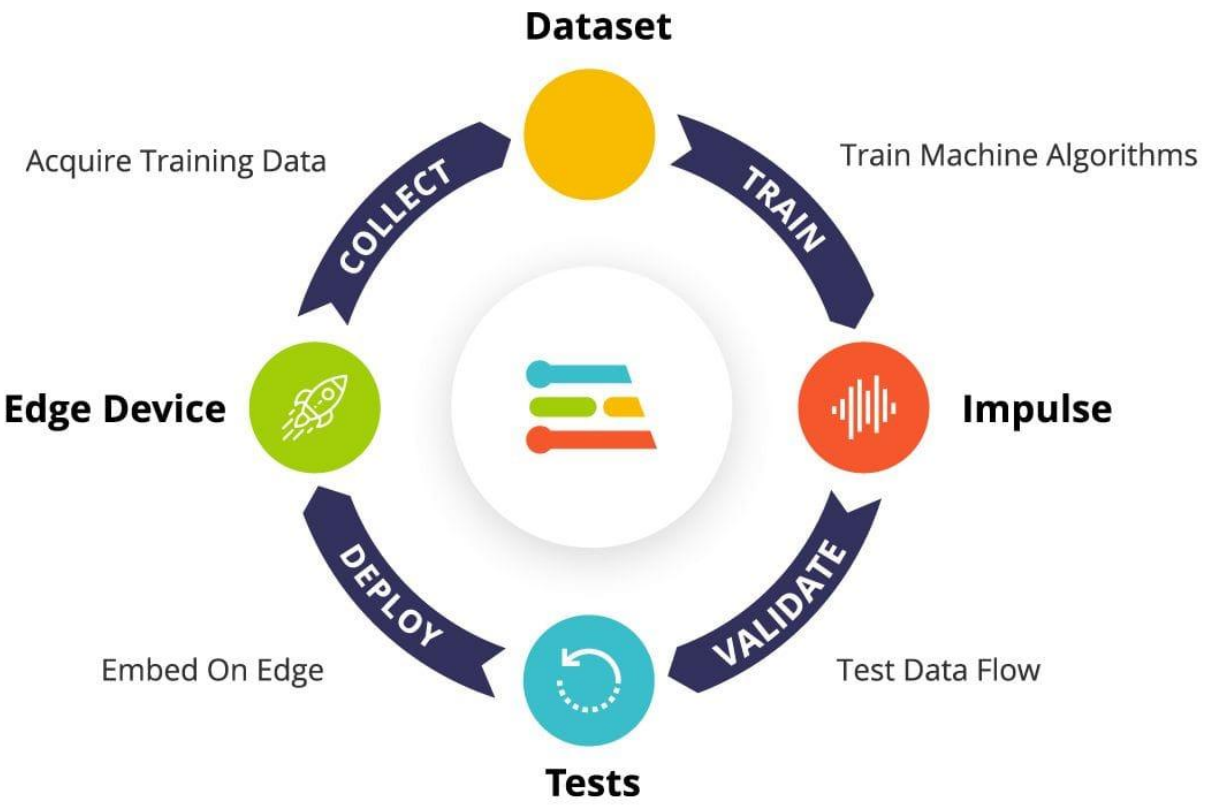
Exemplo: *transfer learning* com *Edge Impulse*



- Criaremos um projeto no *Edge Impulse* onde usaremos um modelo pré-treinado para treinar um modelo que diferencie (classifique) entre folhas saudáveis e doentes de feijão.
- Para esta tarefa, ***temos apenas*** 1296 imagens com 224x224 *pixels* dividas em três classes: *healthy*, *angular spot* e *rust*.

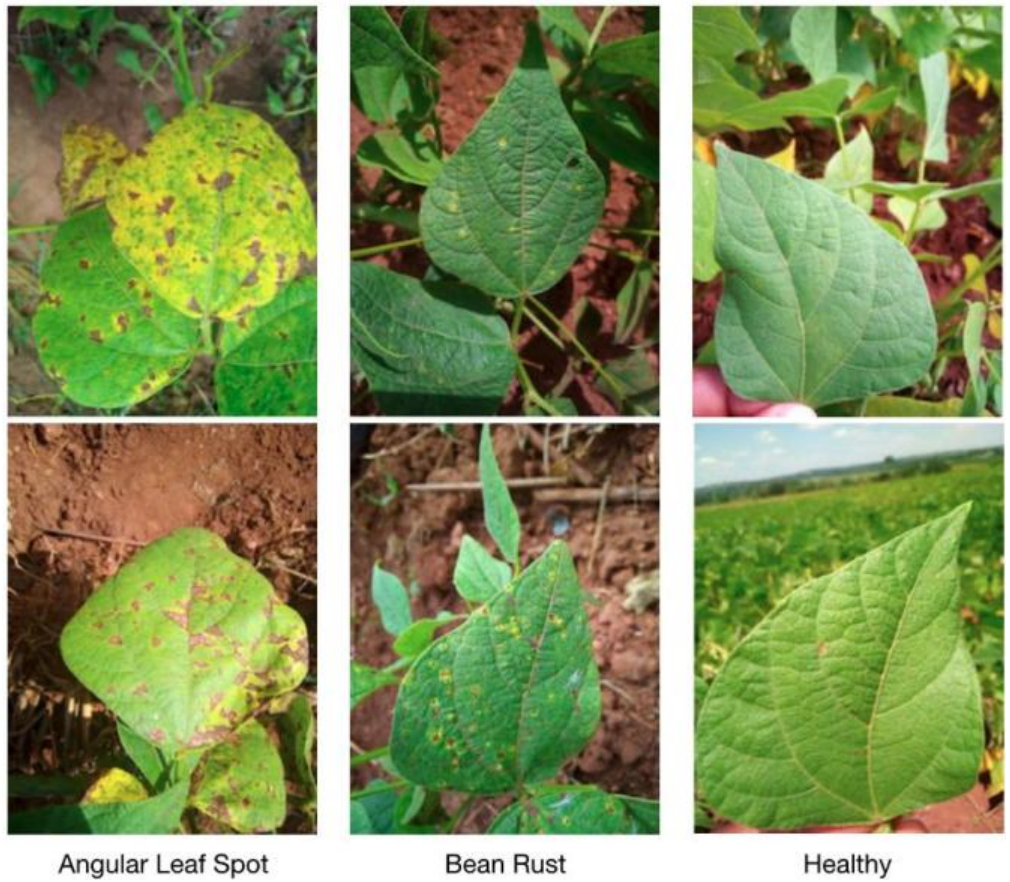
Classe	Número de exemplos
Healthy	428
Angular spot	432
Rust	436

Exemplo: *transfer learning* com *Edge Impulse*



- Edge Impulse é uma plataforma de desenvolvimento de ML focada em dispositivos de borda.
- Seu objetivo é simplificar a criação, o treinamento e a implantação de modelos de ML otimizados para operar diretamente nesses dispositivos.
- É uma ferramenta de AutoML: ***automatiza*** o desenvolvimento de modelos de ML, ***simplificando o processo*** e ***reduzindo a necessidade de conhecimento técnico avançado***.

Exemplo: *transfer learning* com *Edge Impulse*



- Edge Impulse: https://studio.edgeimpulse.com/login
- Base de dados: https://github.com/AI-Lab-Makerere/ibean/?tab=readme-ov-file

Atividades

- Tutorial sobre detecção de objetos: https://docs.edgeimpulse.com/docs/tutorials/end-to-end-tutorials/computer-vision/object-detection/object-detection
- Tutorial sobre classificação de imagens: https://docs.edgeimpulse.com/docs/tutorials/end-to-end-tutorials/computer-vision/image-classification

Obrigada!